



Data Management and Governance with Unity Catalog

Databricks Academy



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).



Course goals

- 1 Describe fundamental concepts about using Unity Catalog for data governance
- 2 Analyze the key components and architecture of Unity Catalog.
- 3 Secure access to external storage
- 4 Evaluate the Unity Catalog security model and Databricks roles to implement robust data governance strategies.
- 5 Create and implement data administration fundamentals.



Agenda

Course Sections	
1. Data Governance Overview	8. Privileges in Unity Catalog
2. Demo: Populating the Metastore	9. Fine-Grained Access Control
3. Introduction to Continuous Deployment (CD)	10. Demo: Controlling Access to Data
4. Organization and Access Patterns	11. Lab: Securing Data in Unity Catalog
5. Demo: Upgrading Tables to Unity Catalog	12. Lakehouse Monitoring
6. Security and Administration in Unity Catalog	13. Demo: (Bonus) Lakehouse Monitoring
7. Databricks Marketplace Overview	





Data Governance Overview

**Data Management and Governance with Unity
Catalog**



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).



80% of organizations seeking to scale digital business **will fail** because they do not take a modern approach to **data and analytics governance**

Source: [Gartner](#)



Data Governance Overview

Four Key Functional Areas

Data Access Control

Control who has access to which data

Data Access Audit

Capture and record all access to data

Data Lineage

Capture upstream sources and downstream consumers

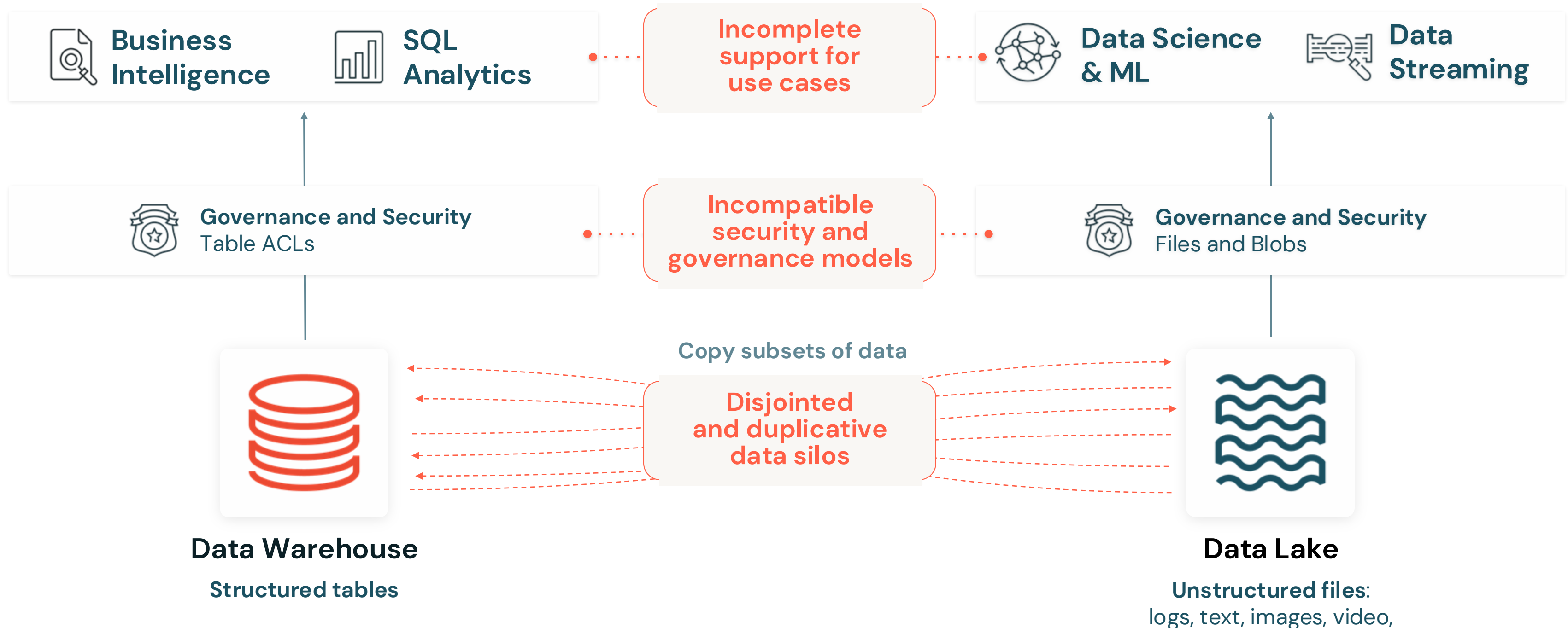
Data Discovery

Ability to search for and discover authorized assets

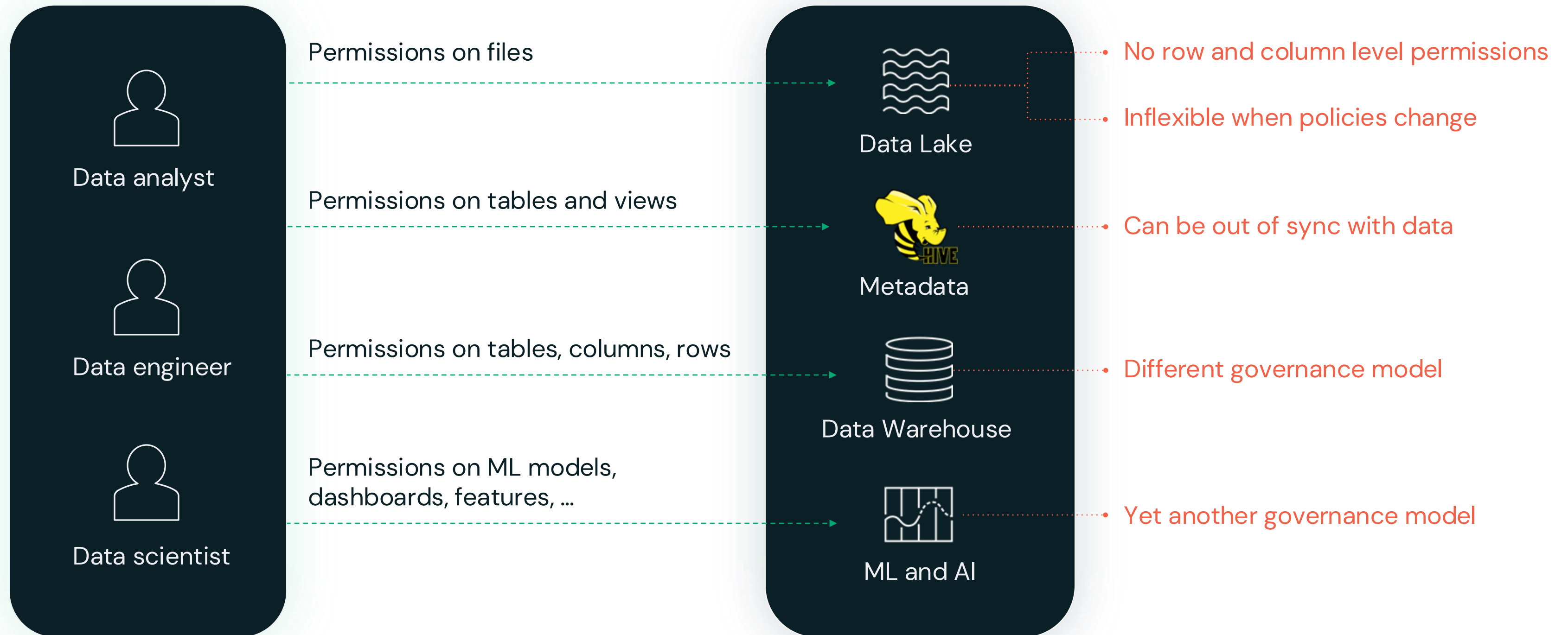


Data Governance Overview

Challenges in the Data Lake



Governance for Data, Analytics, and AI Is Complex



Databricks Unity Catalog

Unified Governance for Data, Analytics and AI



Databricks Unity Catalog

Overview



Unify Governance Across Clouds

Fine-grained governance for data lakes across clouds – based on open standard ANSI SQL.



Unify Data and AI Assets

Centrally share, audit, secure and manage all data types with one simple interface.



Unify Existing Catalogs

Works in concert with existing data, storage, and catalogs – no hard migration required.



Unity Catalog Open Source



Open

Open APIs and OSS server maximize flexibility and customer choice



Interoperable

Universal interface supports any format, engine, data and AI asset



Unified

Unified governance across tabular, non-tabular data and AI assets



Unity Catalog Open Source

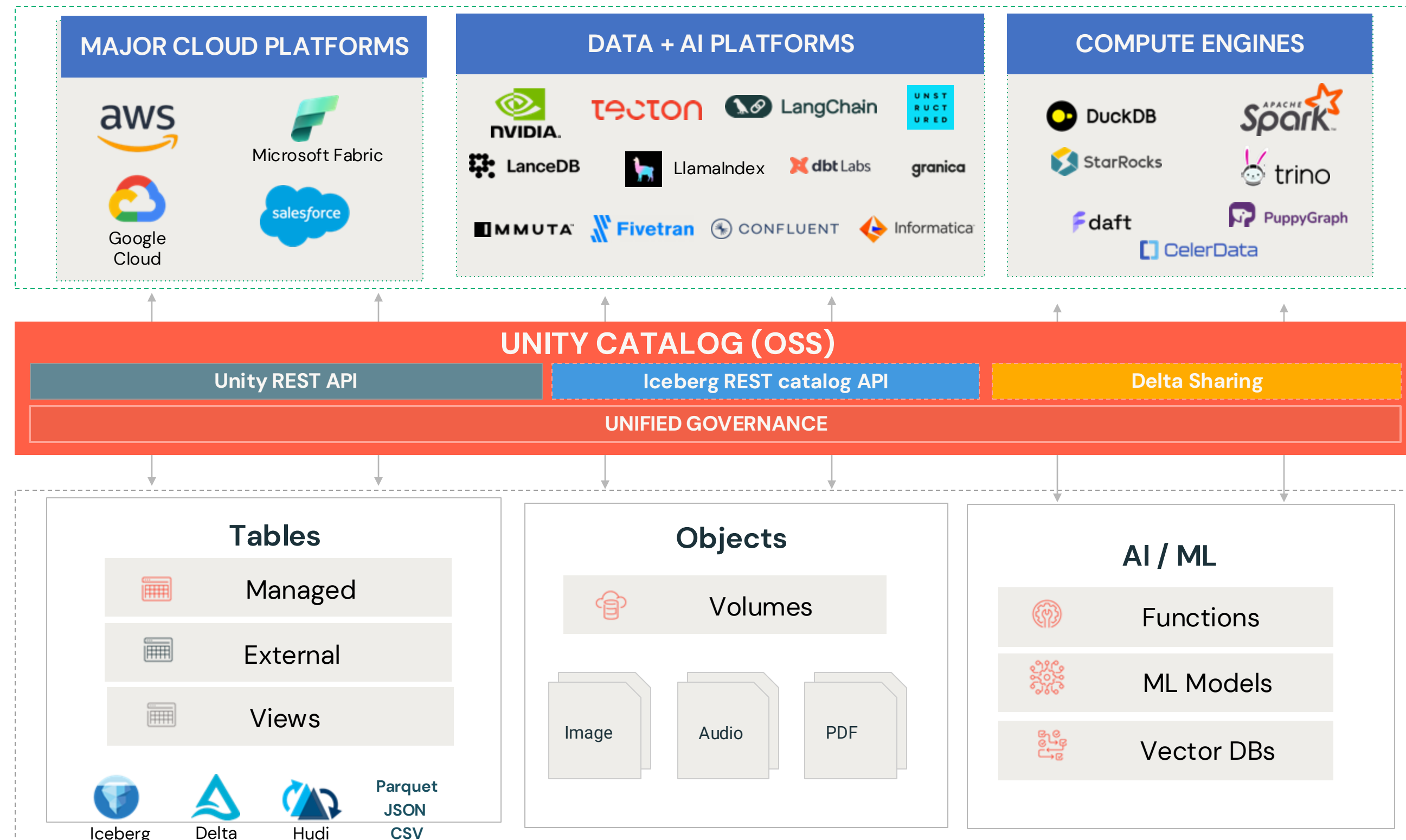
The industry's only
universal catalog for
Data and AI

Any Engine – Client Ecosystem

Any Client – Universal Standard

Any Asset – Data + AI

Any Format – UniForm



Unity Catalog Open Source



OpenAPI spec

Managed tables APIs

External tables APIs

Volumes APIs

Functions APIs

Credential vending APIs

Iceberg REST Catalog APIs



OSS server

OSS server

Available in new Unity
Catalog Github repo



New developer resources

Unity Catalog OSS SDK

REST API docs

Updated Databricks SDKs
(Java, Python, Go)



Secure credential vending

Unity Catalog secure
credential vending

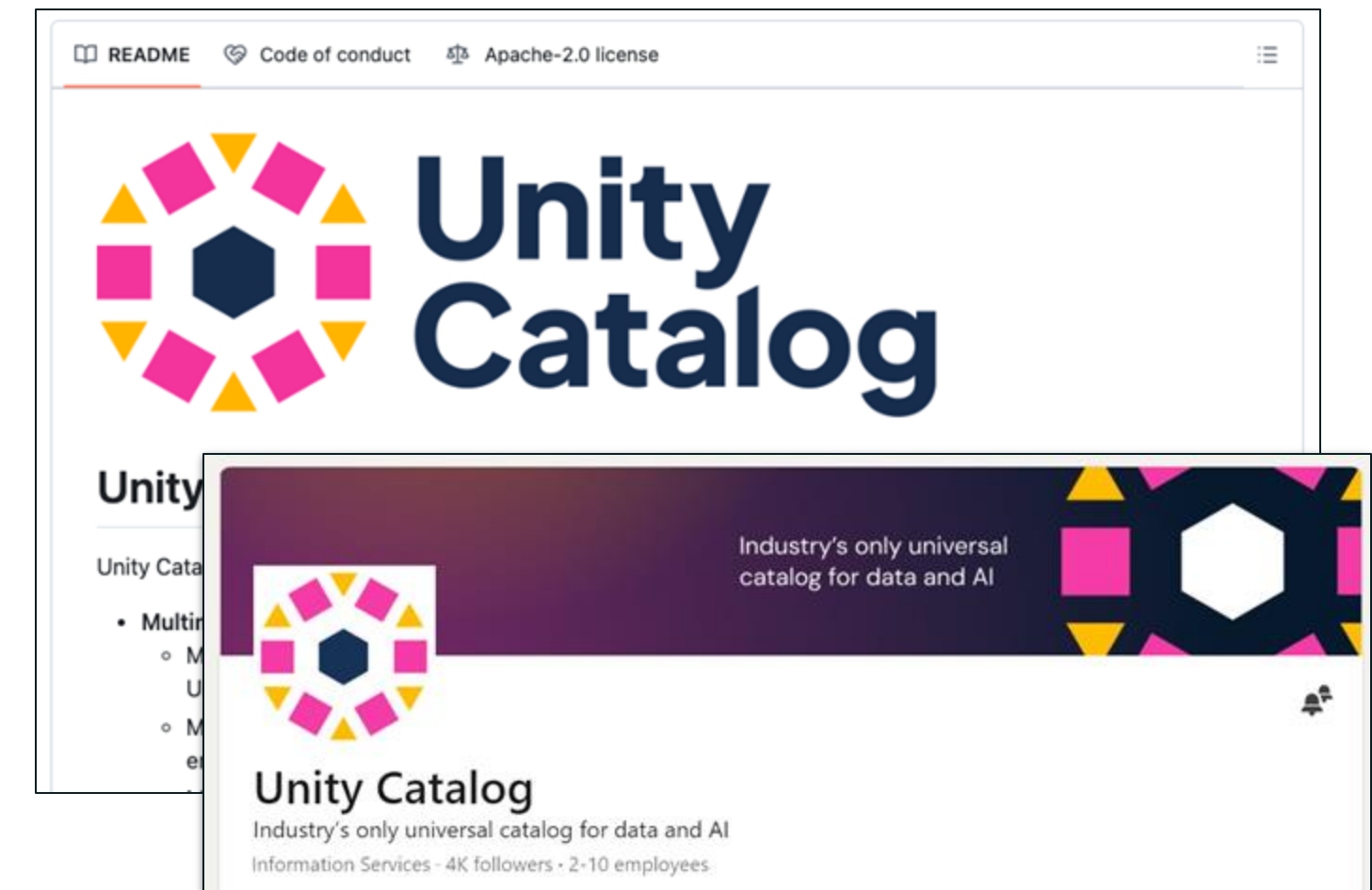
Available on Databricks in
Private Preview



Join the Open Ecosystem

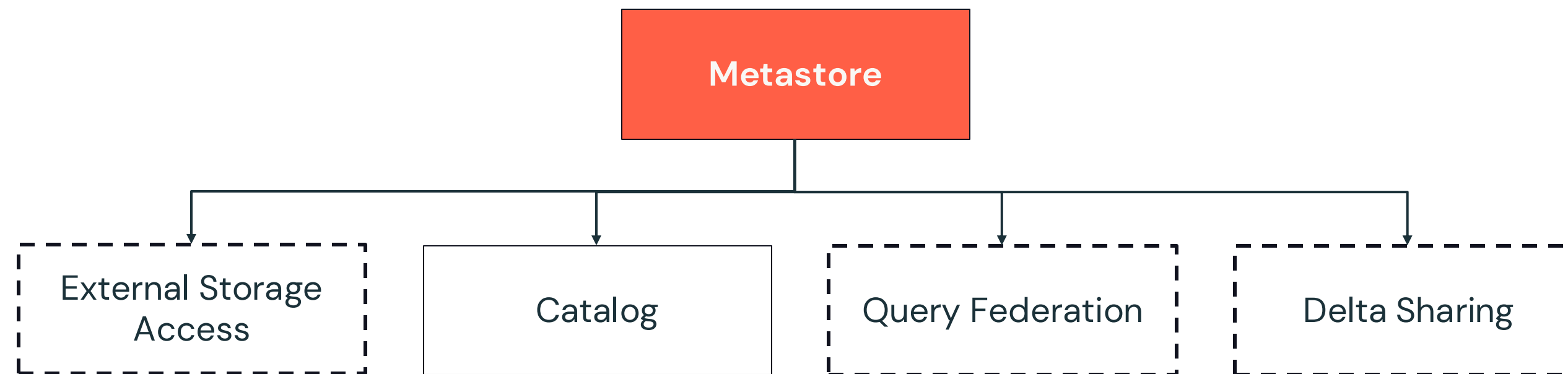
Learn more from the Unity Catalog website, GitHub Repo + LinkedIn

- Learn more on the website!
 - unitycatalog.io
- Follow on LinkedIn for the latest updates!
 - <https://www.linkedin.com/company/unitycatalog/>
- Participate in GitHub Discussions!
 - <https://github.com/unitycatalog/unitycatalog>
- Contact your Databricks Account team for access to the External Table Access Preview!



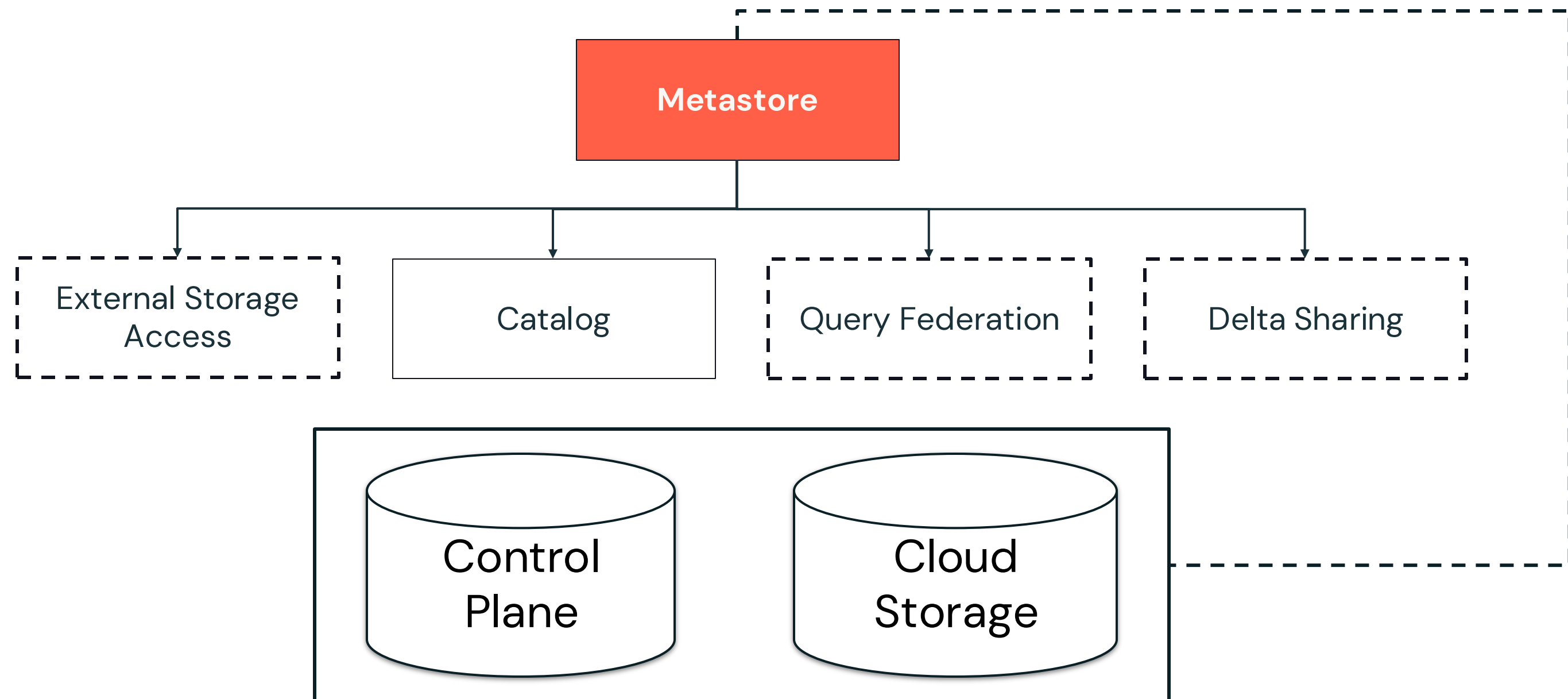
Key Concepts

Metastore



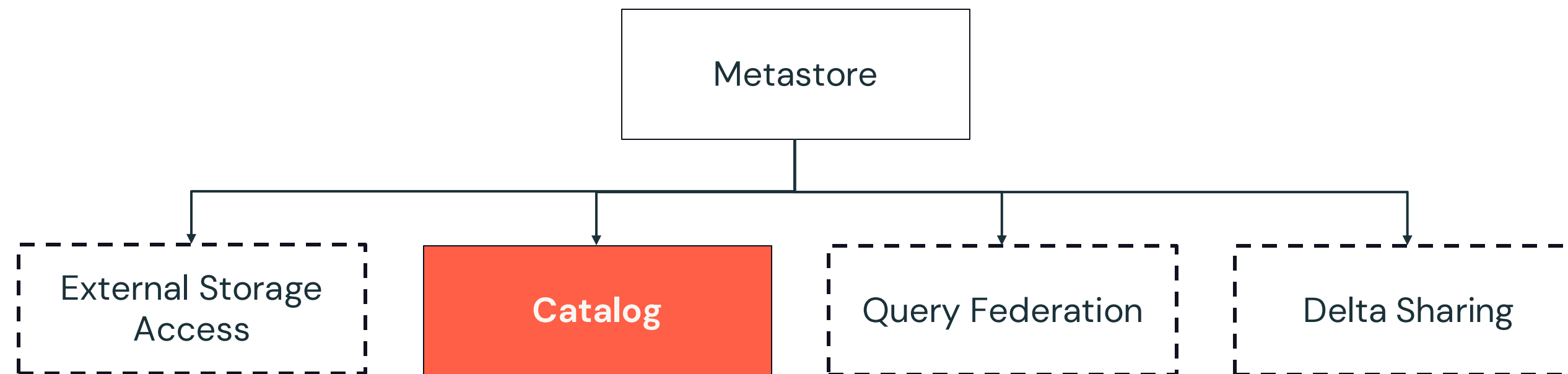
Key Concepts

Metastore Elements



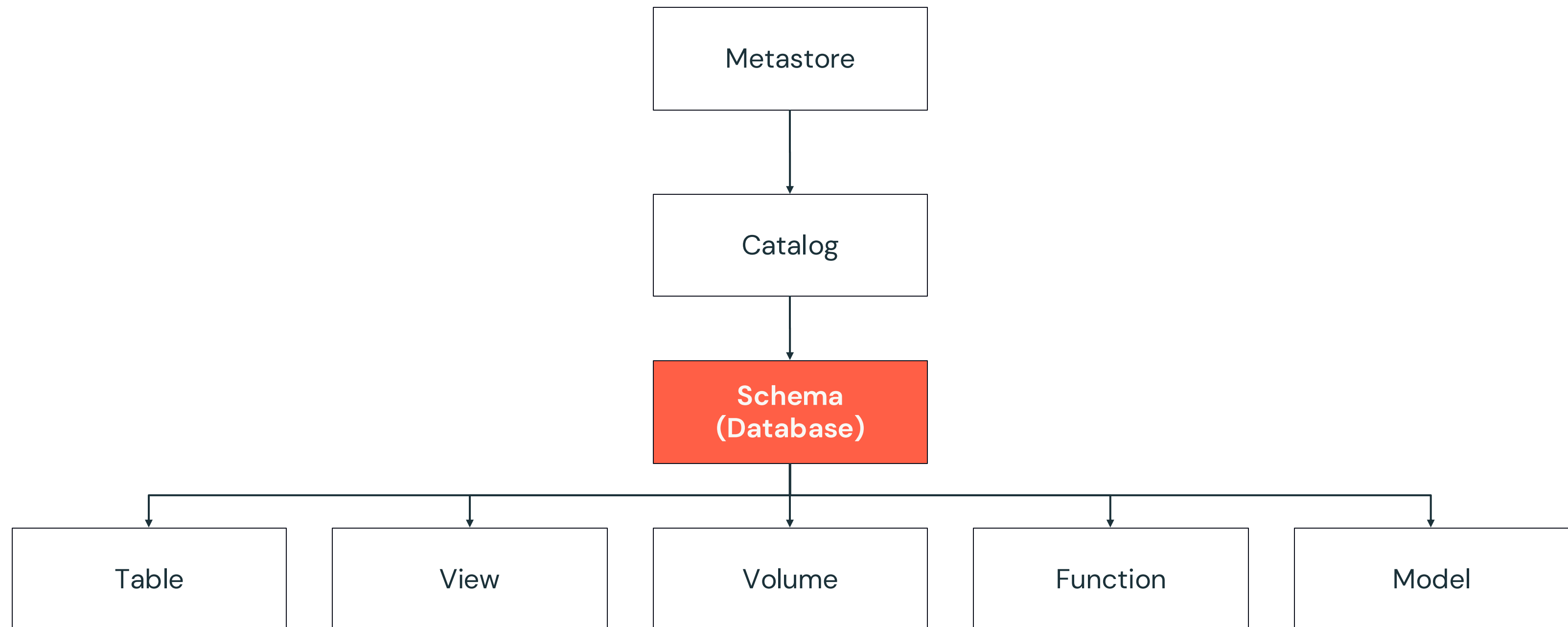
Key Concepts

Catalogs



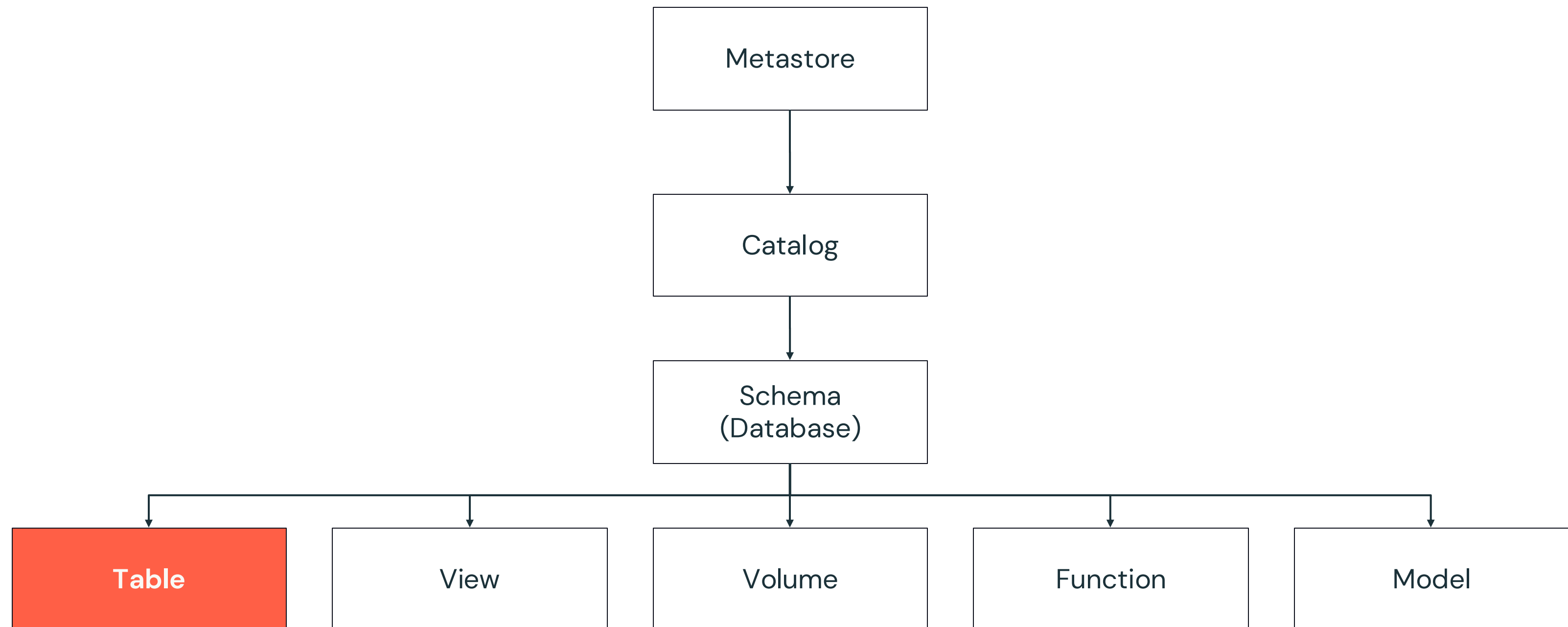
Key Concepts

Schemas/Databases



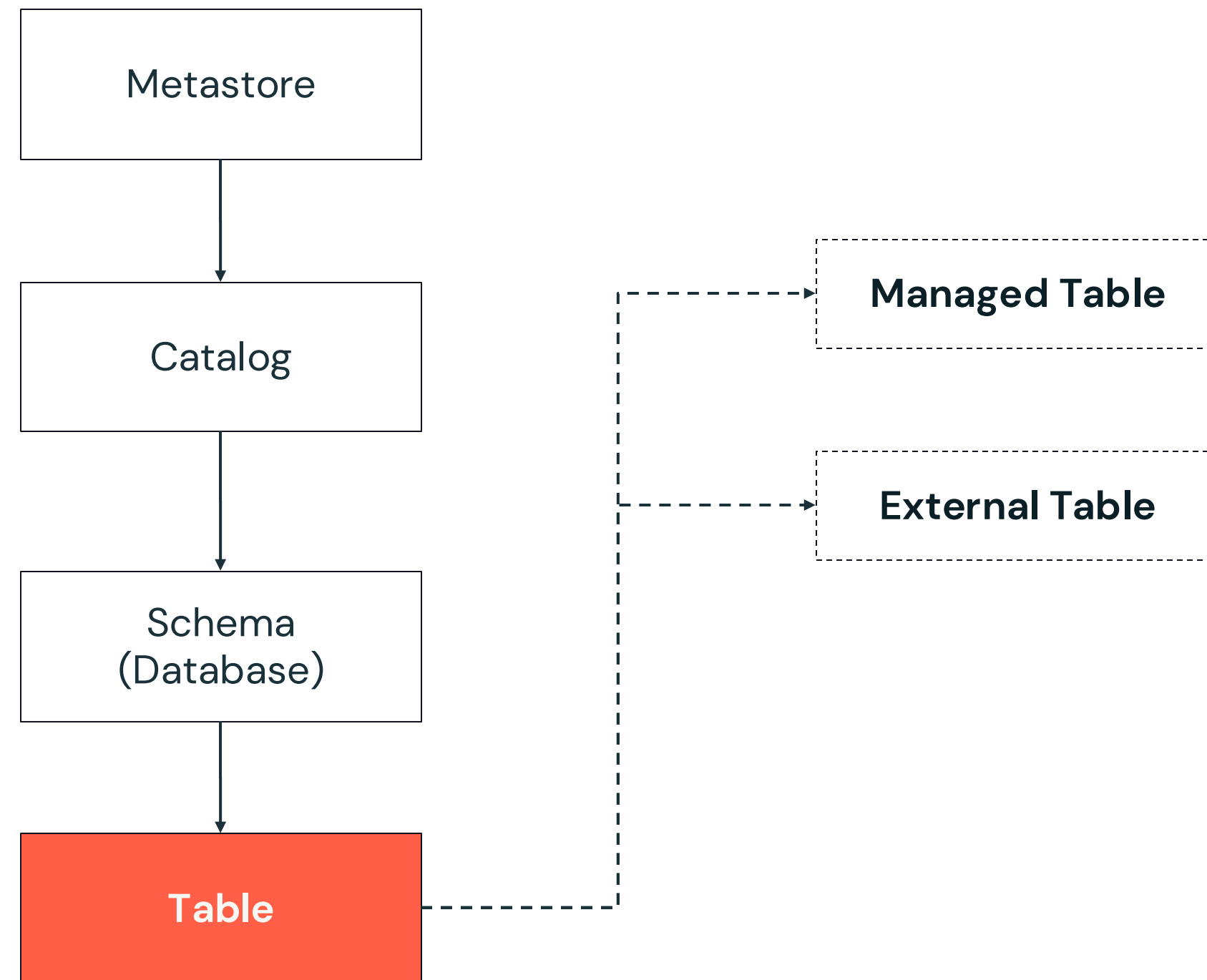
Key Concepts

Tables



Key Concepts

Managed vs. External Tables



Comparison of Managed and Unmanaged UC Tables

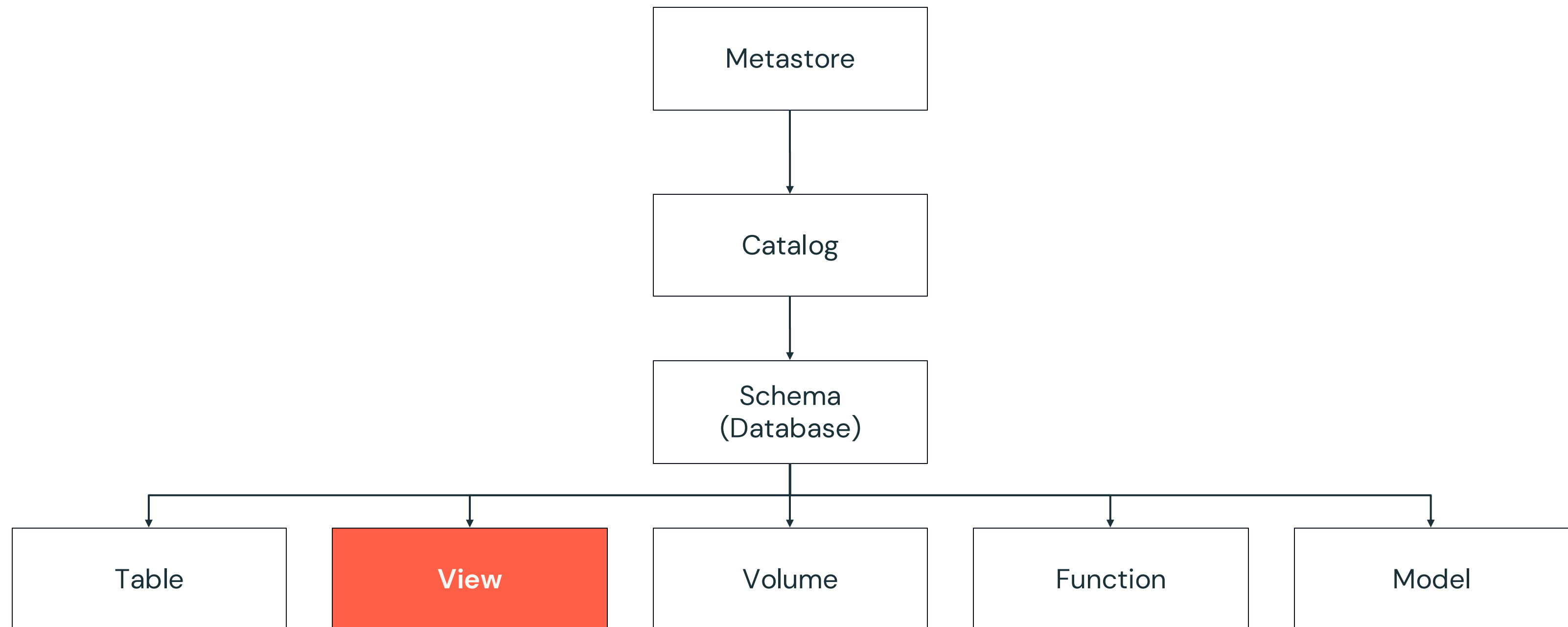
Consider the Benefits of Managed Tables

Characteristic	Managed	Unmanaged (a.k.a. "External")
Table's Type Property Value	"MANAGED"	"EXTERNAL"
DROP Table Behavior	Discards metadata and deletes the associated data • UNDROP exists	Discards metadata only. Does not delete the data • If data needs to be deleted, must be done separately
Create Table Syntax	CREATE TABLE [<catalog>.[<schema>.]<table> ...	CREATE TABLE [<catalog>.[<schema>.]<table> ... LOCATION 'abfss:/cont@stacct.dfs.core.windows.net';
Data File Location	Whichever managed location has been specified and is found first: schema, catalog, metastore.	The path specified by the LOCATION keyword
Performance Optimizations	Auto Tune (Predictive Optimization, more coming)	Manually managed
Data Format Support	DELTA	DELTA, CSV, JSON, AVRO, PARQUET, ORC, TEXT
Pros	New features, performance, simplicity, stricter access	Non-delta tables, enforce storage naming, better compatibility with external readers/writers, simpler migration



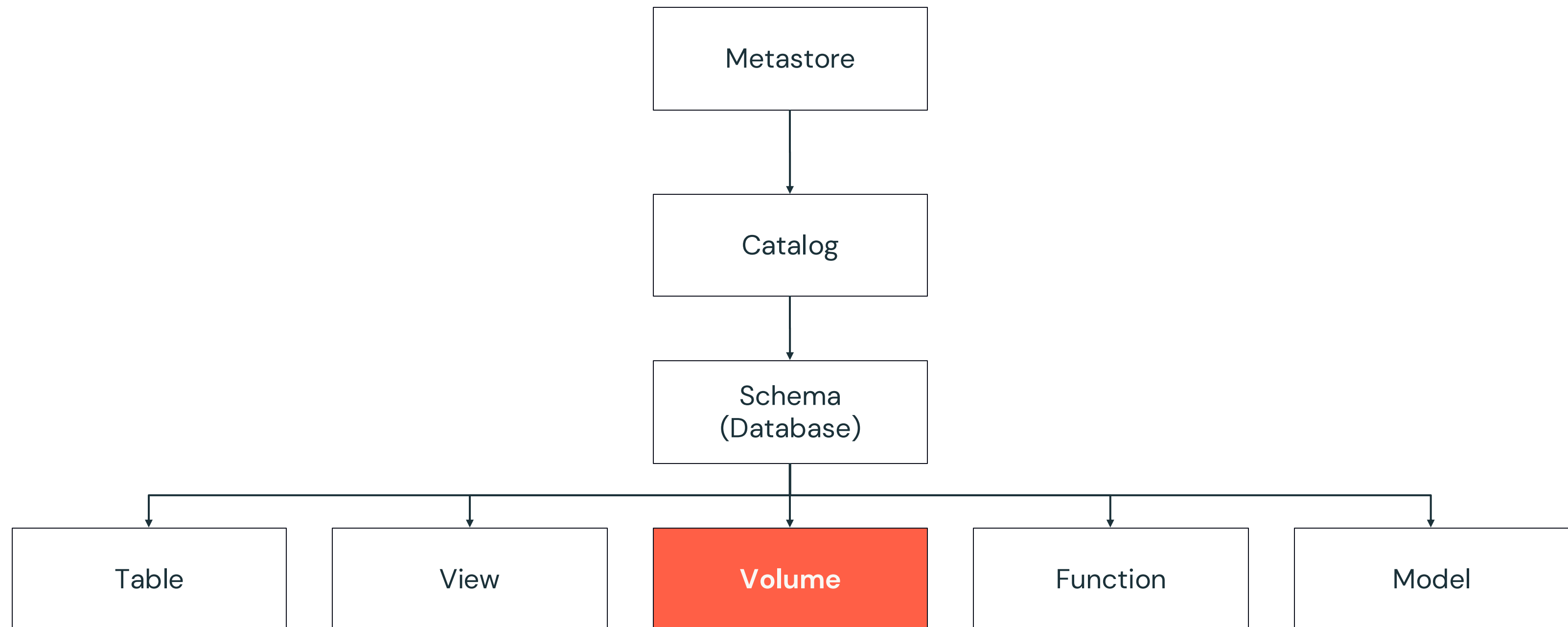
Key Concepts

Views



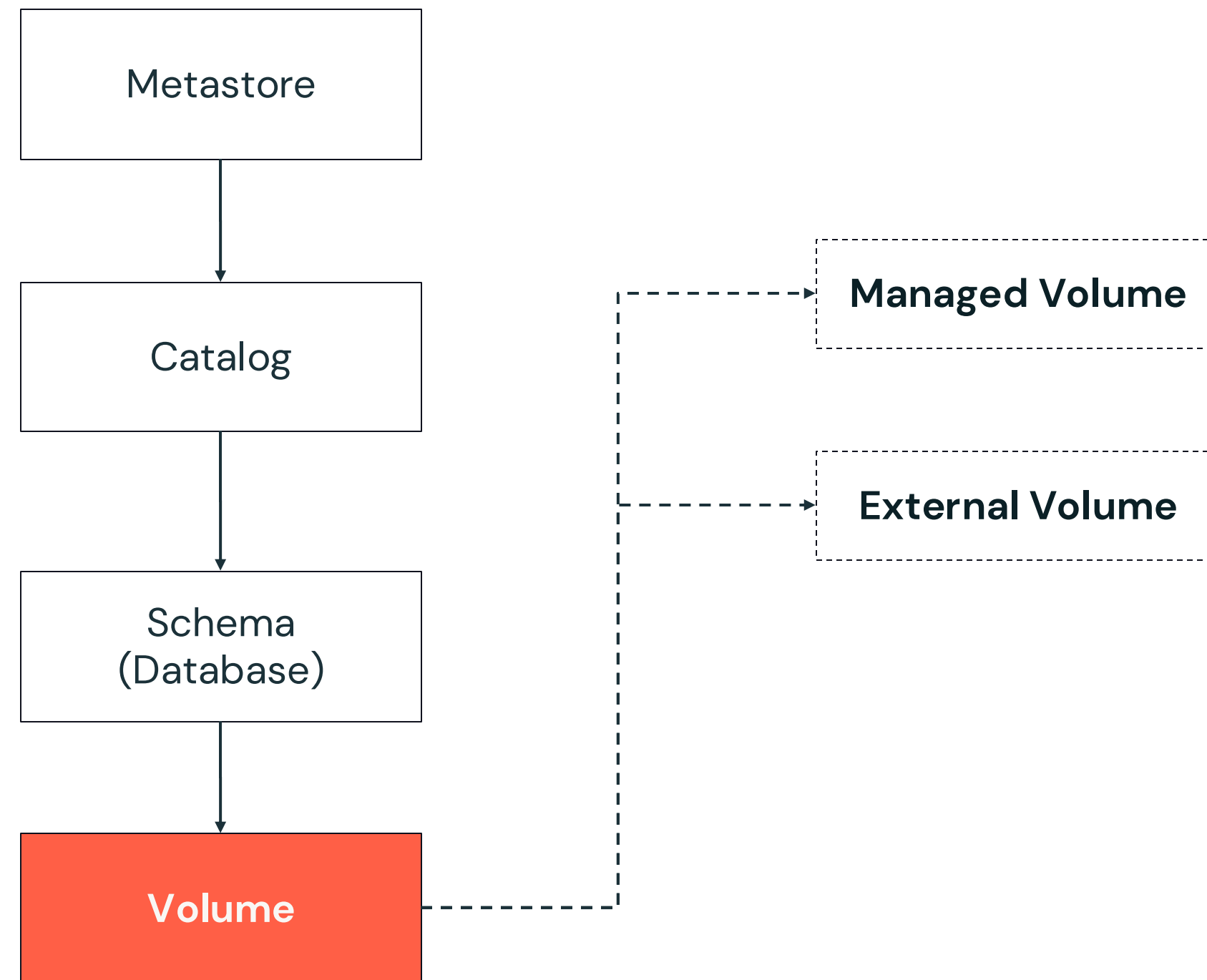
Key Concepts

Volumes



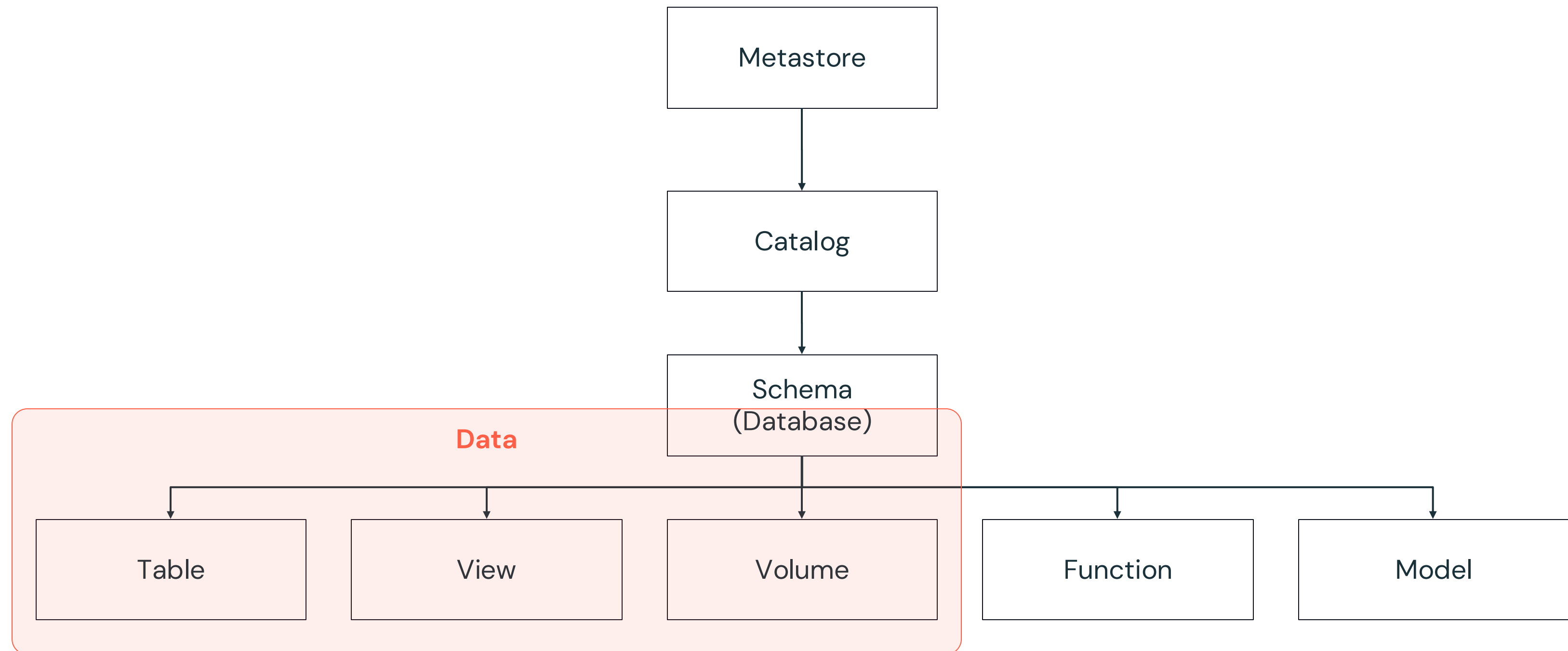
Key Concepts

Managed vs. External Volumes



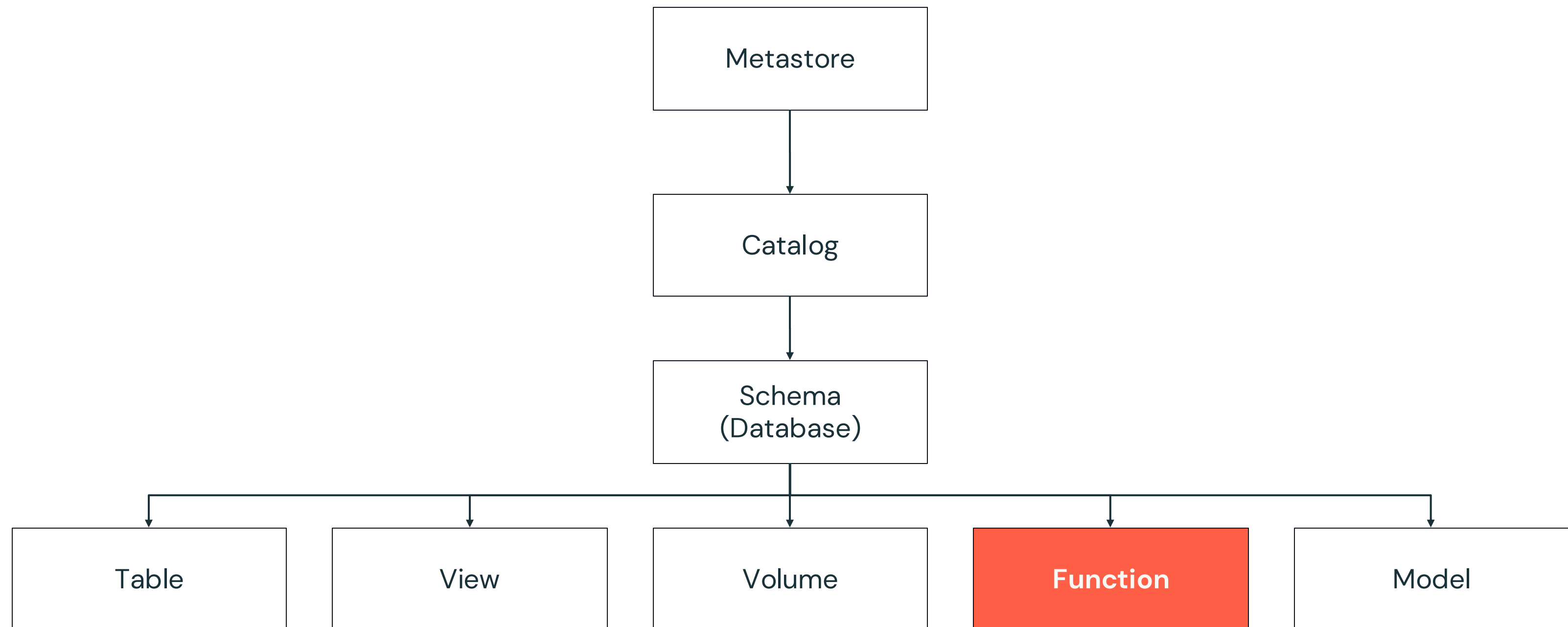
Key Concepts

Volumes



Key Concepts

User-Defined Functions



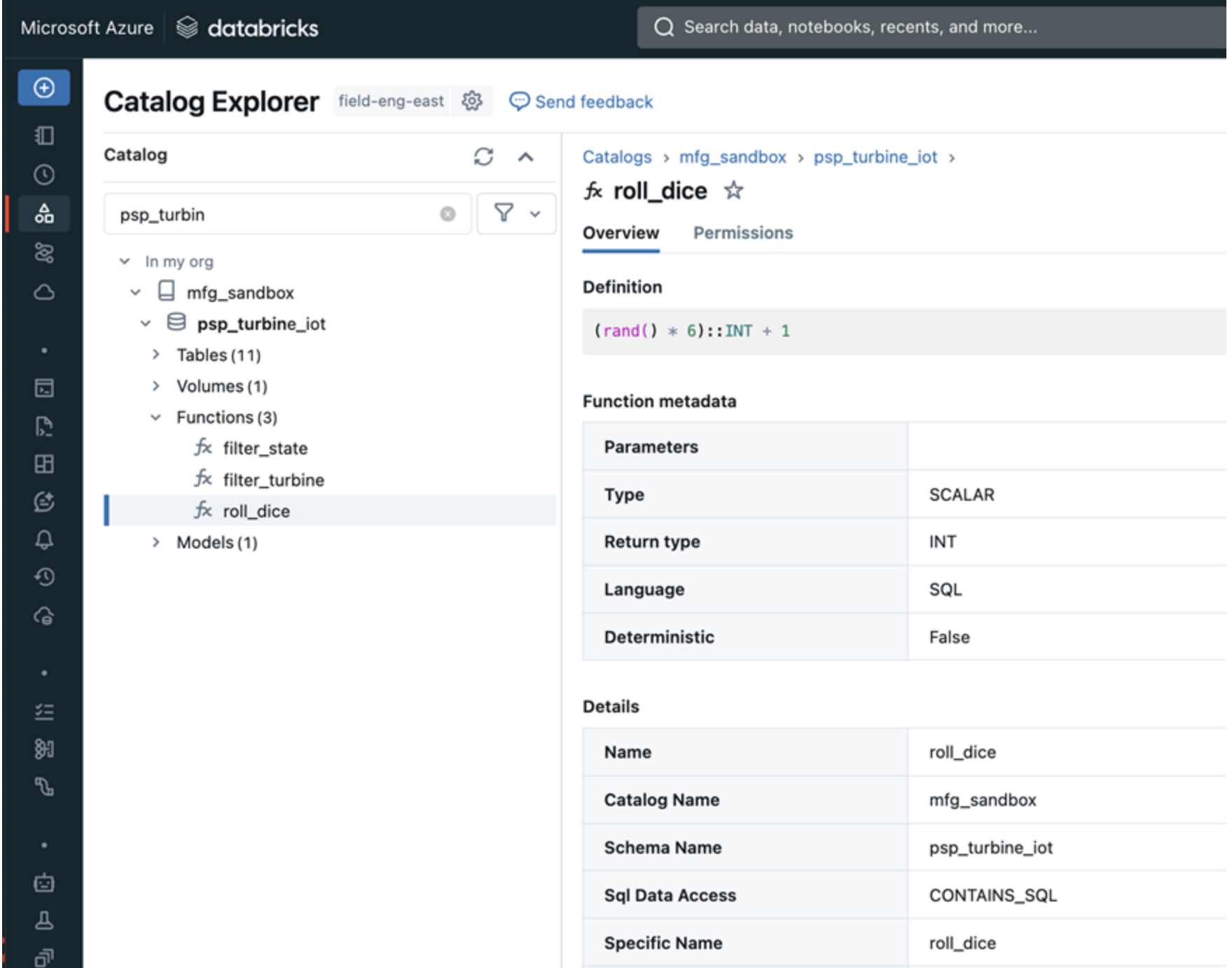
Functions

Custom Functions Governed by Unity Catalog

Use **SQL-native syntax** to register custom functions to schemas

CREATE FUNCTION privilege on the Schema required.

Python UDFs are designed to provide the full expressiveness of Python directly within SQL functions, allowing for customized operations such as advanced transformations, data masking, and hashing.



The screenshot displays the Databricks Catalog Explorer interface. The left sidebar shows a tree view of the catalog structure: 'Catalog' > 'In my org' > 'mfg_sandbox' > 'psp_turbine_iot' > 'Functions (3)'. The 'roll_dice' function is selected. The main panel shows the function's details:

- Overview** tab is active.
- Definition:** `(rand() * 6)::INT + 1`
- Function metadata table:**

Parameters	
Type	SCALAR
Return type	INT
Language	SQL
Deterministic	False

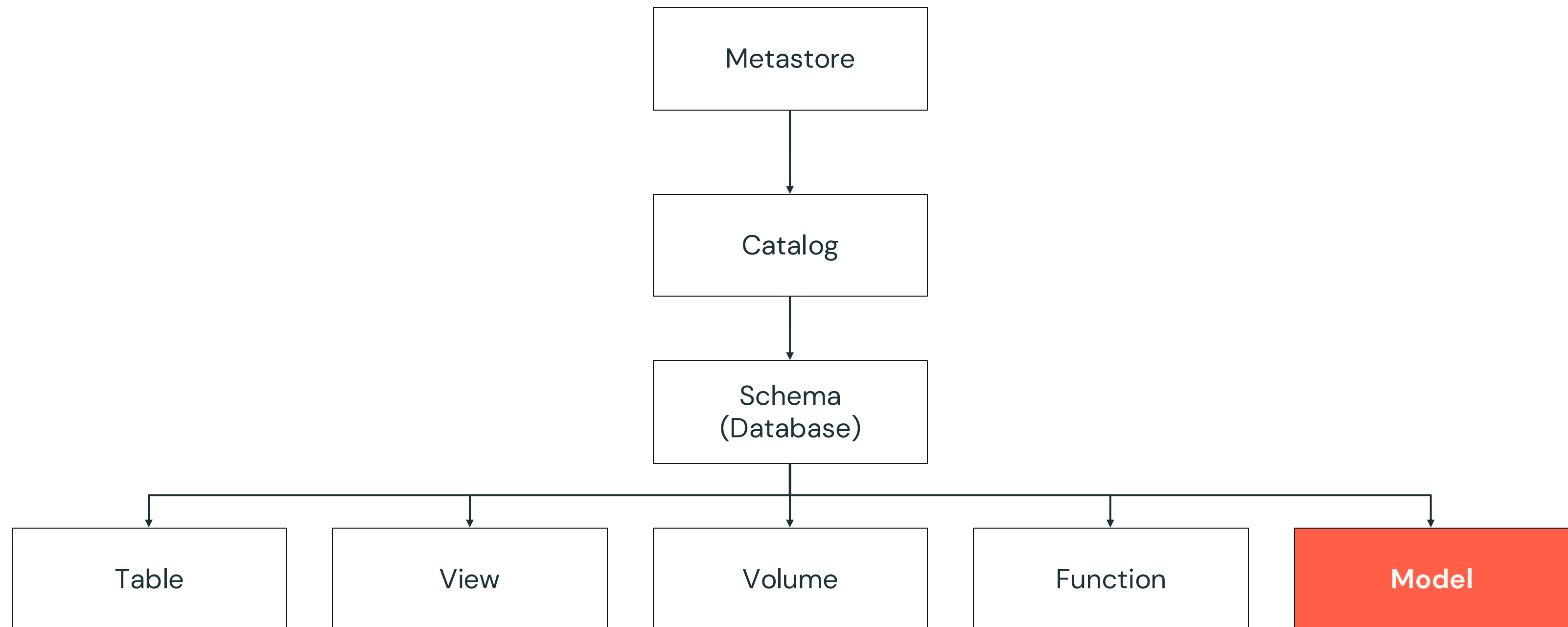
- Details table:**

Name	roll_dice
Catalog Name	mfg_sandbox
Schema Name	psp_turbine_iot
Sql Data Access	CONTAINS_SQL
Specific Name	roll_dice



Key Concepts

Models



Models

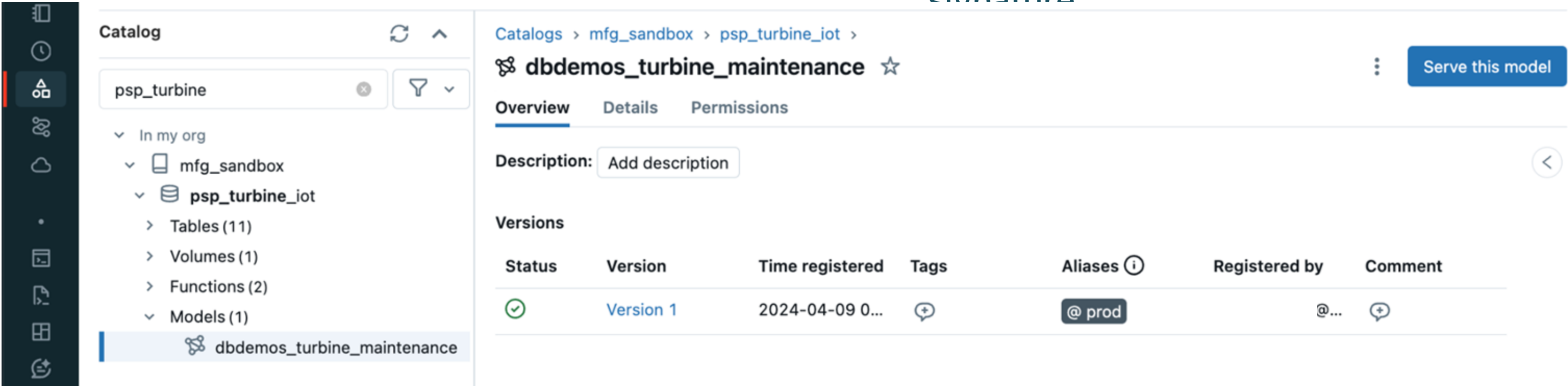
MLFlow Model Registry in Unity Catalog

Discover, manage, organize and execute **models across workspaces** with Unity Catalog governance

CREATE MODEL privilege on the Schema required to register a new model.

Owners of the model are able to create new versions of registered models.

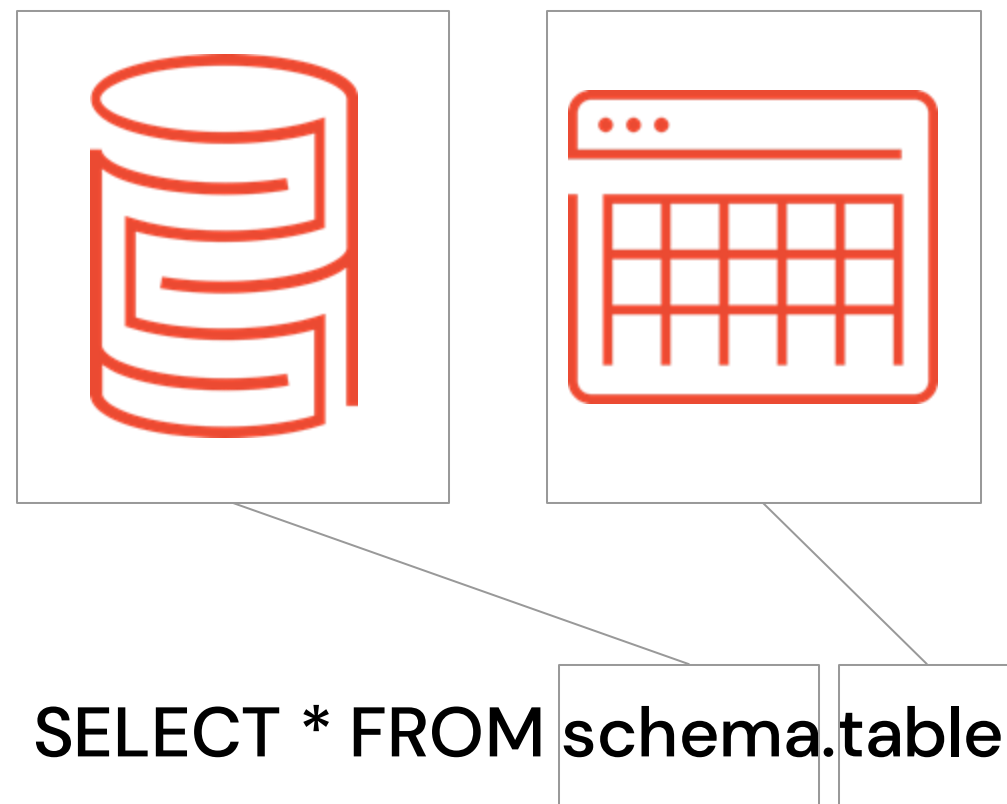
ML model versions in UC must have a model signature



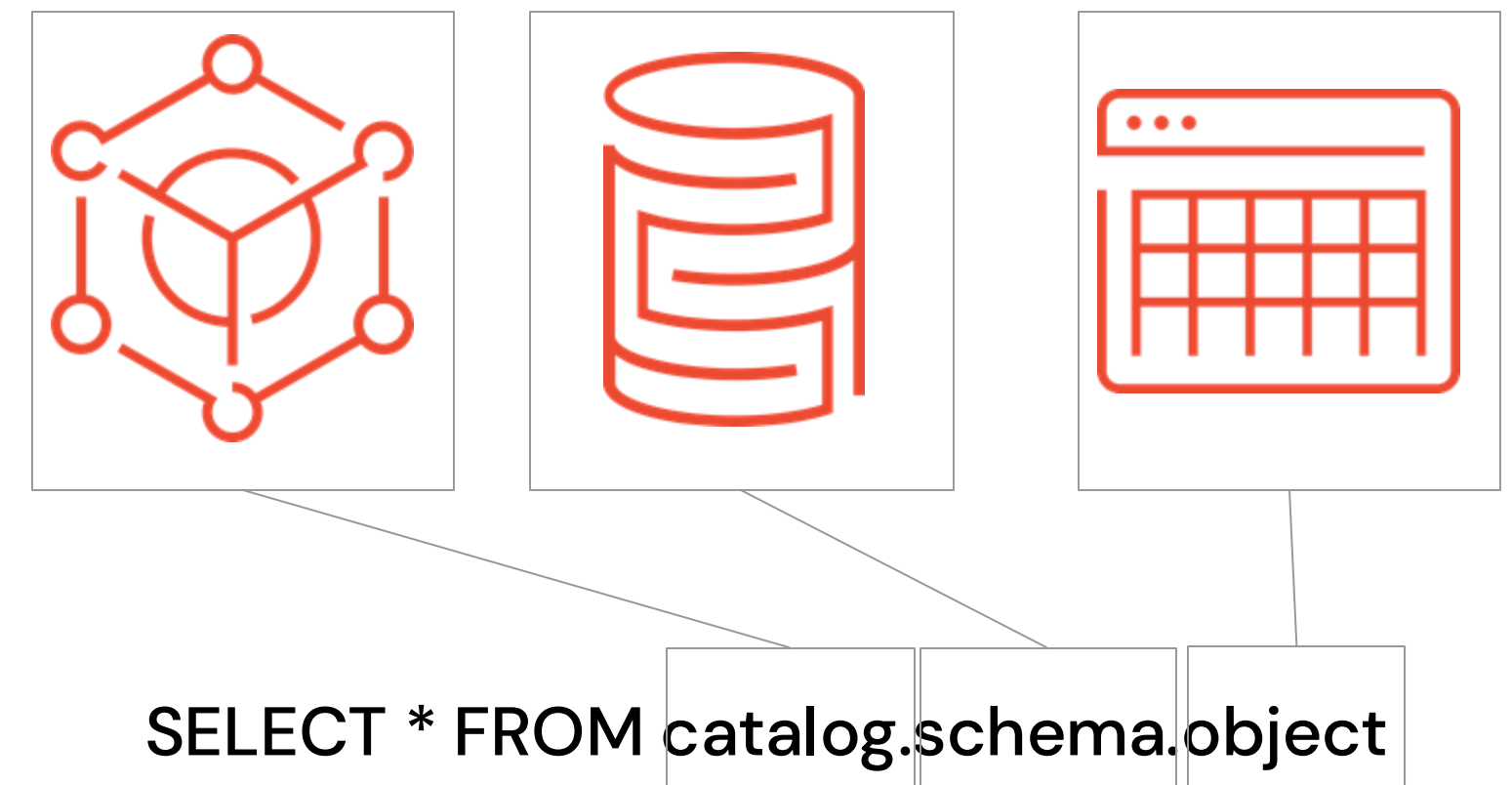
Key Concepts

Three-Level Namespace

Traditional SQL Two-Level Namespace

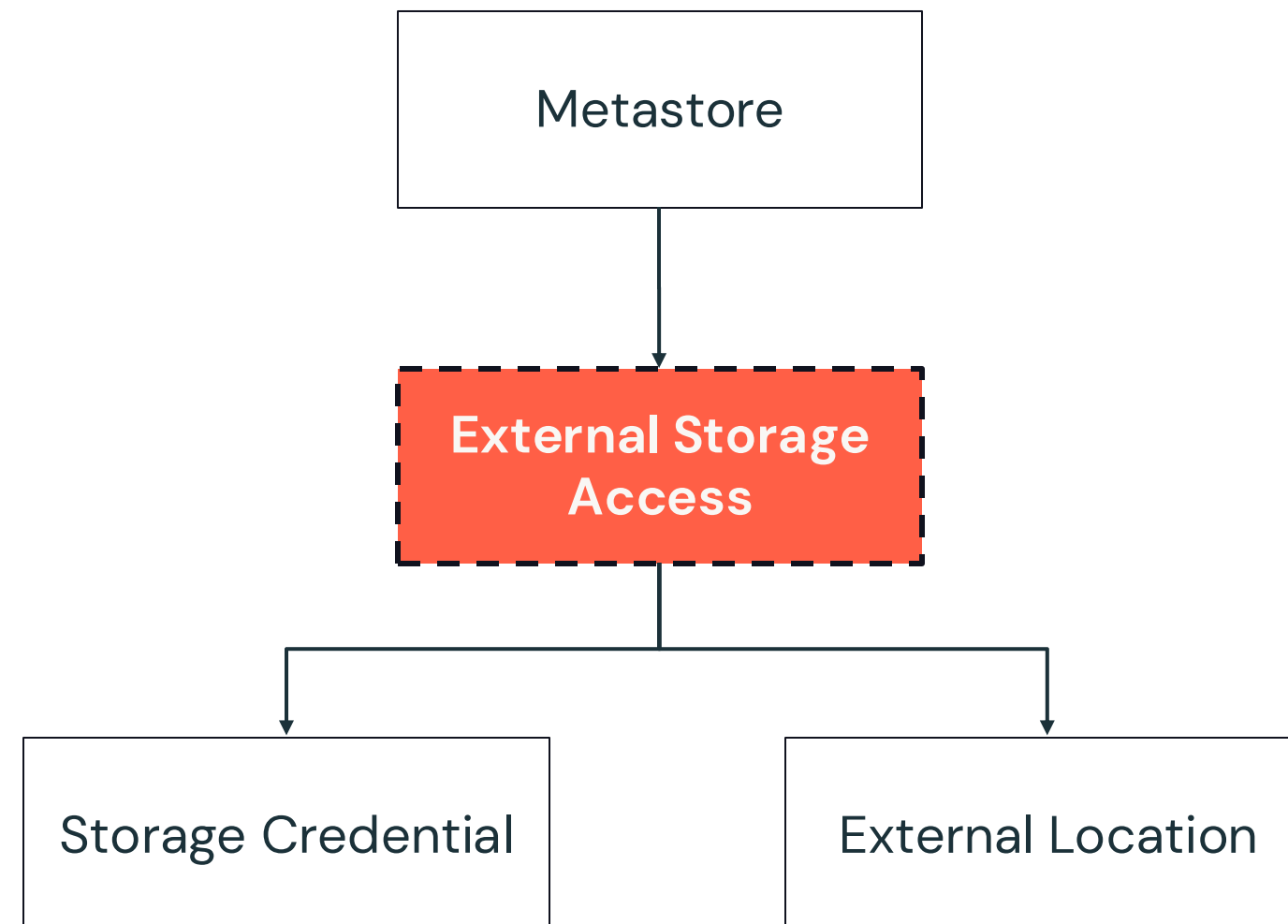


Unity Catalog Three-Level Namespace



Key Concepts

External Storage Access



Key Concepts

Lakehouse Federation



Lakehouse Federation

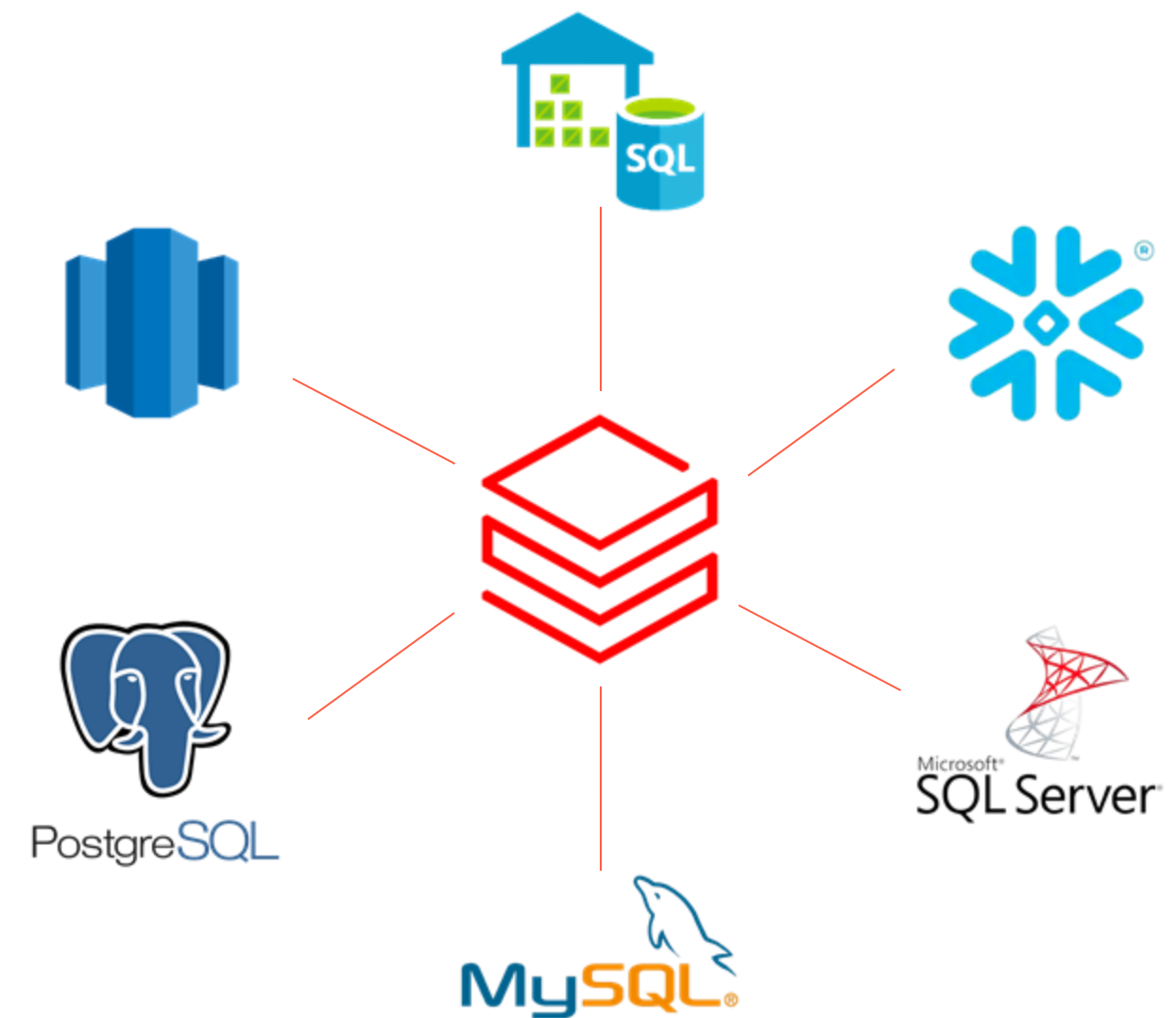
Discover, query, and govern all your data – no matter where it lives

Lakehouse Federation provides one **single point of secure access** to all your data with unified permission controls and intelligent **pushdown optimizations**.

Supported sources GA: MySQL, PostgreSQL, Redshift, Snowflake, SQL Server, Synapse, Databricks

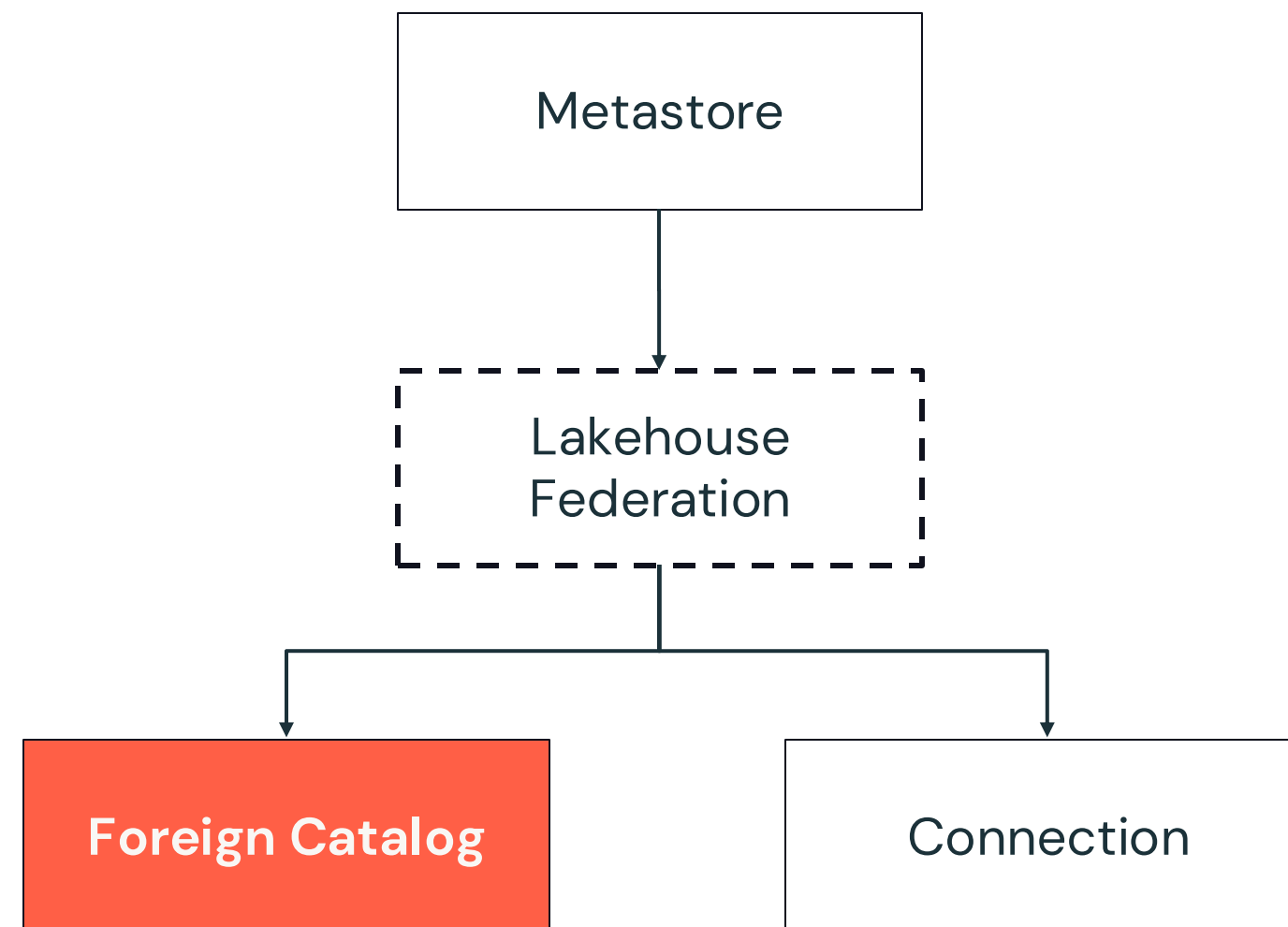
Supported sources Preview: BigQuery, Hive, AWS Glue

In roadmap: Teradata, Oracle, SFDC



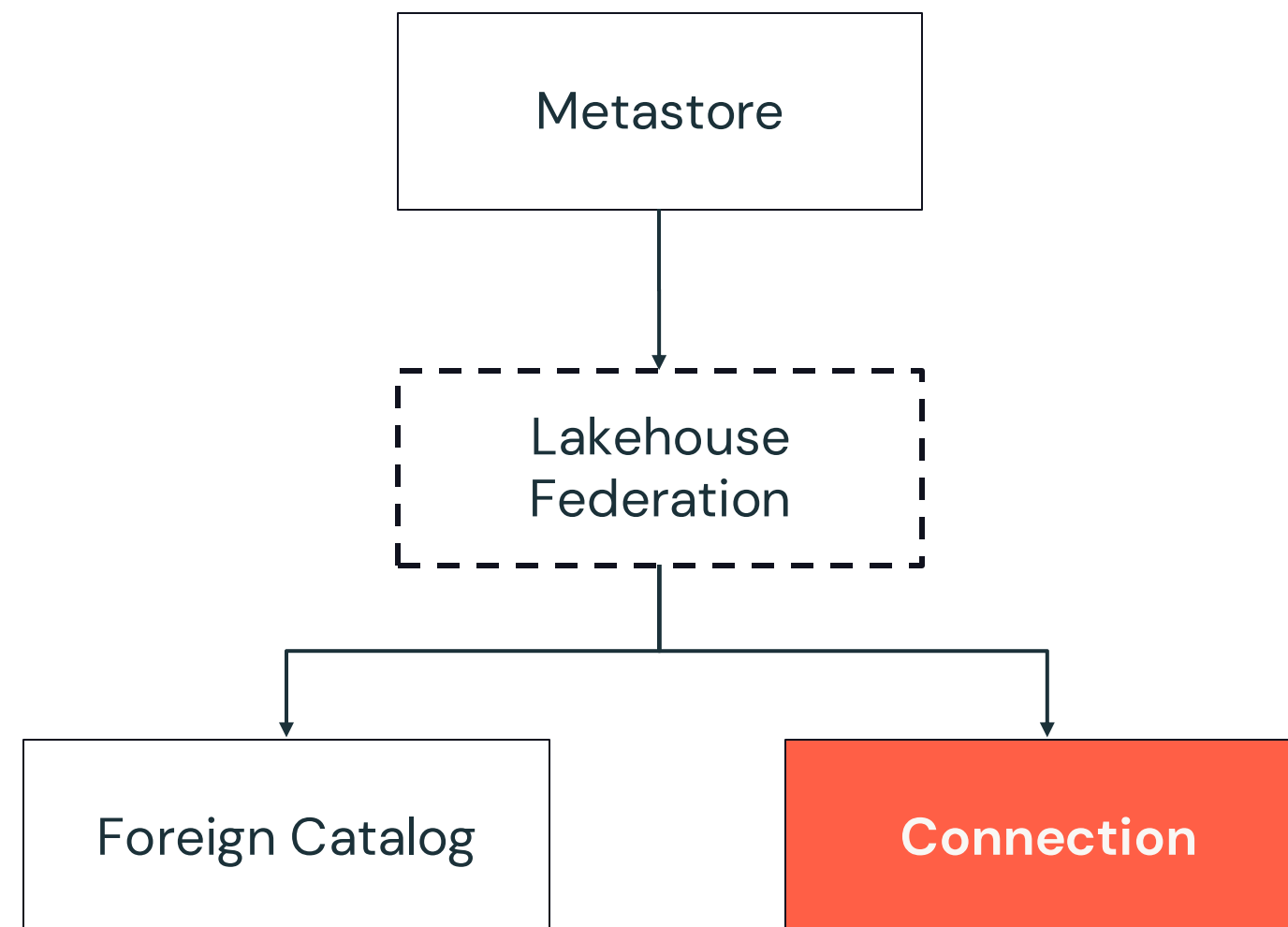
Key Concepts

Query Federation



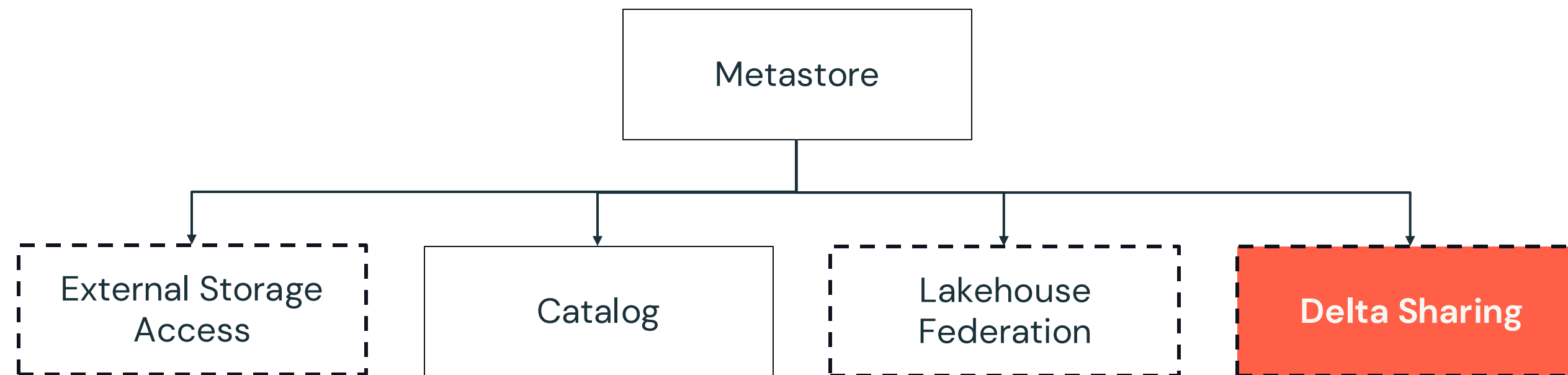
Key Concepts

Lakehouse Federation



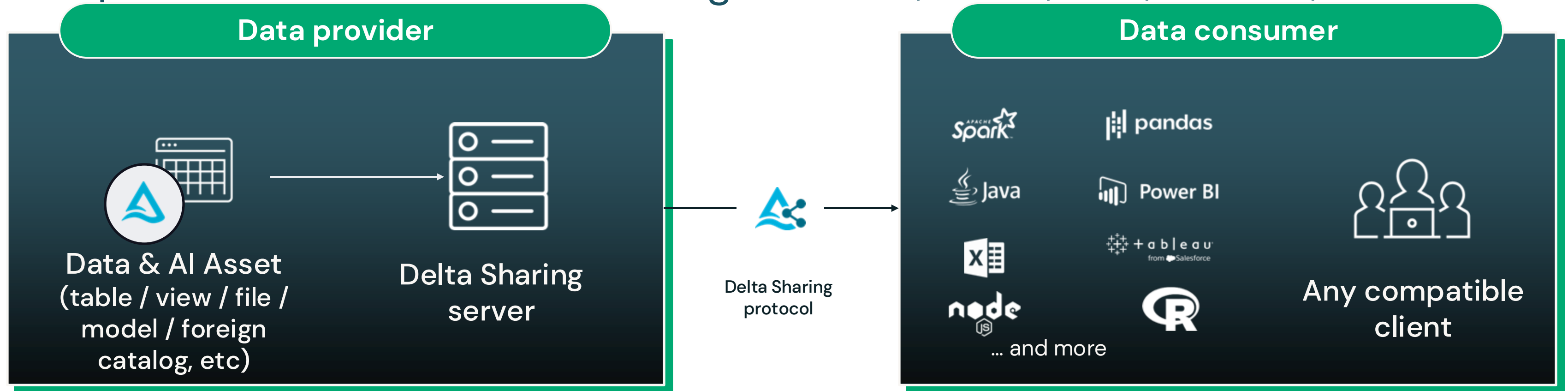
Key Concepts

Delta Sharing

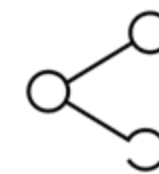


Delta Sharing

An open standard for secure sharing of tables, views, files, models, and more



Share cross-platform w/ open protocol

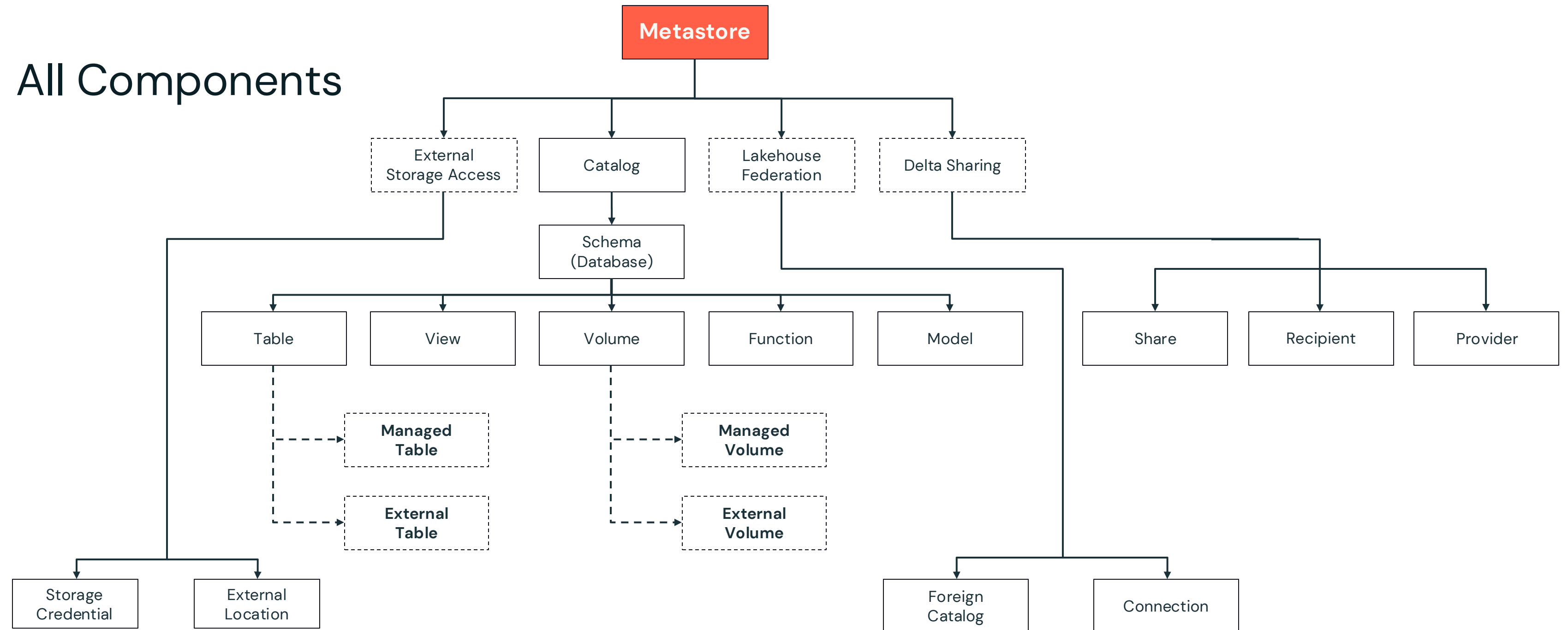


Share data with no replication



Key Concepts

All Components



Comparison of Managed and Unmanaged UC Tables

Consider the benefits of Managed tables

Characteristic	Managed (a.k.a. “Internal”)	Unmanaged (a.k.a. “External”)
Table’s Type Property Value	MANAGED	EXTERNAL
DROP Table Behavior	Deletes the data	Does not delete the data Deleted, must be done separately
Create Table Syntax	<code>CREATE TABLE <table> (<column> <data type> (<properties>))</code>	<code>CREATE TABLE <table> (<column> <data type> (<properties>)) LOCATION <location></code>
Data File Location	Managed by Databricks	Managed by user LOCATION keyword
Performance Optimizations	Auto Tune (Adaptive Optimization, Equi-join, Clustering, more coming)	Manually managed
Data Format Support	DELTA	DELTA, CSV, JSON, AVRO, PARQUET, ORC, TEXT
Pros	New features, performance, simplicity, stricter access	Non-delta tables, enforce storage naming, better compatibility with external readers/writers, simpler migration

Best Practice!

★ Lead with Managed Tables

Use Managed tables when possible for best performance and simplicity. Open APIs and ongoing interoperability work will continue reducing need for external tables.

Feature Store

Leveraging Data Foundation for Accelerated ML Development

You can use **any Delta table** in Unity Catalog with a **primary key** as a feature table for model training or inference.

~~CREATE TABLE~~ privilege on the Schema required.

Unity Catalog provides feature discovery, ~~governance~~, lineage, and cross-workspace access.

You can use tags, which are simple key-value pairs, to categorize and manage your feature tables and

Microsoft Azure | databricks | Search data, notebooks, recents, and more... | dais-uc101-eastus

Features

Any Delta table with a primary key can be used as a feature table. [Learn more](#)

iot | Unity Catalog | Hive Metastore | Catalogs: dev | Tag: department | Owned by me | Reset filters | Sort by: Relevance

Table name	Owner	Online stores	Last written	Tags	Comment
dev.iot.turbine_hourly_features	@databricks.com		3 minutes ago	department: MFG	These features are derived from the turbine_t...



Vector Search Index

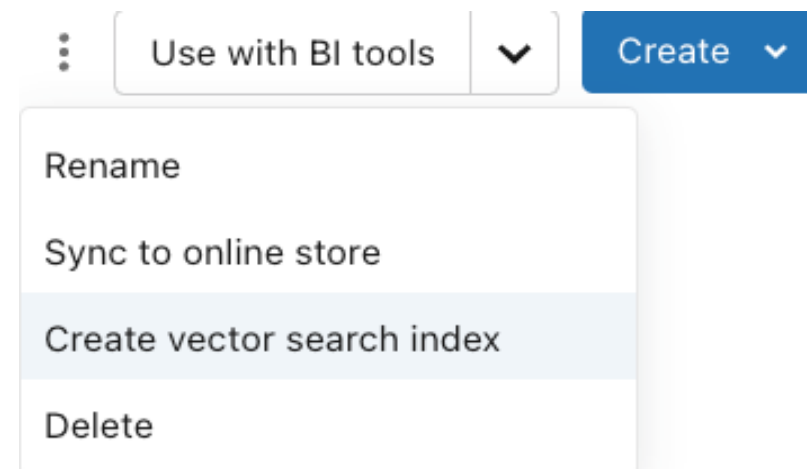
Create auto-updating vector indexes, managed by Unity Catalog

Choose your source table



id	text	col1	col2
1	The quick brown fox jumps ...		
2	How quickly daft...		
3	The five boxing wizards...		

Create semantic search index via Unity Catalog UI or via API



Choose any embedding model

Model Serving

- Foundation Model API
- Custom model
- External model

Call endpoint for real-time retrieval

```
result = index.similarity_search(  
    query_text="What is Spark  
Connect?",  
    columns=["id", "text", "link"],  
    filters={"doctype": "wiki"})
```

- Ingestion pipelines managed for you
- Indexes managed by Unity Catalog
- Also, APIs for
 - Self-managed embeddings
 - CRUD API upsert/delete

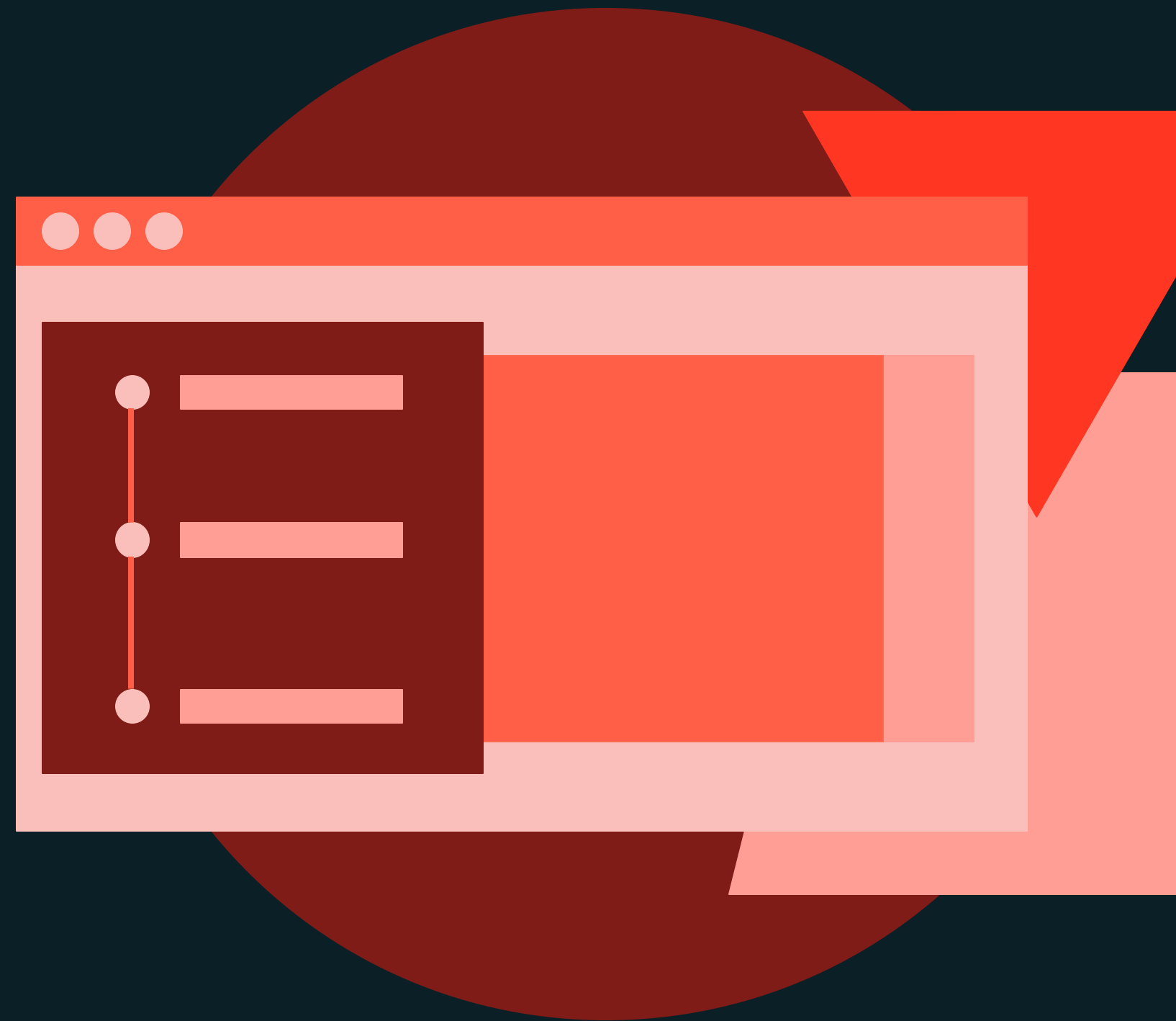
- Integrate with LangChain, LlamaIndex, etc.
- Scale out endpoints as needed





DEMONSTRATION

Populating the Metastore





LAB EXERCISE

Navigating the Metastore

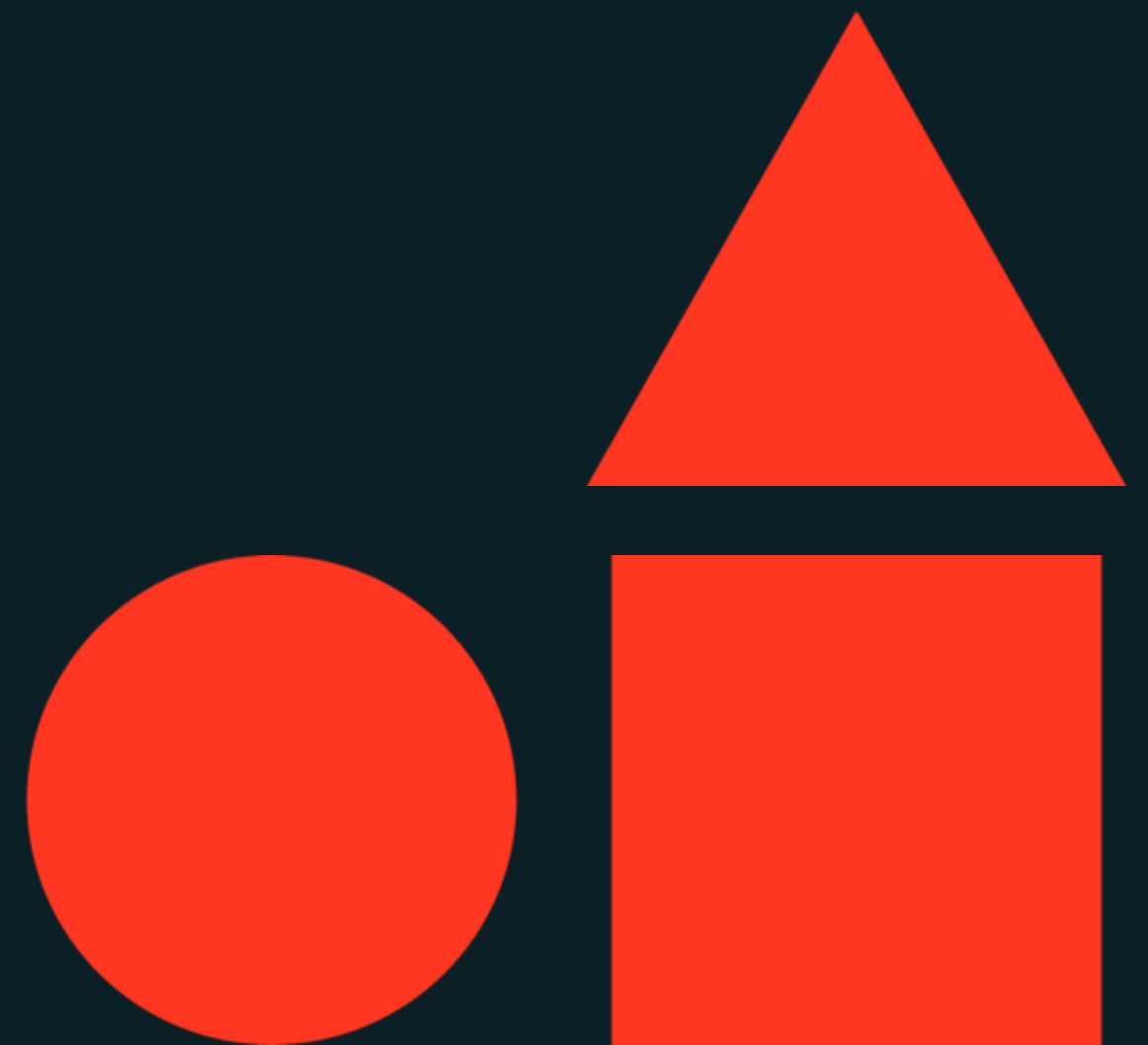


Organization and Access Patterns

Data Management and Governance with Unity Catalog



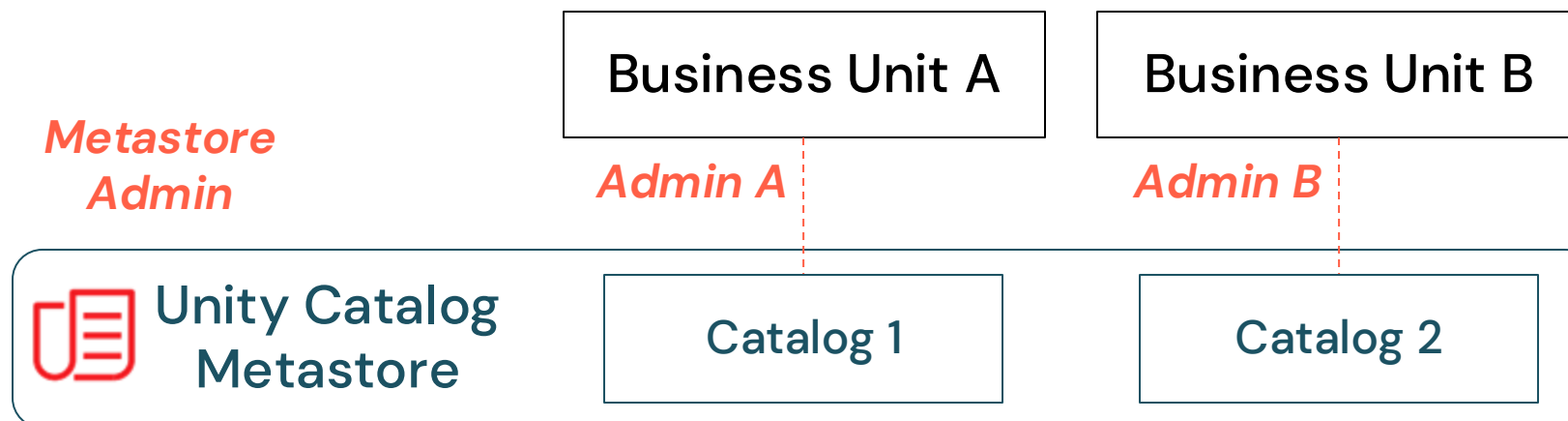
© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).



Unity Catalog Isolation features

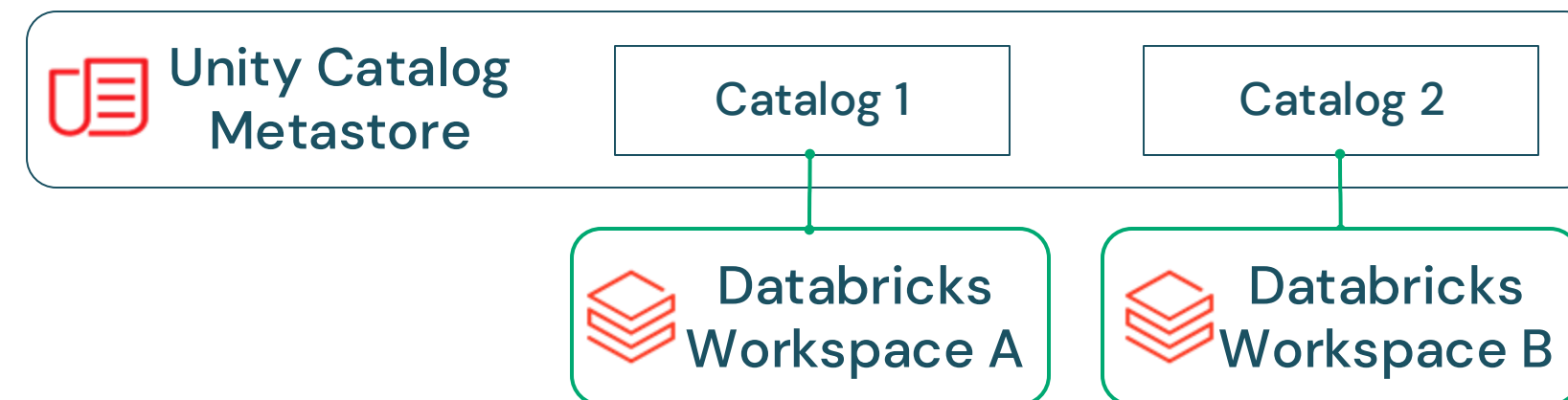
Delegation of Management (admin isolation)

Data should be managed by designated people/teams, based on the purpose/ownership of that data



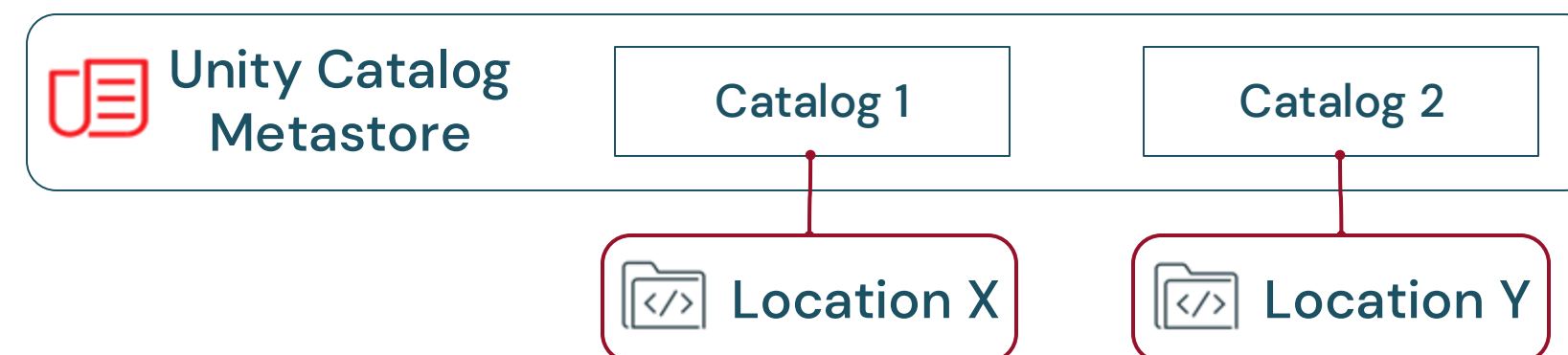
Workspace to catalog binding

Data should only be accessed in designated environments, based on the purpose of that data



Storage isolation

Data should be physically separated in storage



Note: The examples on catalog level also work on schema level

Unity Catalog Isolation combined

Delegation of Management (admin isolation)

Data should be managed by designated people / teams, based on the purpose/ownership of that data

Workspace to catalog binding

Data should only be accessed in designated environments, based on the purpose of that data

- One catalog can be bound to multiple workspaces
- One Workspaces can be bound to multiple catalogs

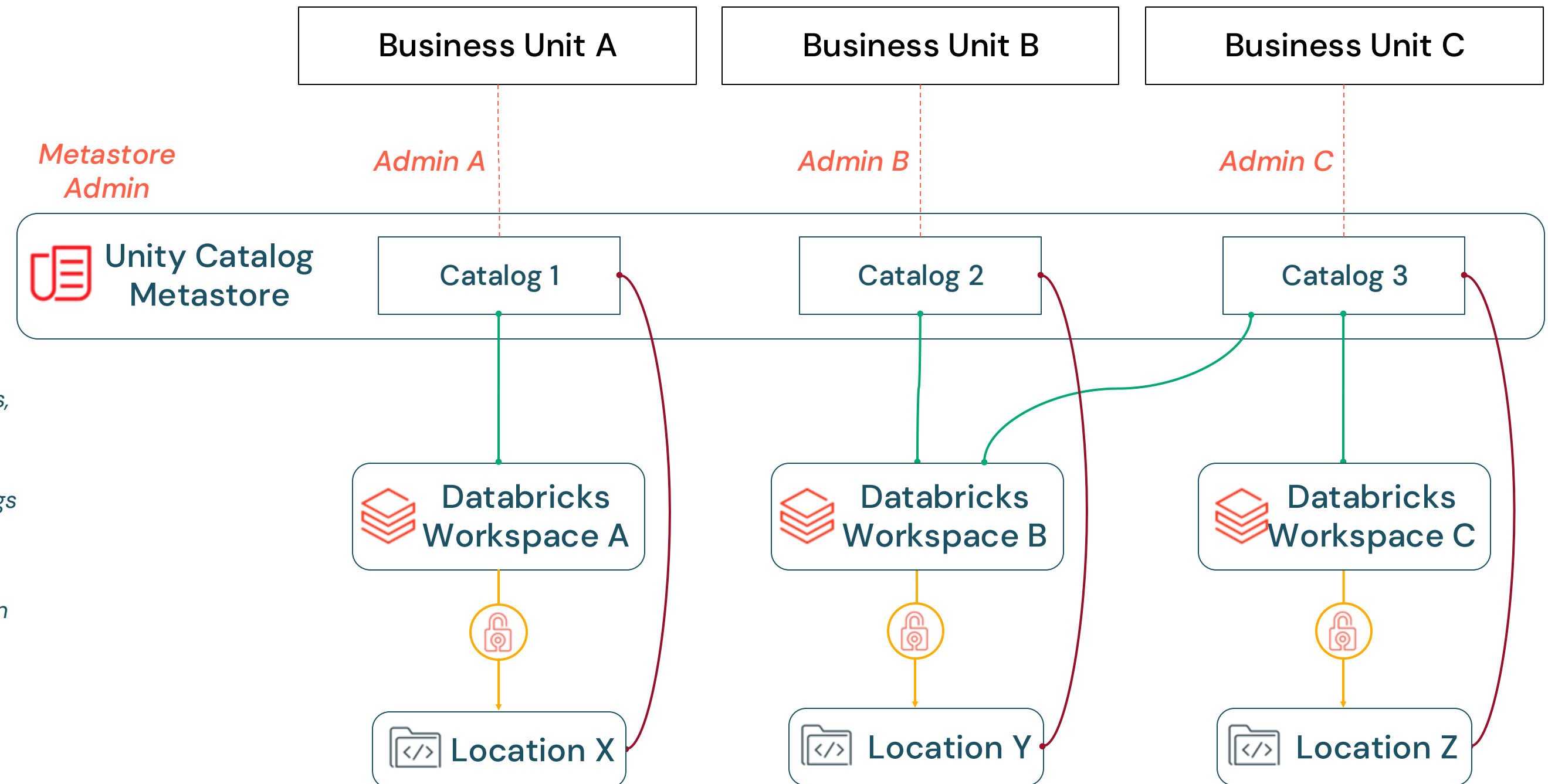
UC Access Control

Users should only gain access to data/ metadata based on agreed access rules

Storage isolation

Data should be physically separated in storage

"Location" is the unique combination of container/bucket + path



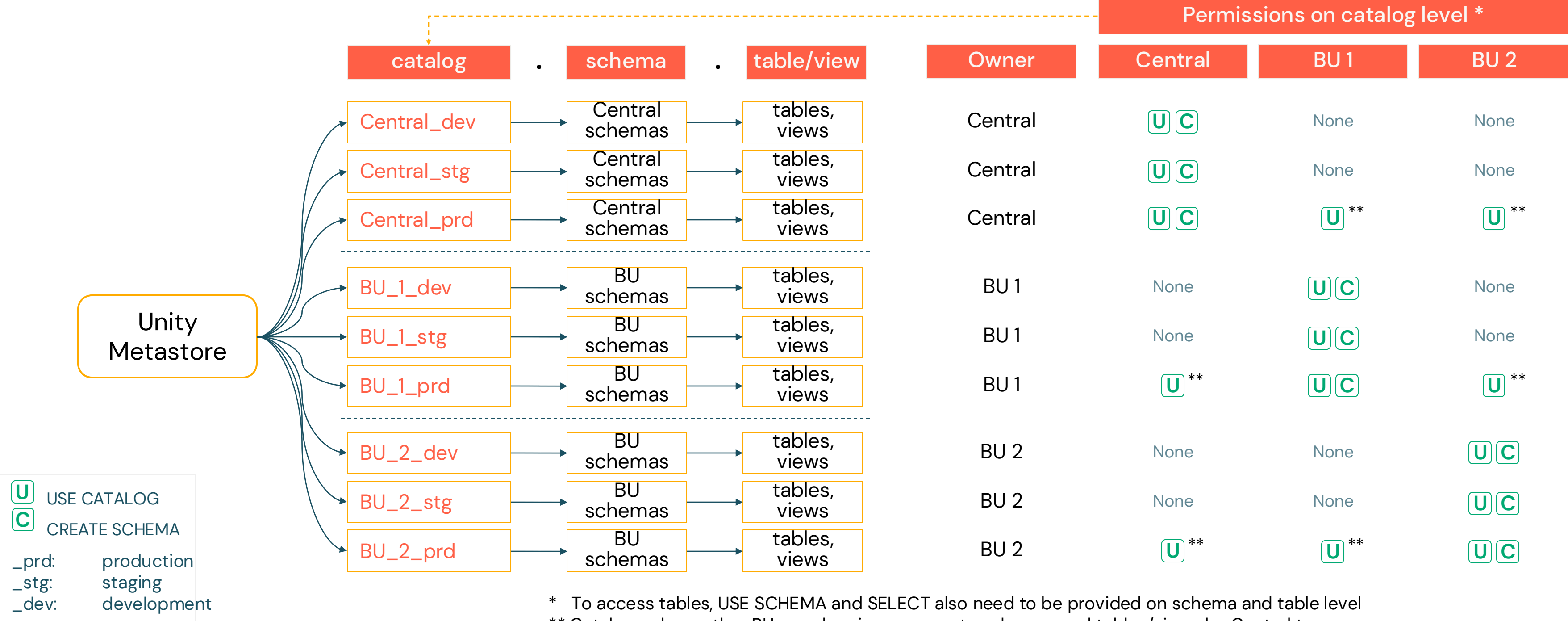
Note: The examples on catalog level also work on schema level



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

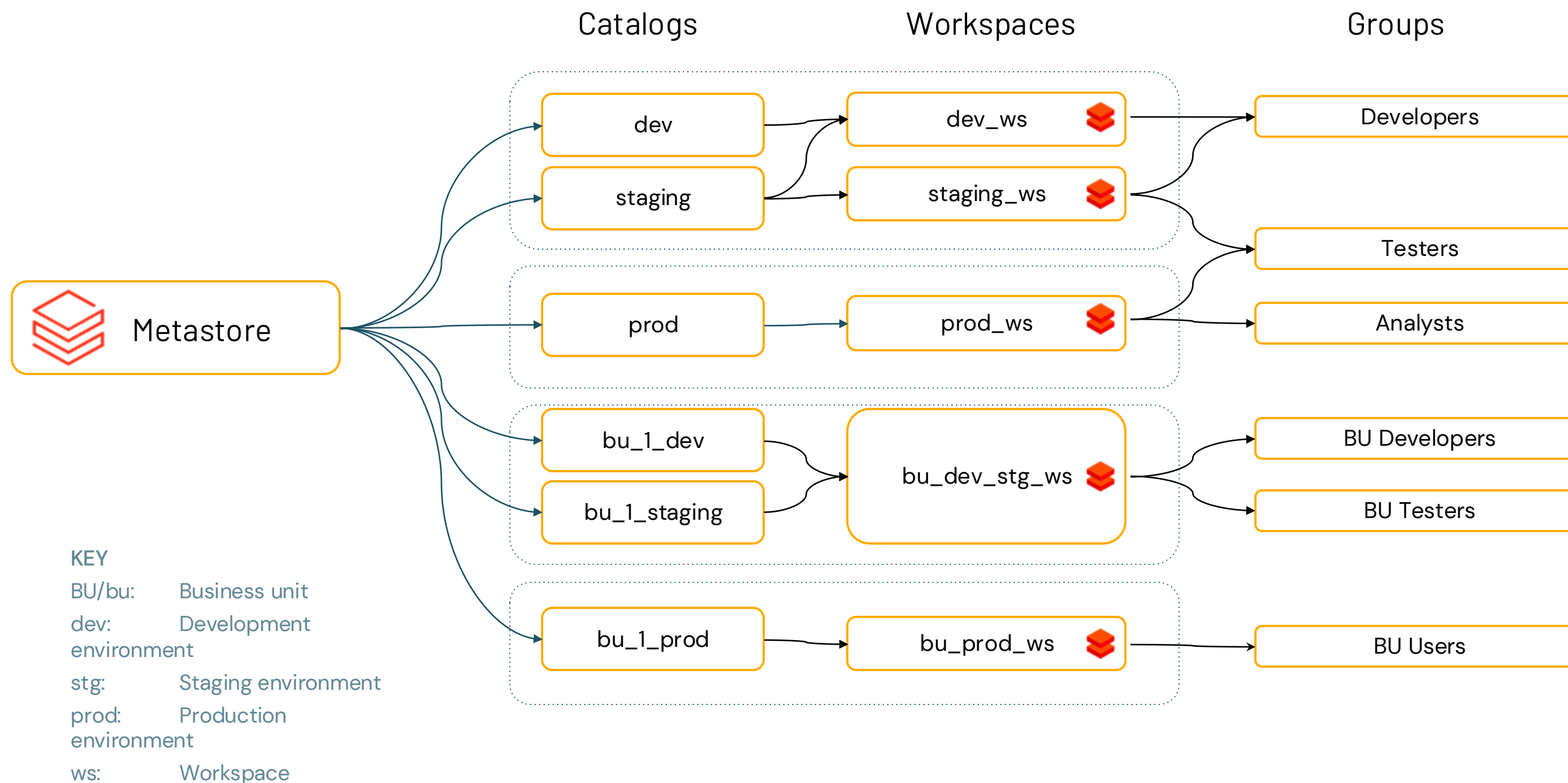
Practical Permissioning Example

Catalog Organization and Ownership



Access data from specified environments only

Restrict catalog access by environment or purpose



Access to data and availability of data can be isolated across workspaces and groups



Access data from specified environments only

Restrict access to data

Best Practice!

★ Separate Catalogs by Responsibility

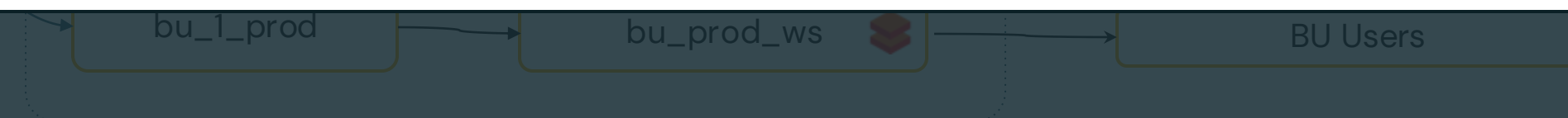
- Development Environment (dev / qa / prod)
- Business Unit (IT, Finance, Sales, etc.)
- Sharing / Foreign Databases

★ Use Workspace-Catalog Binding where required

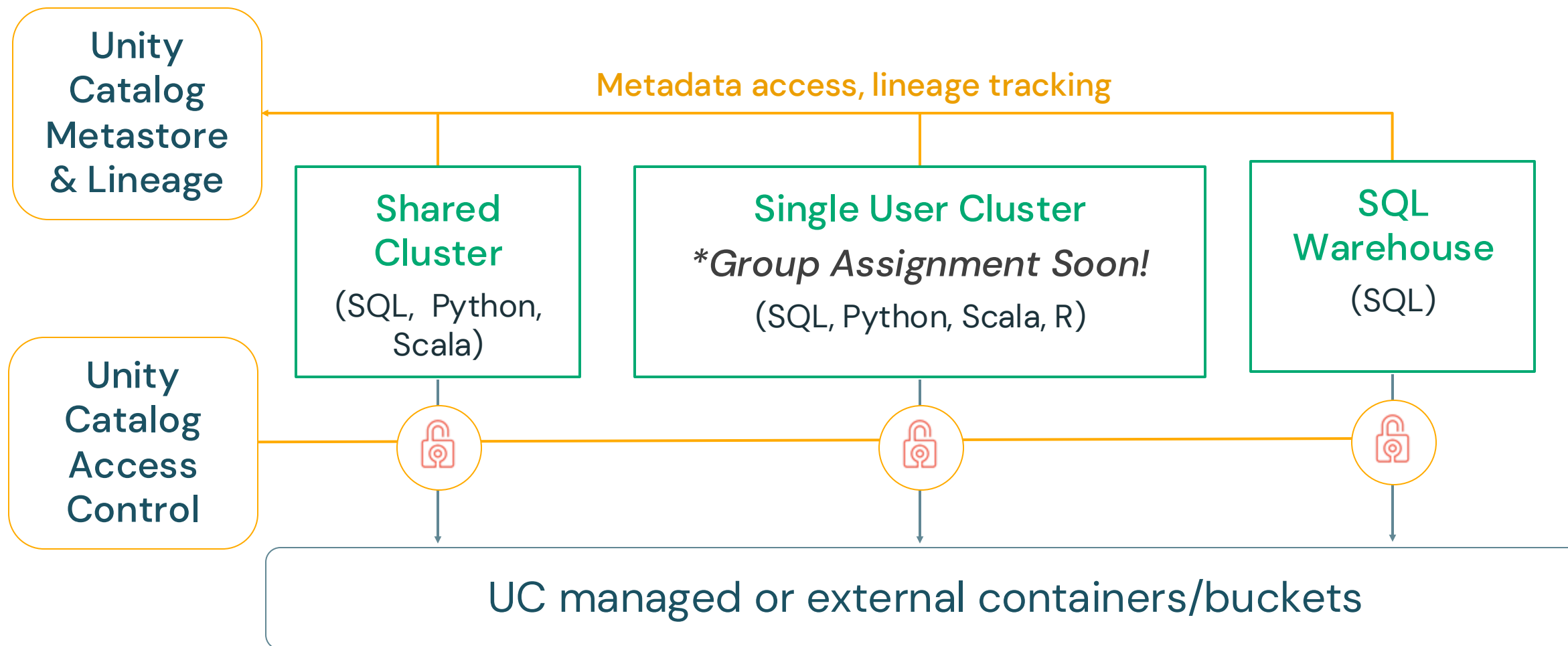
Create the minimum catalogs required to meet your team's isolation and ownership requirements.

Access to data
and availability of
data can be
isolated across
workspaces and
groups

KEY
BU/bu: Business Unit
dev: Development environment
stg: Staging environment
prod: Production environment
ws: Workspace



Clusters/warehouses with Unity Catalog



Shared clusters (Access mode: "Shared")

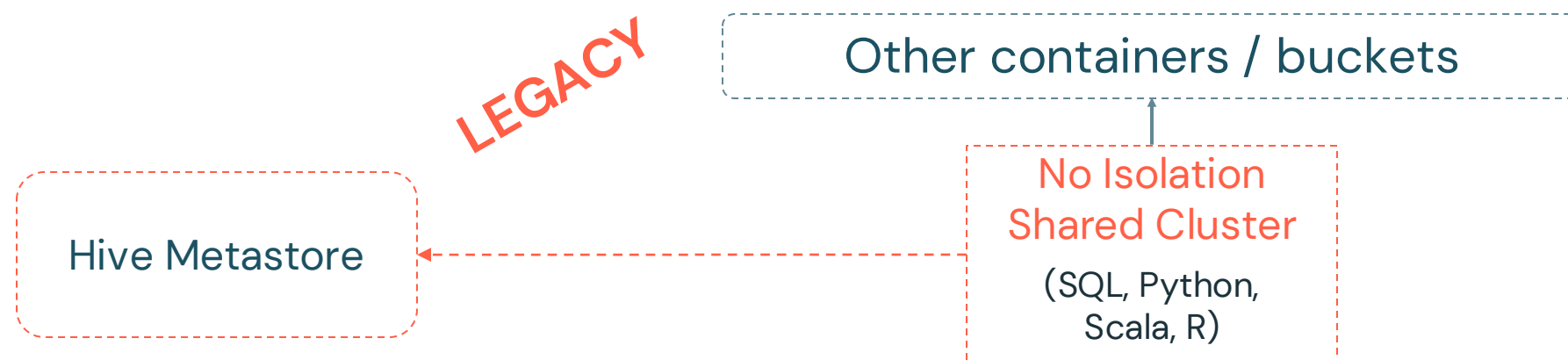
- Multiple users can work on a cluster in parallel
- Python, SQL and Scala

Single user clusters (Access mode: "Single User")

- A single user cluster is dedicated to a single user at creation time
- Assignment to a Group coming soon!
- Additionally supports R + ML Runtimes, Spark submit jobs
- Fine-grained access control coming soon!

SQL Warehouse (Classic, Pro and Serverless)

- Multiple users can work on a shared cluster in parallel
- SQL only



No isolation shared clusters (legacy)

- Multiple users can work on a cluster in parallel
- Any language, any workload
- No governance by Unity Catalog (access control, lineage, ...) and no access to UC tables



Clusters/warehouses with Unity Catalog

Best Practice!

- ★ Use Shared Clusters as your default compute, and fall-back to single user in the case of limitations.
- ★ Develop and Deploy using the same access mode!

LEGACY

Other containers / buckets

Hive Metastore

No Isolation
Shared Cluster
(SQL, Python,
Scala, R)

No isolation shared clusters (legacy)

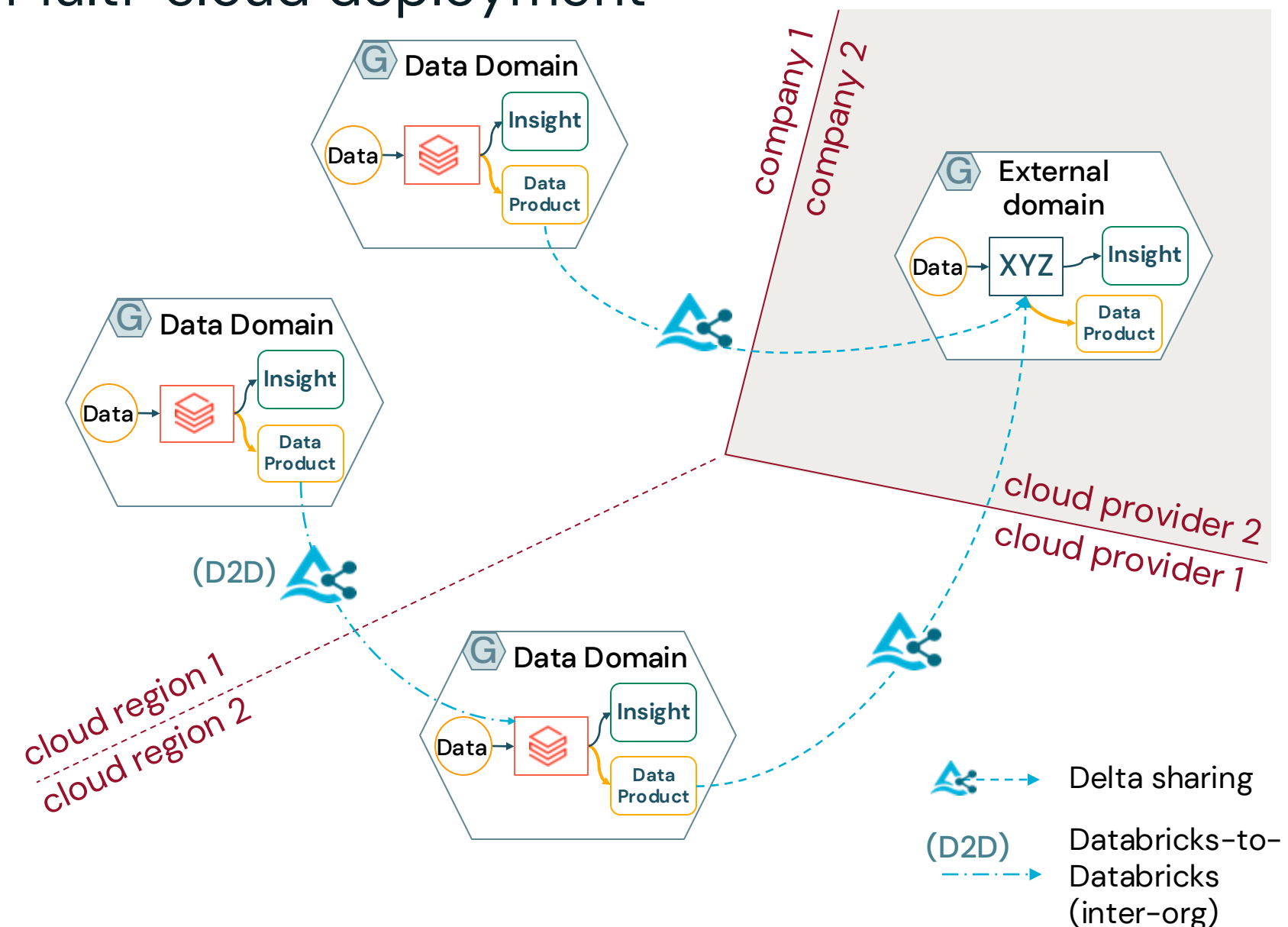
- Multiple users can work on a cluster in parallel
- Any language, any workload
- No governance by Unity Catalog (access control, lineage, ...) and no access to UC tables



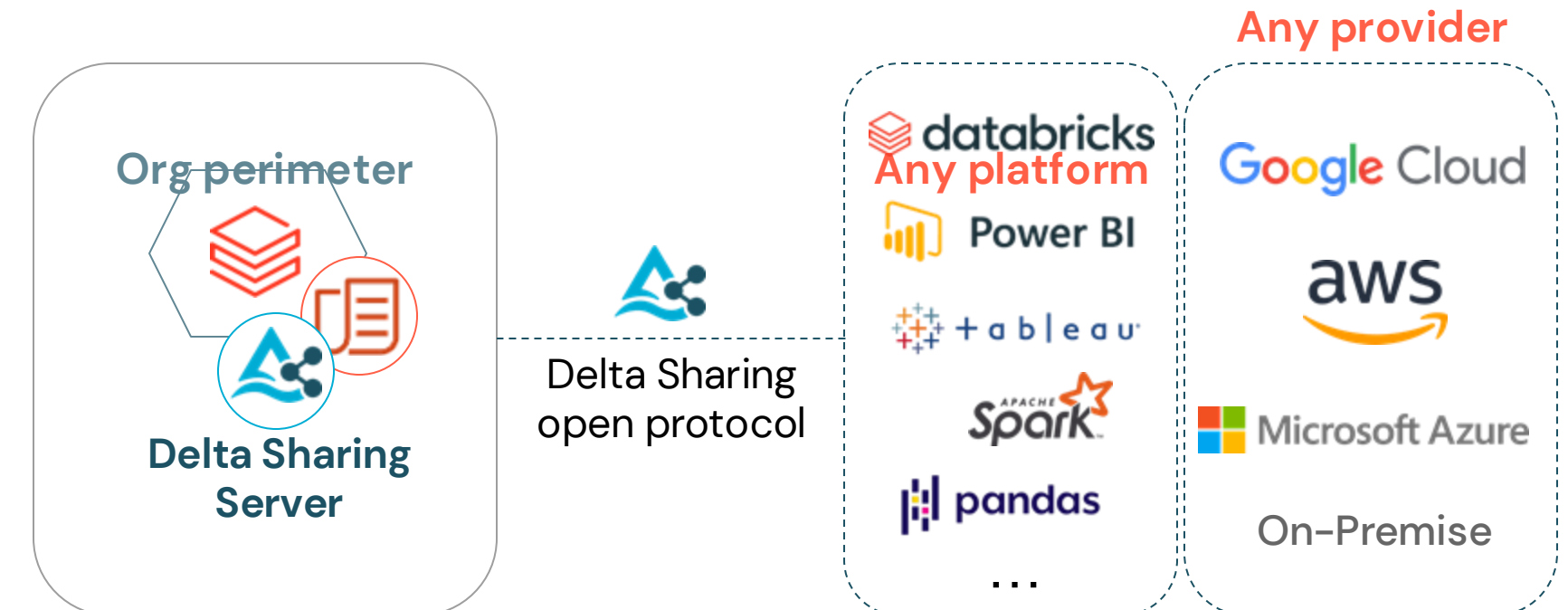
Data Sharing

Secure access across organisational and regional boundaries

Multi-cloud deployment



Delta Sharing protocol



Unity Catalog manages all internal governance and access controls, and processes Delta Sharing requests

Delta Sharing Server receives requests via Delta Sharing protocol for secure access to data by external parties

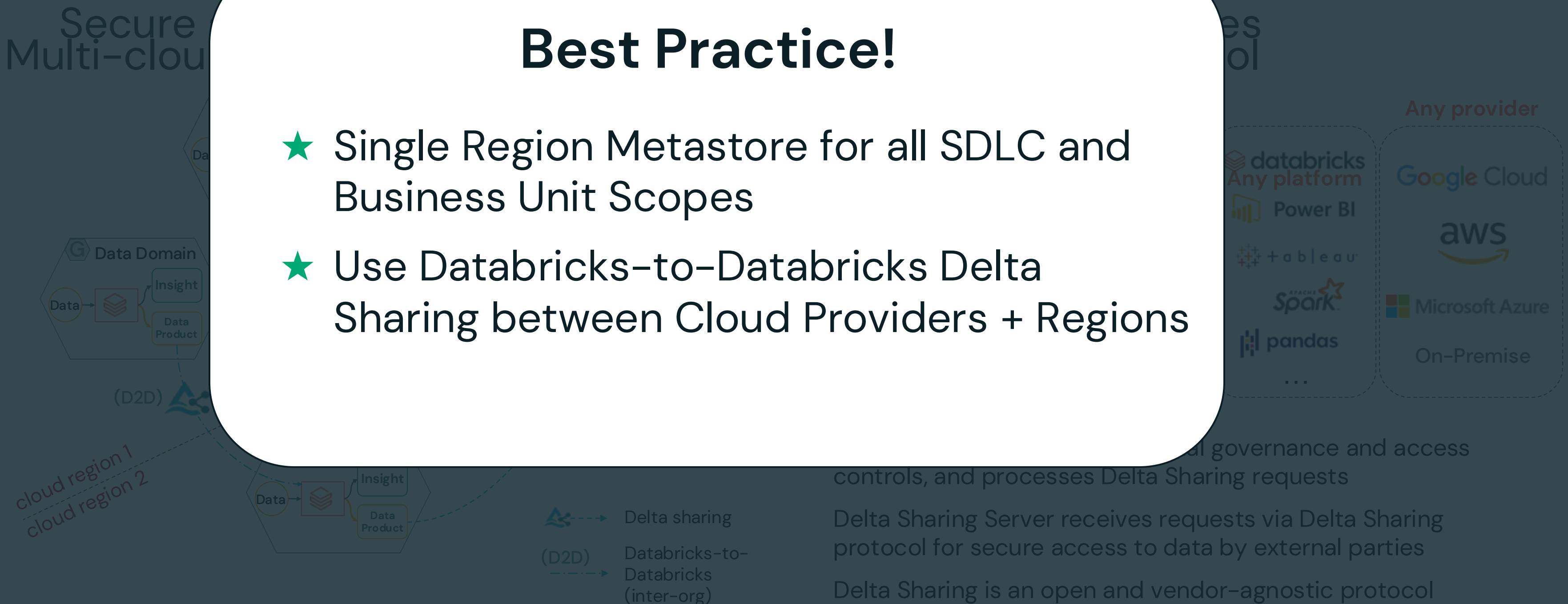
Delta Sharing is an open and vendor-agnostic protocol



Data Sharing

Best Practice!

- ★ Single Region Metastore for all SDLC and Business Unit Scopes
- ★ Use Databricks-to-Databricks Delta Sharing between Cloud Providers + Regions



controls, and processes Delta Sharing requests

Delta Sharing Server receives requests via Delta Sharing protocol for secure access to data by external parties

Delta Sharing is an open and vendor-agnostic protocol



Tagging + AI generated Documentation

- Auto-generate concise and informative **table and column comments** for Unity Catalog
- Document your backlog of data assets with **missing documentation** in minutes
- **Tag** your data for easier discovery and to facilitate tag-based policies

The screenshot displays the Databricks Unity Catalog interface. On the left, a sidebar shows the catalog structure: 'sandbox.features'. The main panel shows the 'retention_prod.sandbox.features' table. A table lists columns with their types and AI-generated comments:

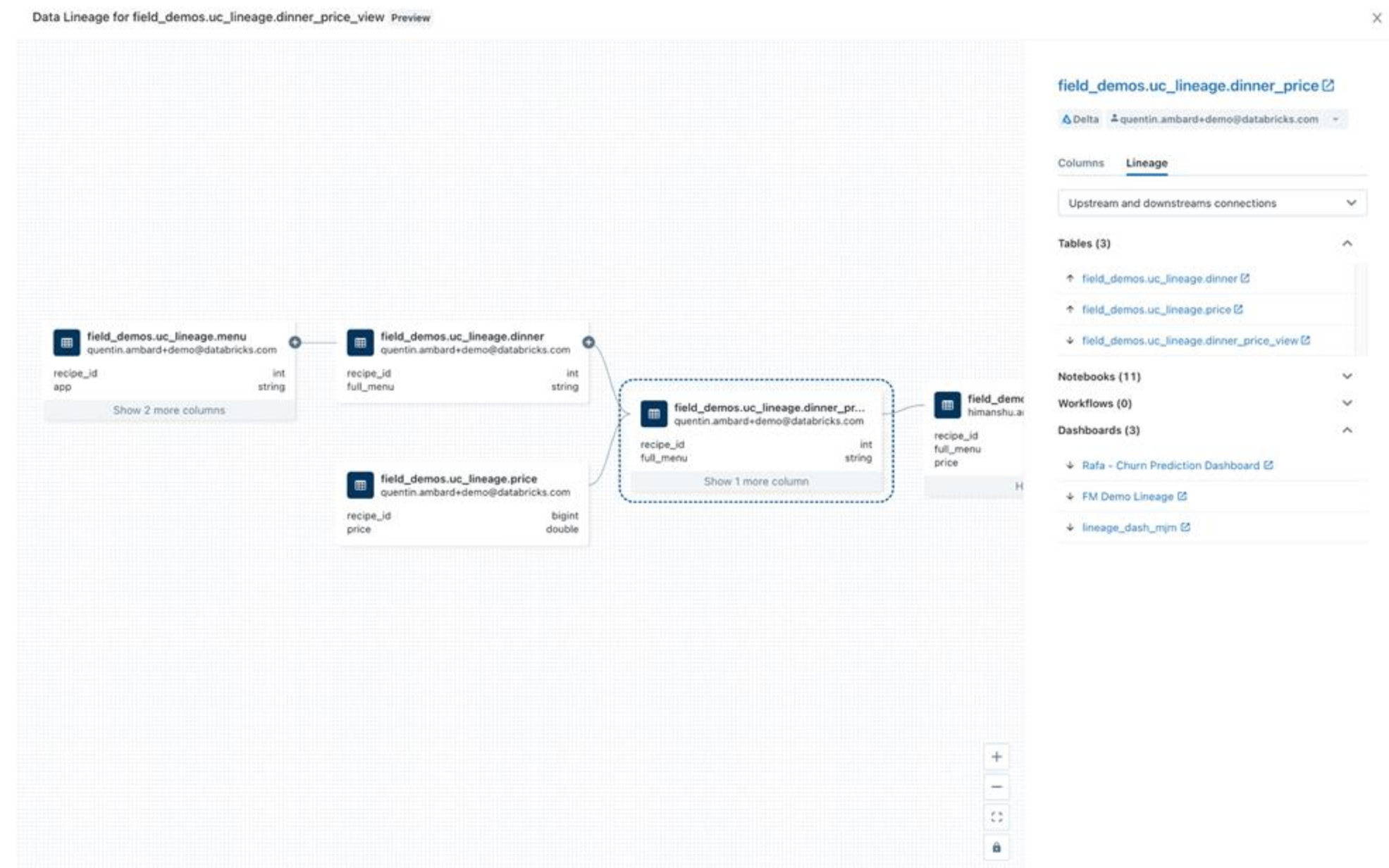
Column	Type	Comment	Tags	Mask
user_id	string	Unique identifier for the user, allowing tracking of user behavior and preferences.	saptype: Customer	
event_id	string	Identifier for the event, allowing tracking of different events and their associated user actions.		
platform	string	Represents the platform or application where the user interacted with the event.		
date	string	Date when the user interacted		

On the right, the 'About this table' section provides metadata: Owner: paul.roome@databricks.com, Data source format: Delta, Last Updated: 21 hours ago, Popularity: .11, Size: 69.8KiB, 1 file. A 'Comment' section at the bottom explains the table's purpose: 'The 'features' table captures user engagement data across different platforms. It includes information about the user's actions and interactions, such as the date of the event, the platform used, and the specific action taken. The table also includes details about the user's session, such as the URL they were on and any rescued data they may have encountered. This data can be used to understand user behavior patterns, identify popular features or areas of interest, and track user engagement over time. It can also be useful in predicting user churn or rescuing data for users in distress.'

Automated lineage for all workloads

End-to-end visibility into how data flows and consumed in your organization

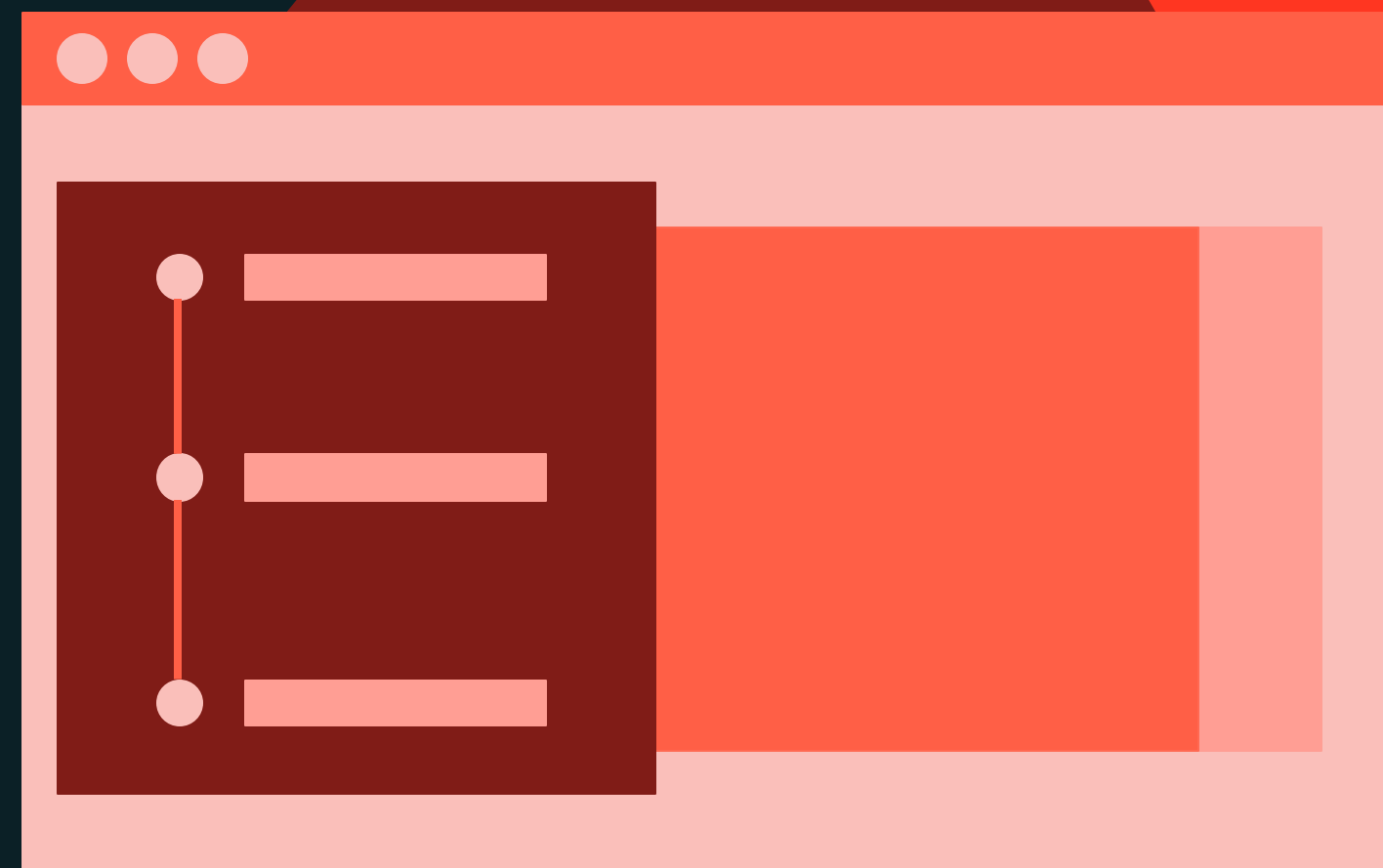
- Auto-capture runtime data lineage on a Databricks cluster or SQL warehouse
- Leverage common permission model from Unity Catalog
- Lineage across tables, columns, dashboards, workflows, notebooks, files, external sources, and models





DEMONSTRATION

Upgrading Tables to Unity Catalog



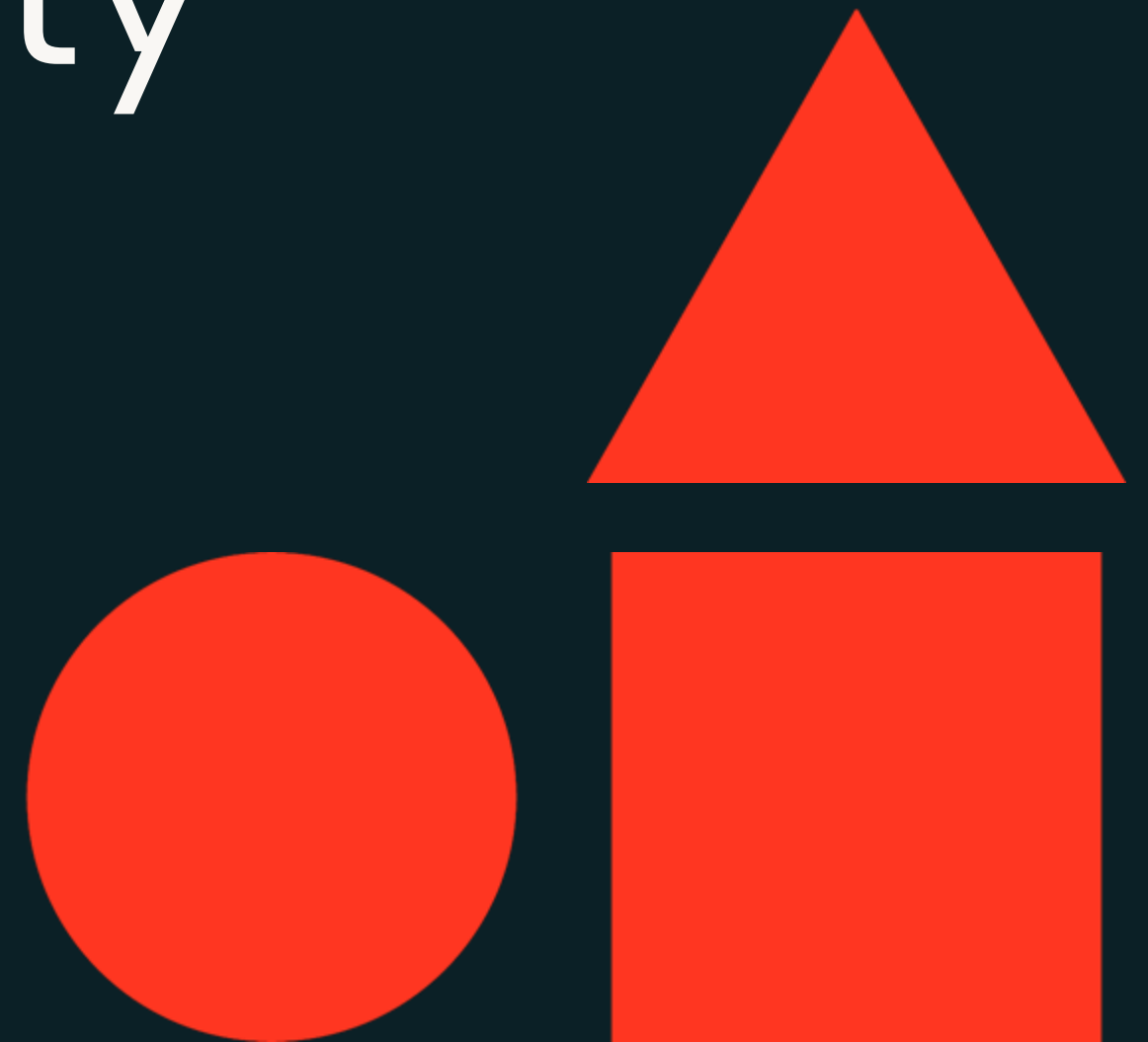


Security and Administration in Unity Catalog

**Data Management and Governance with Unity
Catalog**

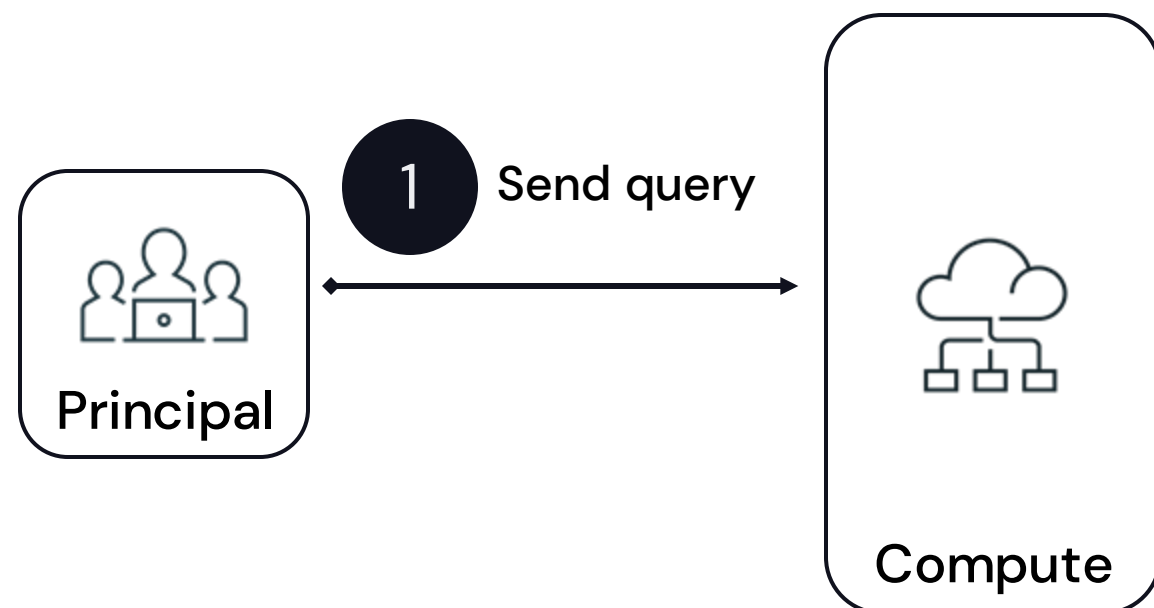


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).



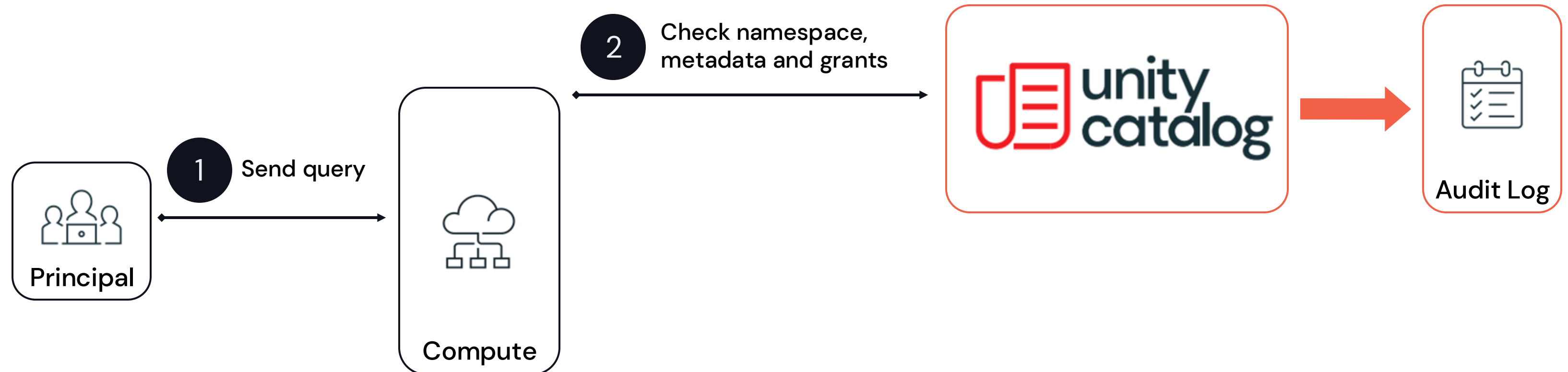
Security Model

Query Lifecycle



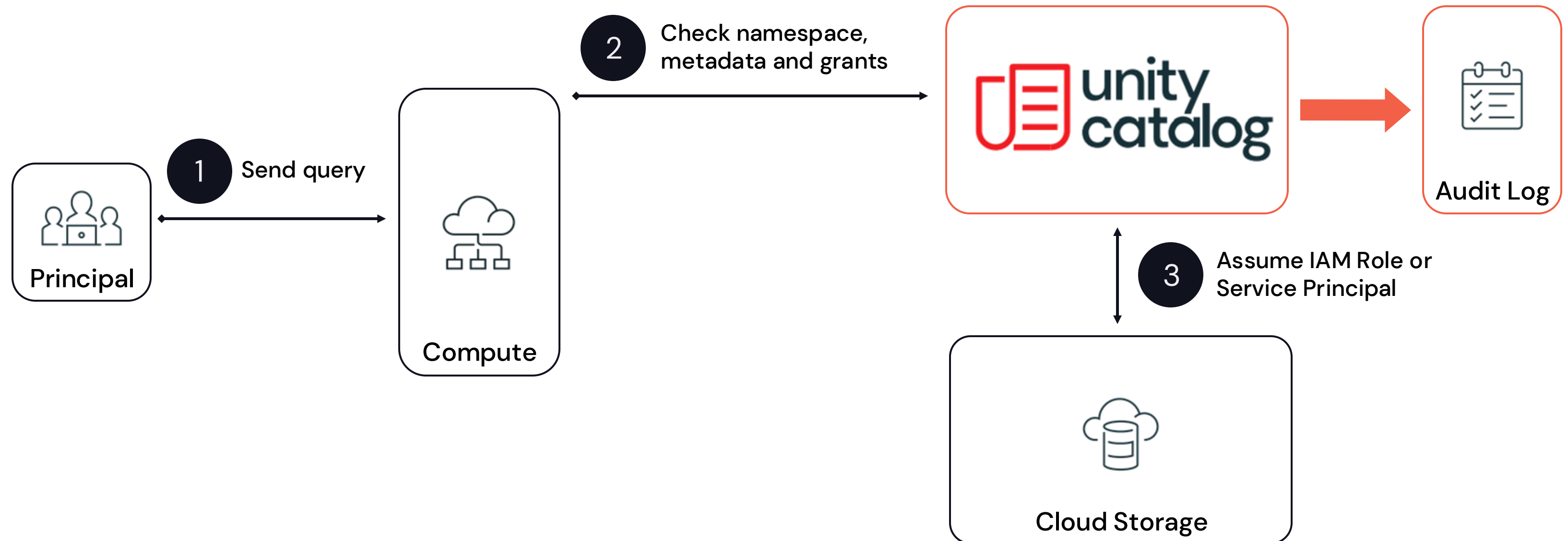
Security Model

Query Lifecycle



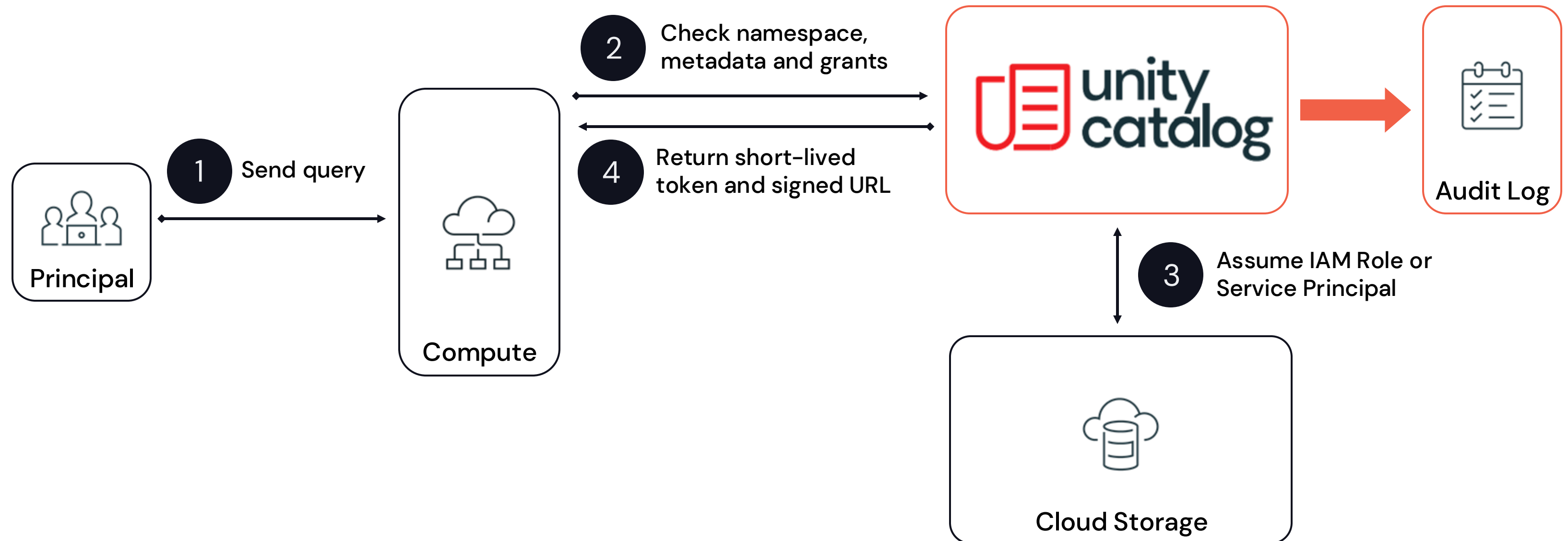
Security Model

Query Lifecycle



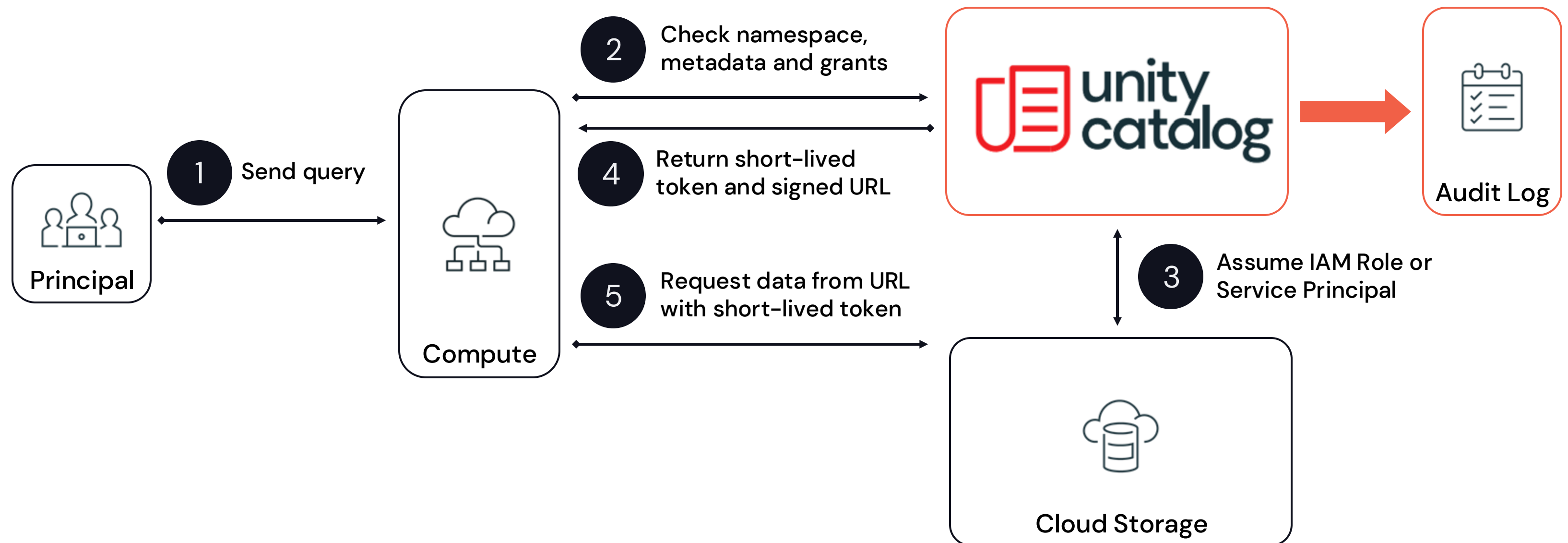
Security Model

Query Lifecycle



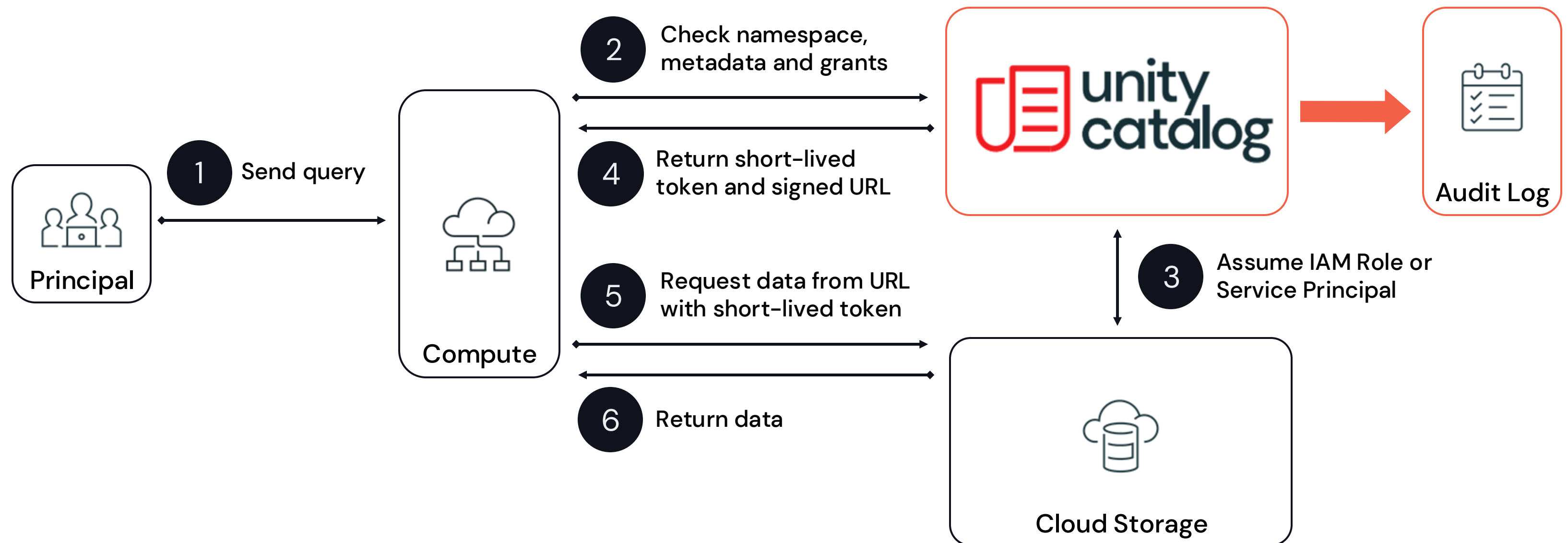
Security Model

Query Lifecycle



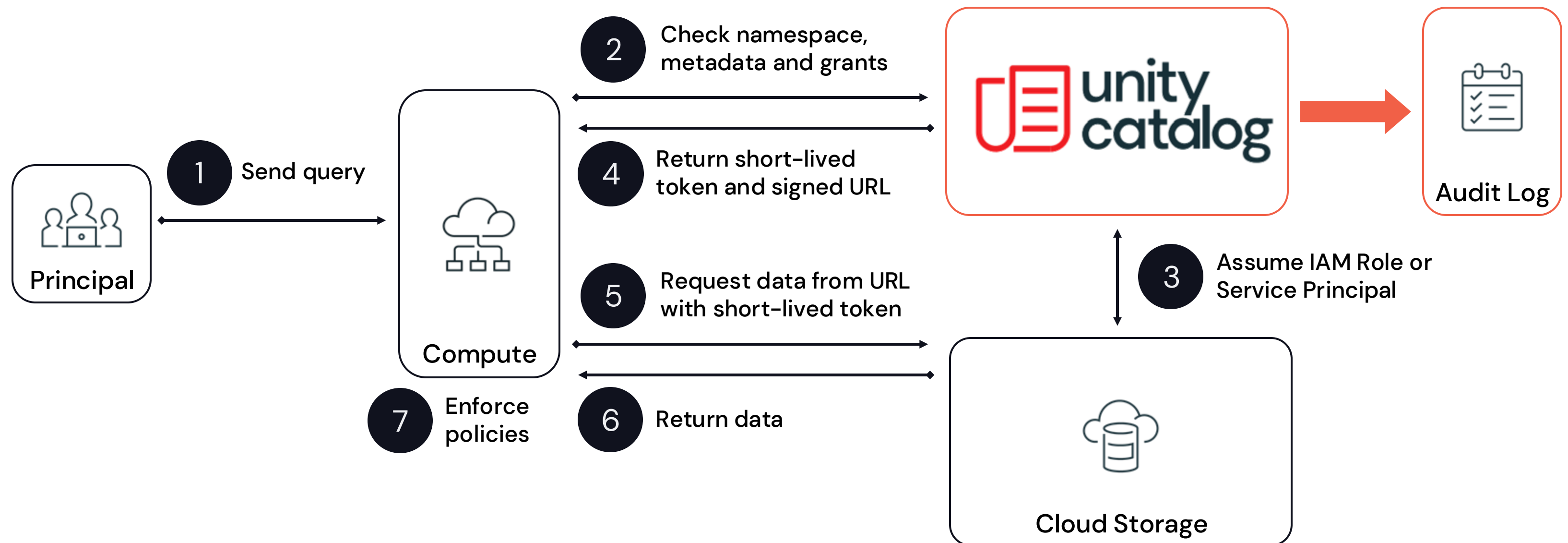
Security Model

Query Lifecycle



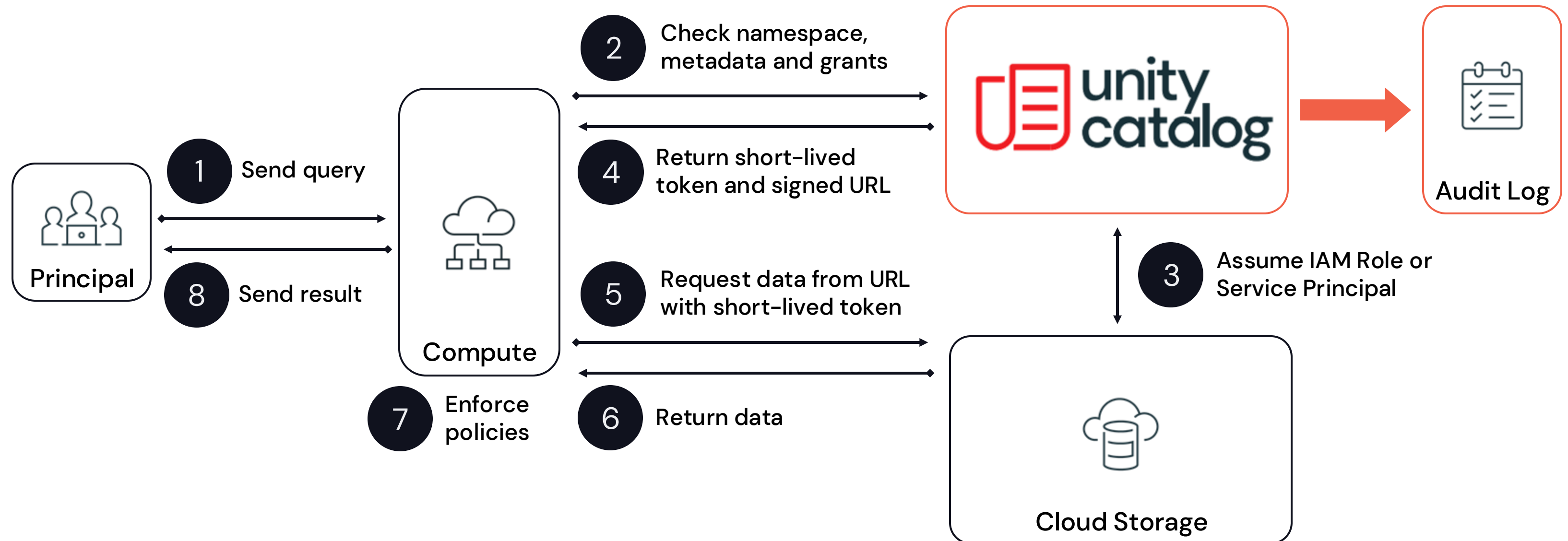
Security Model

Query Lifecycle



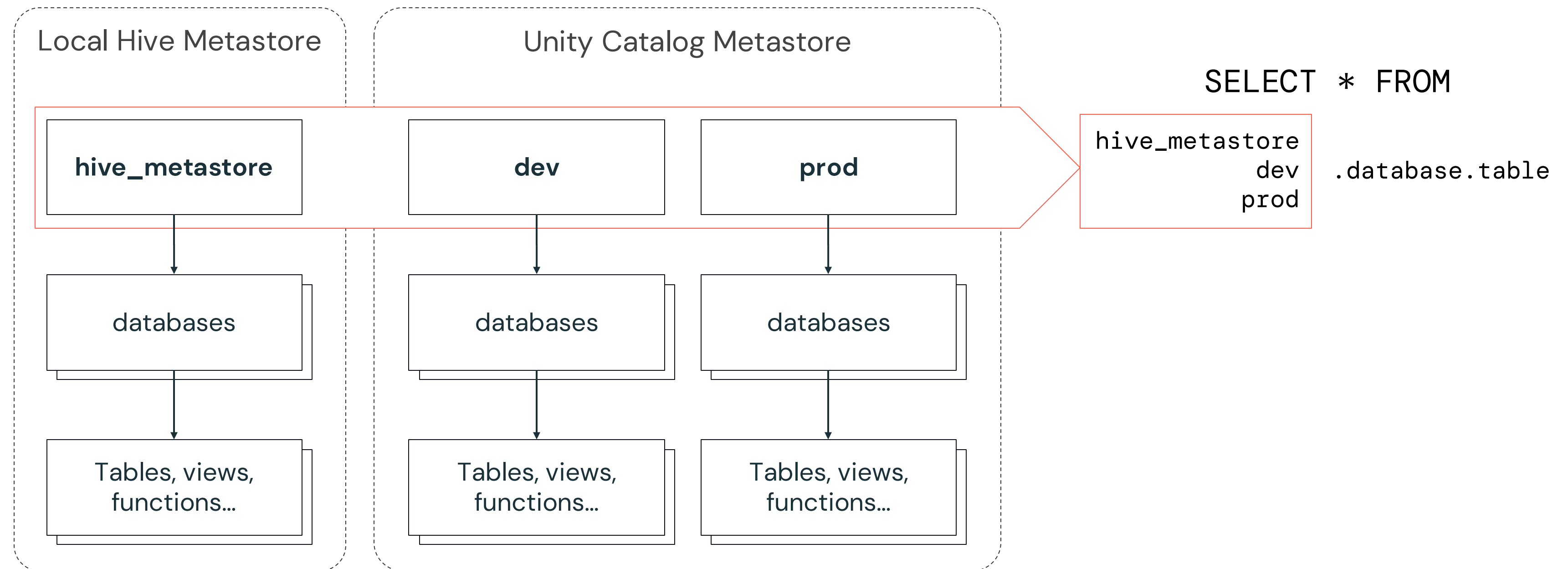
Security Model

Query Lifecycle



Security Model

Accessing Legacy Metastore





Databricks Marketplace Overview

**Data Management and Governance with Unity
Catalog**



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).



Databricks Marketplace

Open Marketplace for All Your Data, Analytics, and AI



Open exchange for all data products

- **Datasets**
- **Notebooks**
- **Dashboards**
- **ML models**
- **Solutions Accelerators**

Powered by



Delta Sharing



Databricks Marketplace

Open Marketplace for All Your Data, Analytics, and AI





Privileges in Unity Catalog

Data Management and Governance with Unity Catalog



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).



Data Security Model

Overview

Table t

Owned by `jdoo@company.com`

Principal	Privilege
analysts	SELECT
dbadmins	ALL PRIVILEGES



Centralized Access Controls

Centrally grant and manage access permissions across workloads and foreign databases

Using ANSI SQL DCL

```
GRANT <privilege> ON <securable_type>  
<securable_name> TO <principal>
```

```
GRANT SELECT ON iot.events TO engineers
```

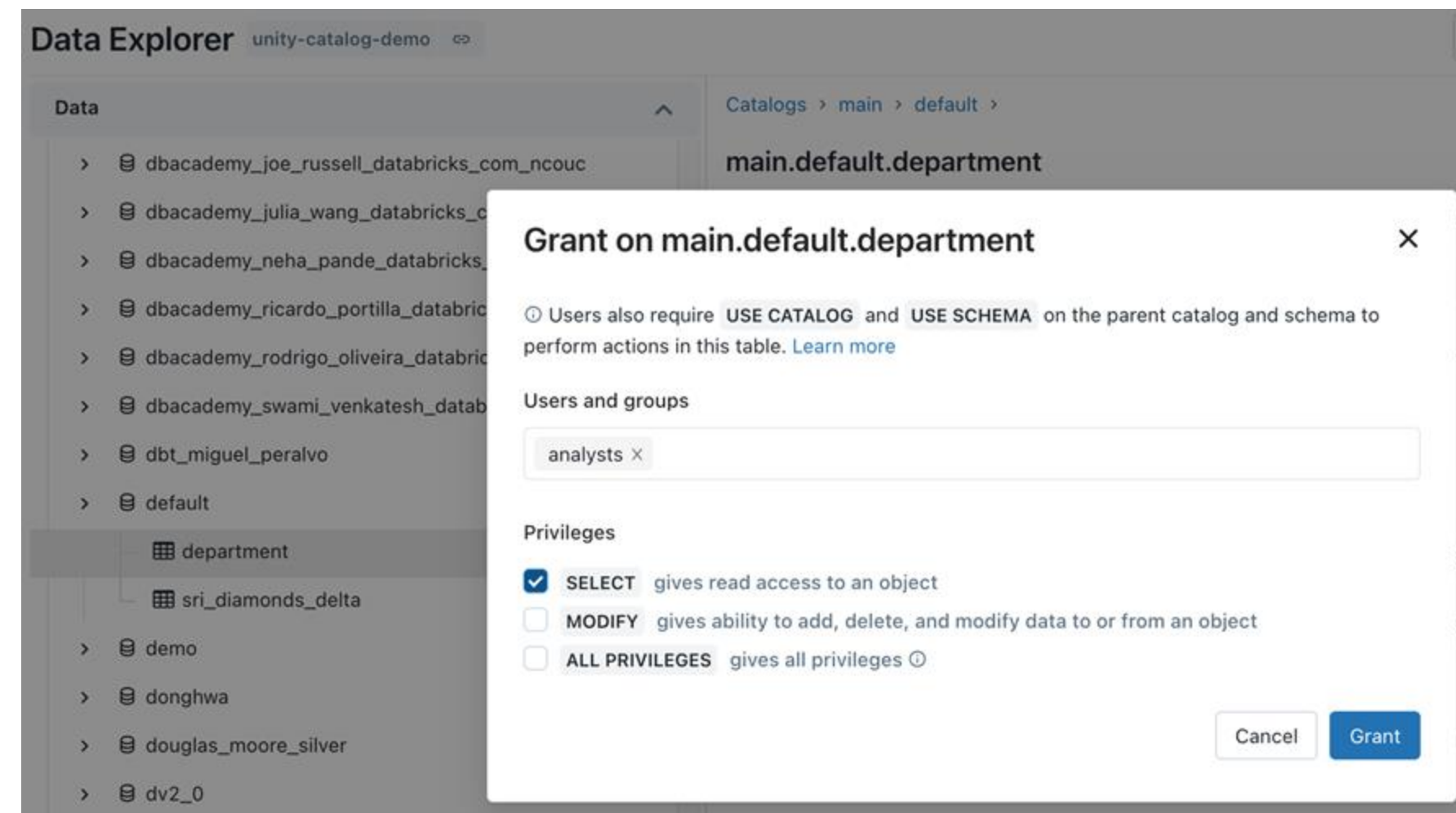
Choose
permission level

'Table' = collection of
files in S3/ADLS

Sync groups from
your identity
provider

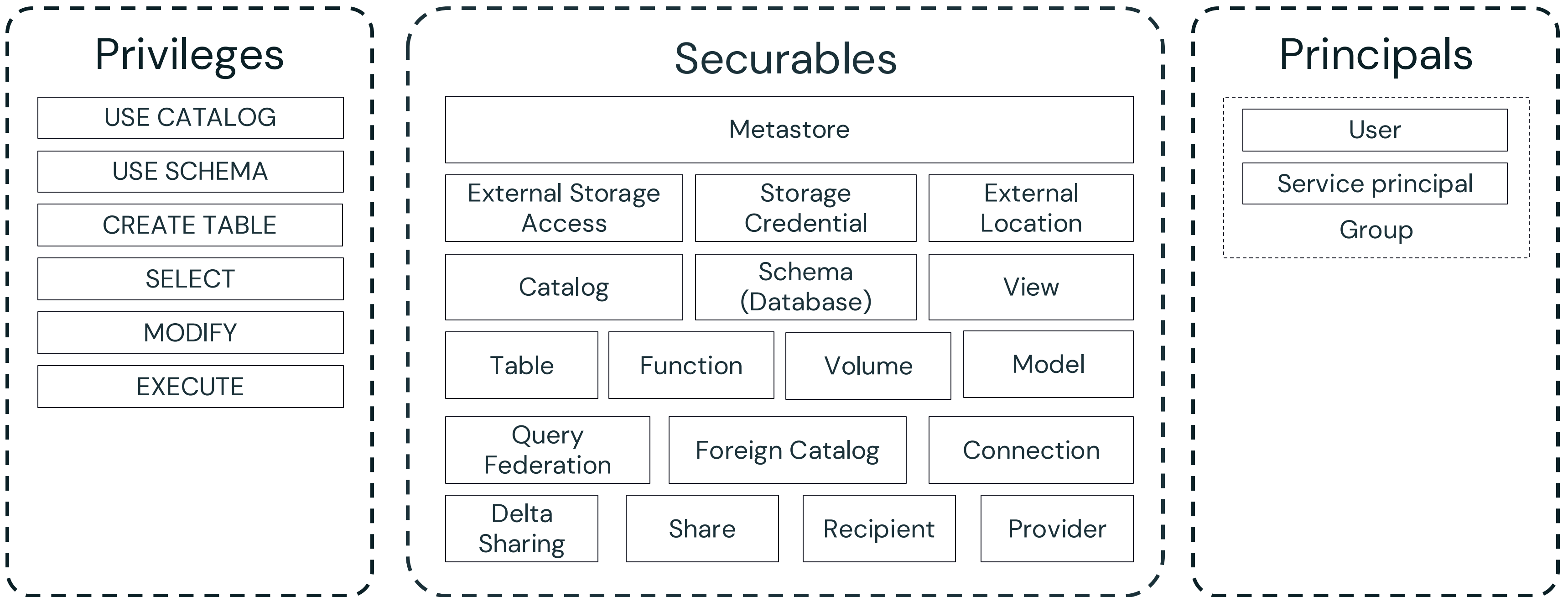
*securable = catalog, schema, table, view,
function, share, volume, model, etc*

Using UI



Data Security Model

Access control lists (ACLs)

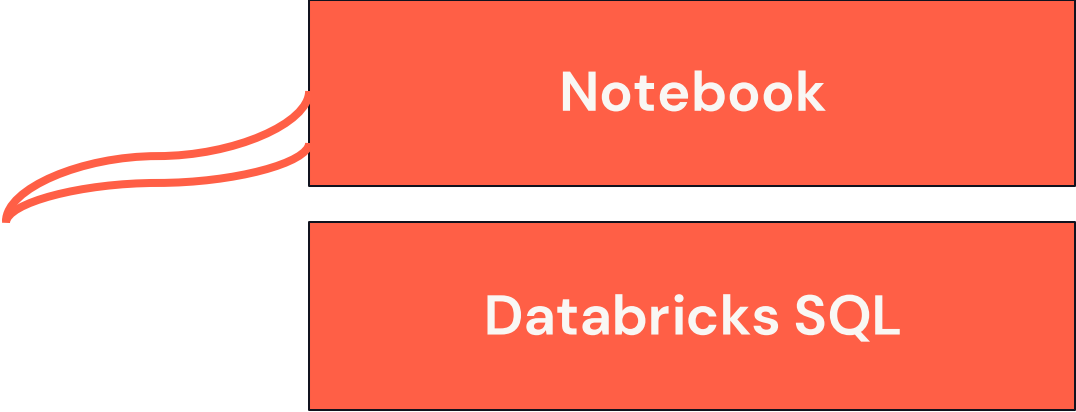


Data Security Model

Managing ACLs

1. SQL

GRANT SELECT ON TABLE t TO analysts
REVOKE SELECT ON TABLE t FROM analysts



Notebook

Databricks SQL

2. Data explorer

Data Science & Engineering workspace or DBSQL

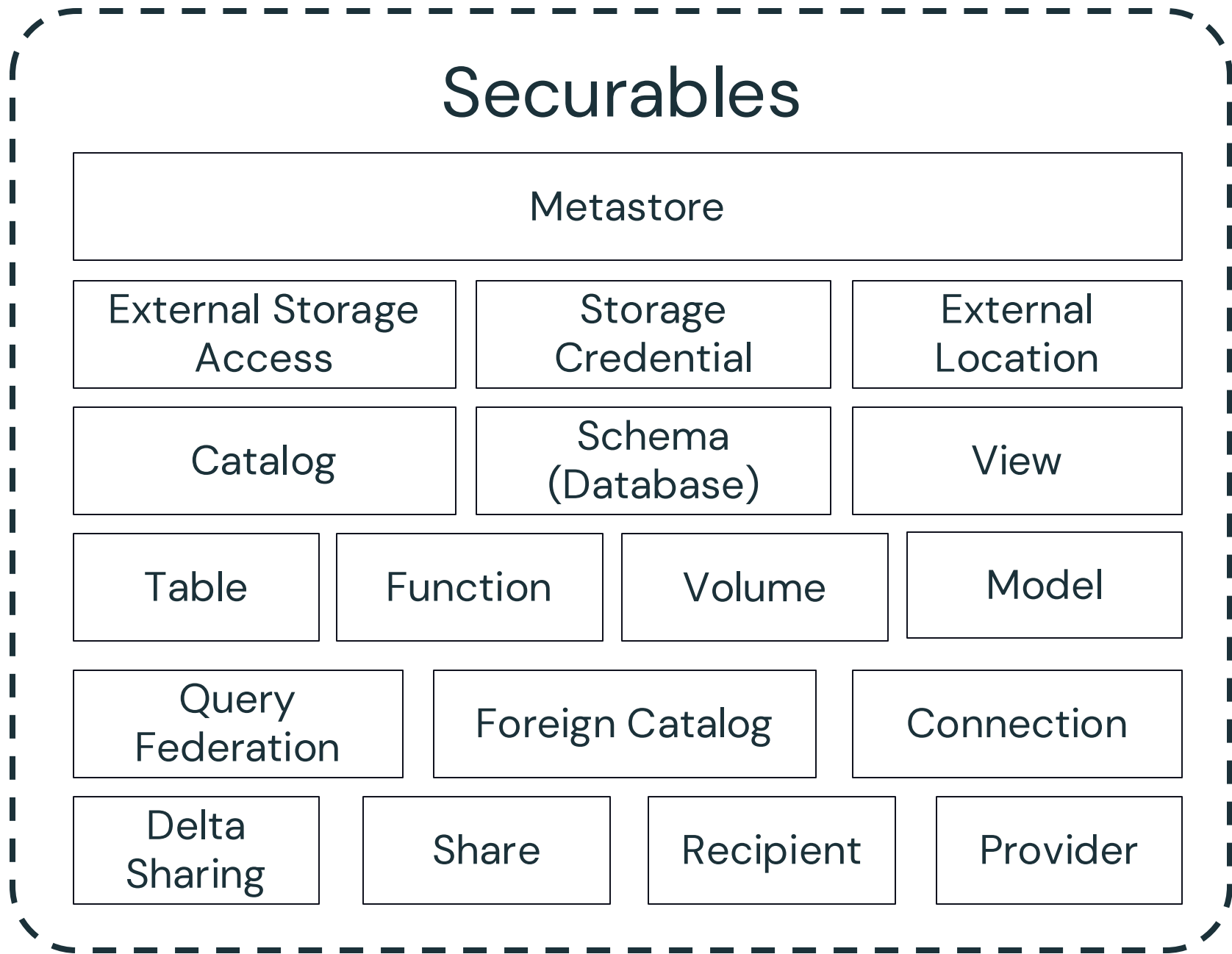
3. Programmatically

Databricks CLI, Terraform, and REST APIs



Privilege Types

Metastore



Privileges

- CREATE CATALOG

CREATE CONNECTION

CREATE EXTERNAL LOCATION

CREATE PROVIDER

CREATE RECIPIENT

CREATE SHARE

CREATE STORAGE CREDENTIAL

READ FILES

WRITE FILES

CREATE EXTERNAL LOCATION

CREATE EXTERNAL TABLE

And more.....
- SET SHARE PERMISSION

USE MARKETPLACE ASSETS

USE PROVIDER

USE RECIPIENT

USE SHARE

USE SCHEMA

CREATE TABLE

CREATE VOLUME

CREATE FUNCTION

CREATE MODEL

CREATE MATERIALIZED VIEW

And more.....



Privilege Scenarios

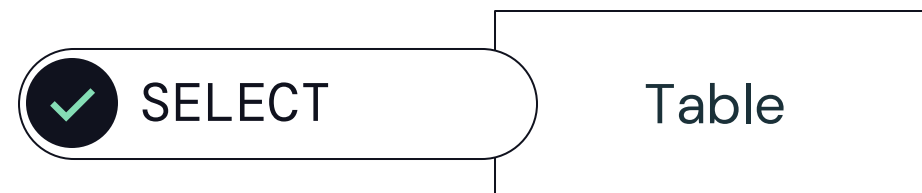
Querying a Table

Table



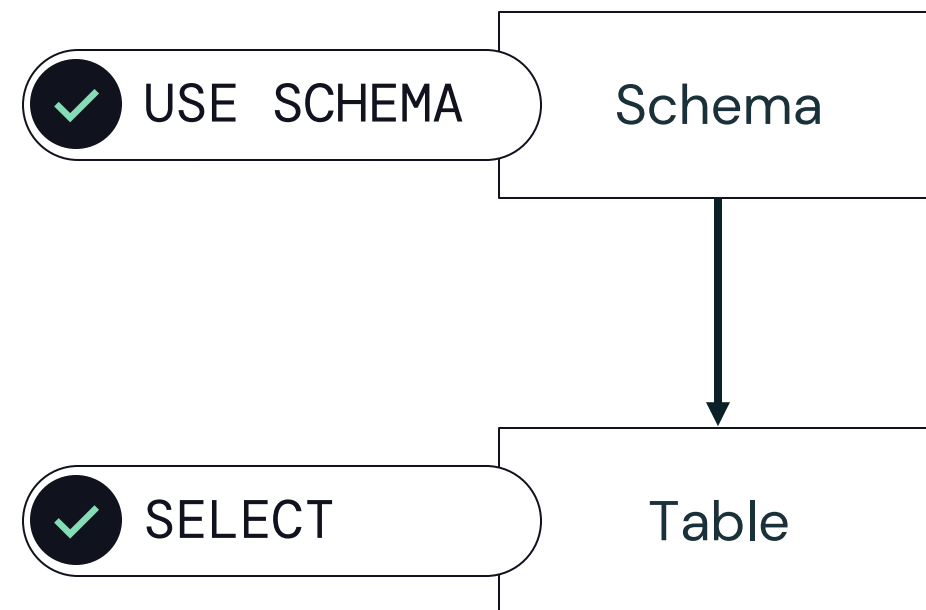
Privilege Scenarios

Querying a Table



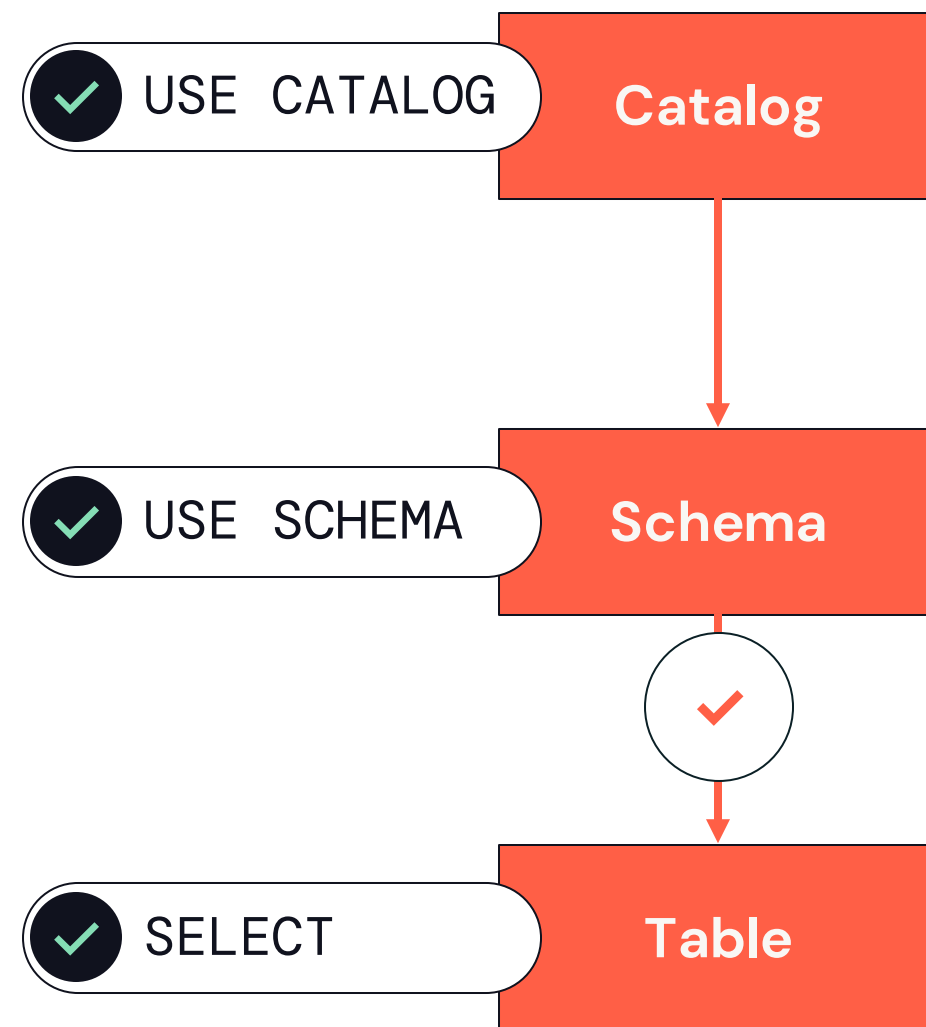
Privilege Scenarios

Querying a Table



Privilege Scenarios

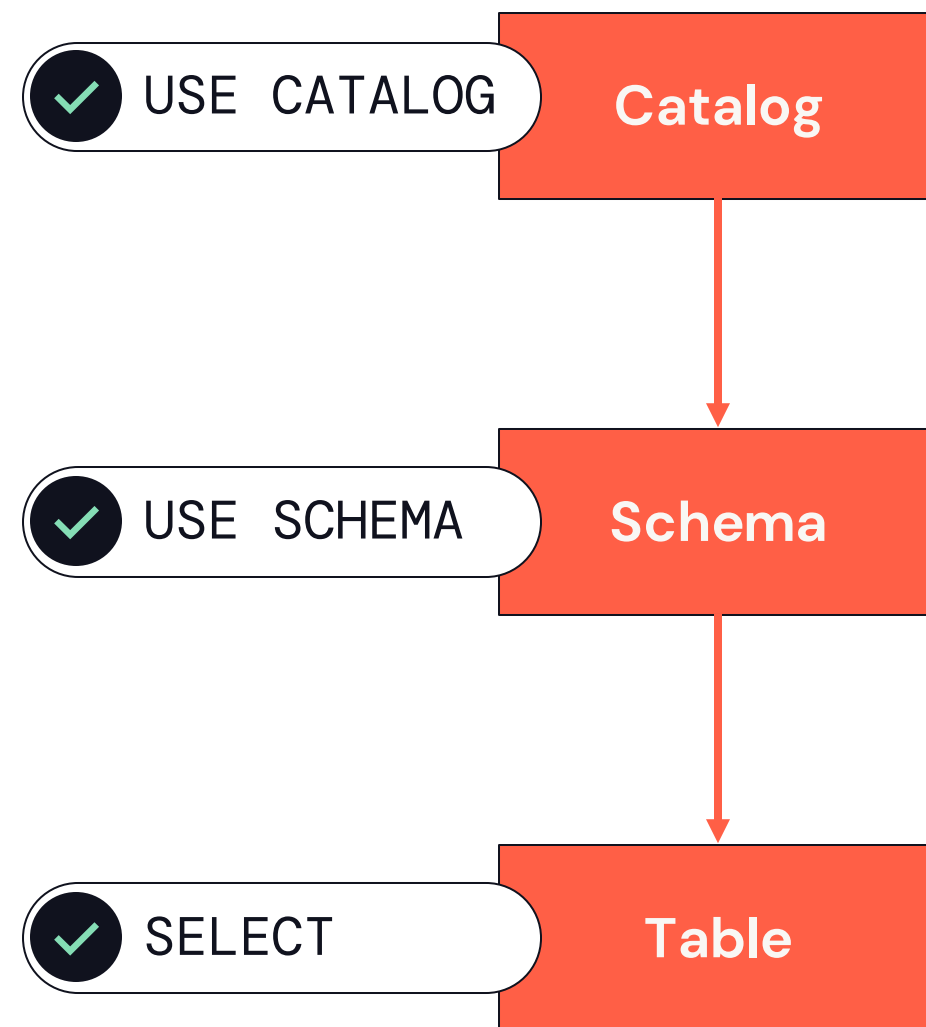
Querying a Table



Privilege Scenarios

Querying a Table

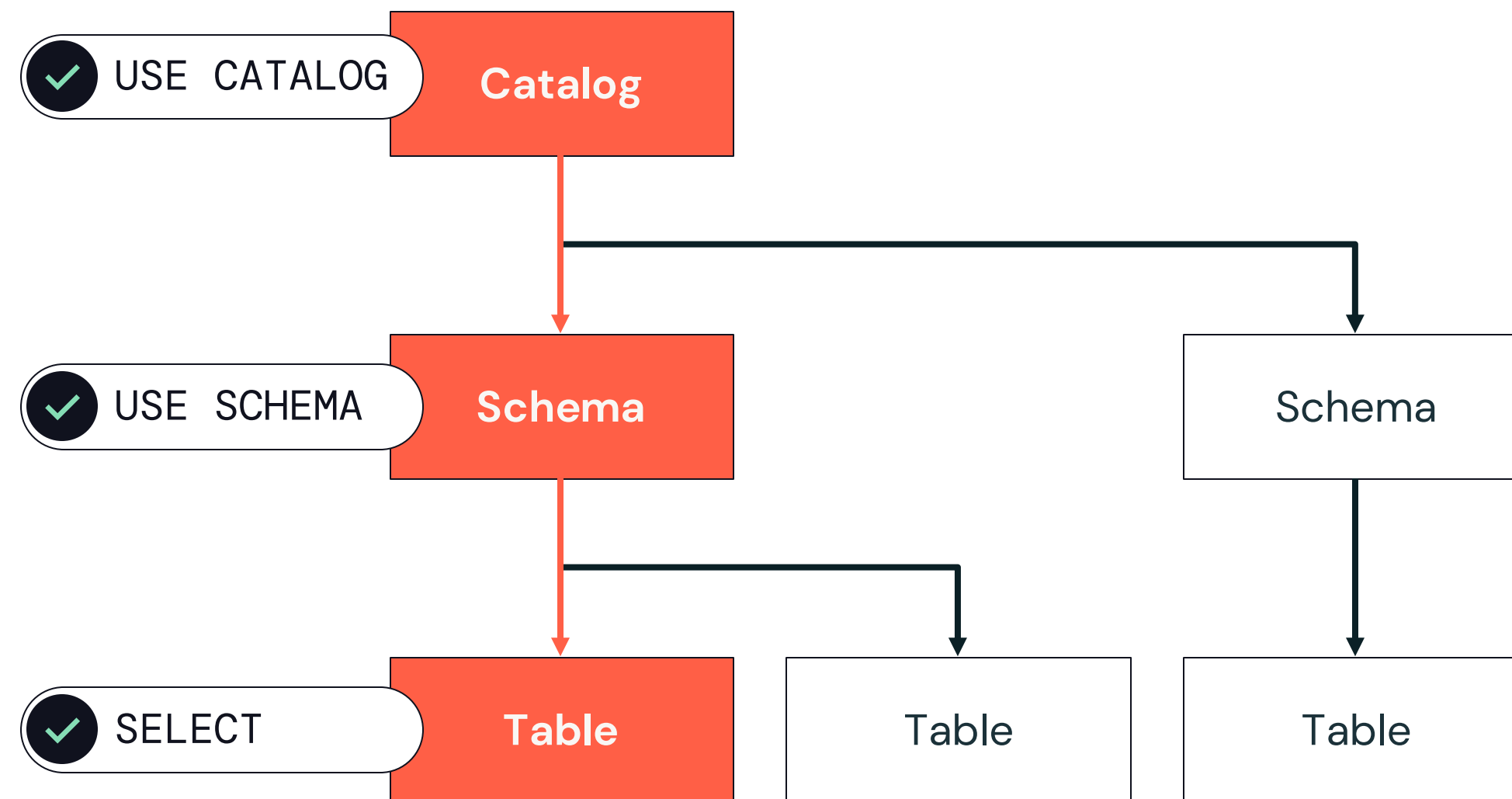
Explicit



Privilege Scenarios

Querying a Table

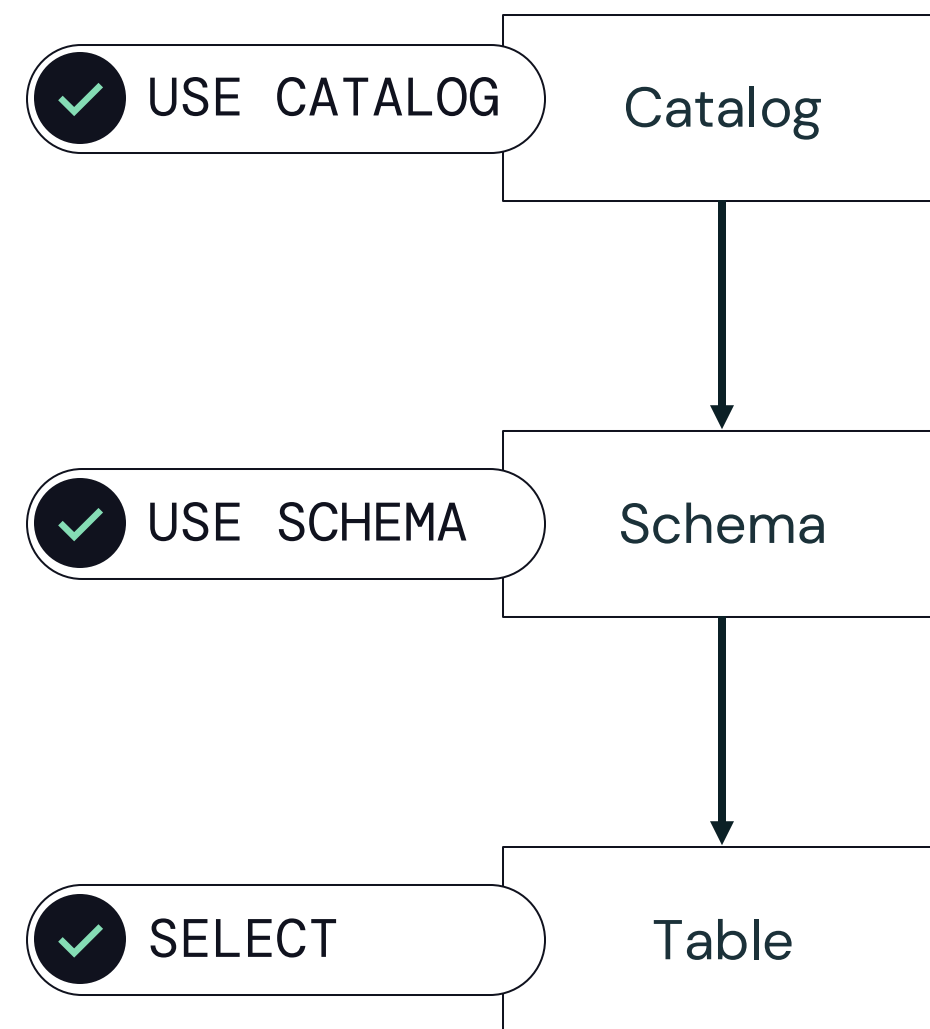
Explicit



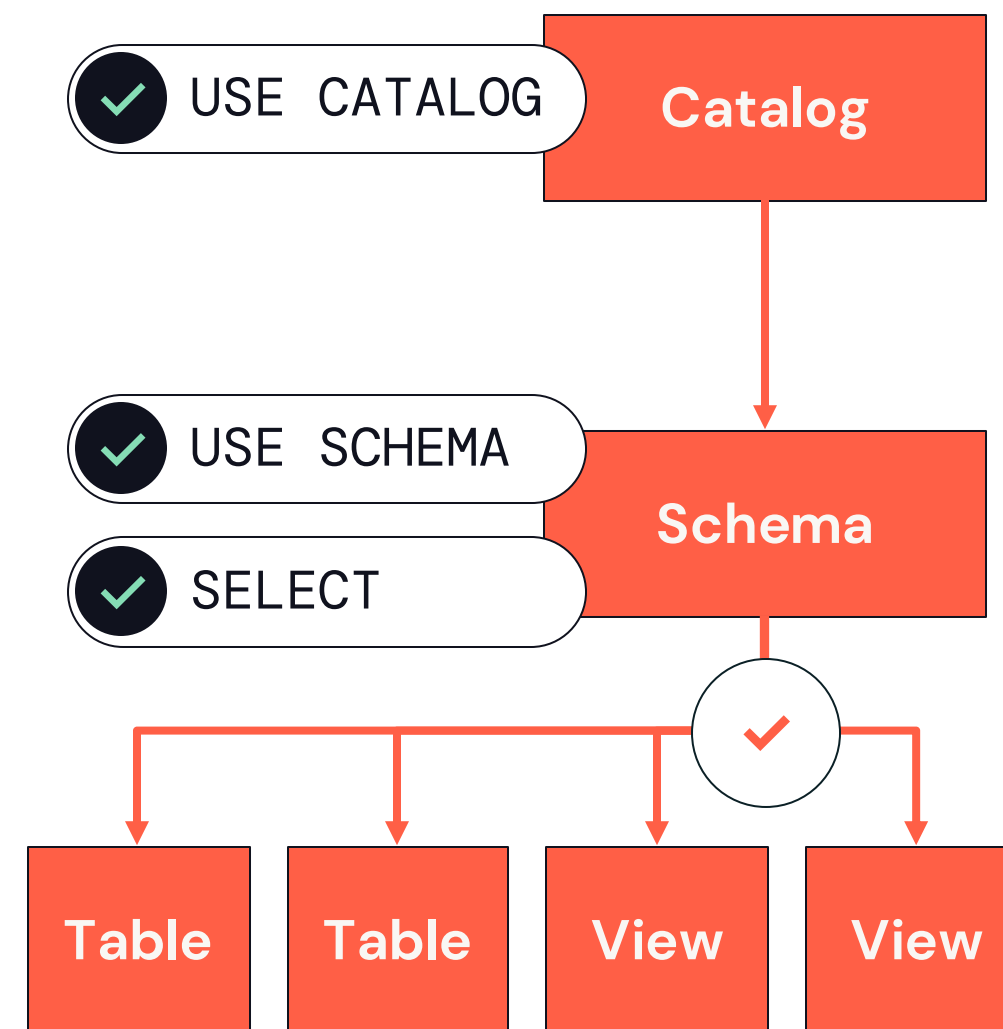
Privilege Scenarios

Querying a Table

Explicit



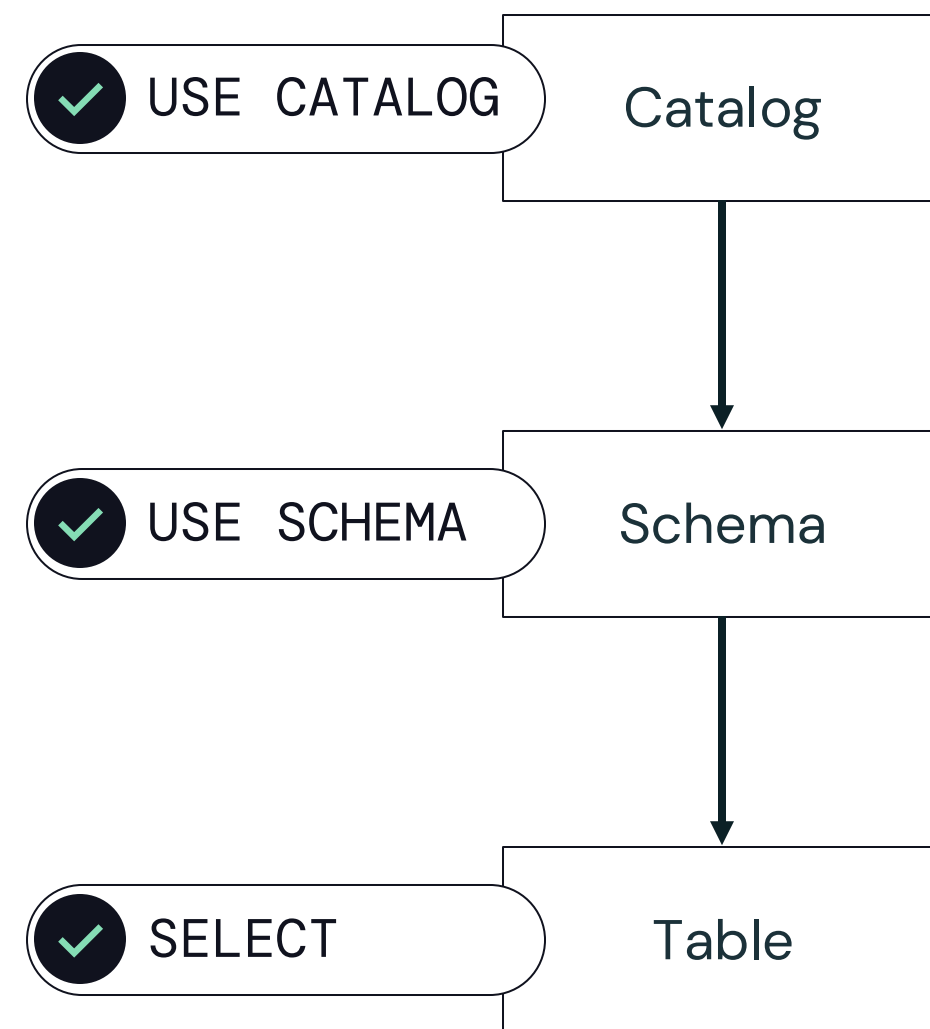
Inherited from schema



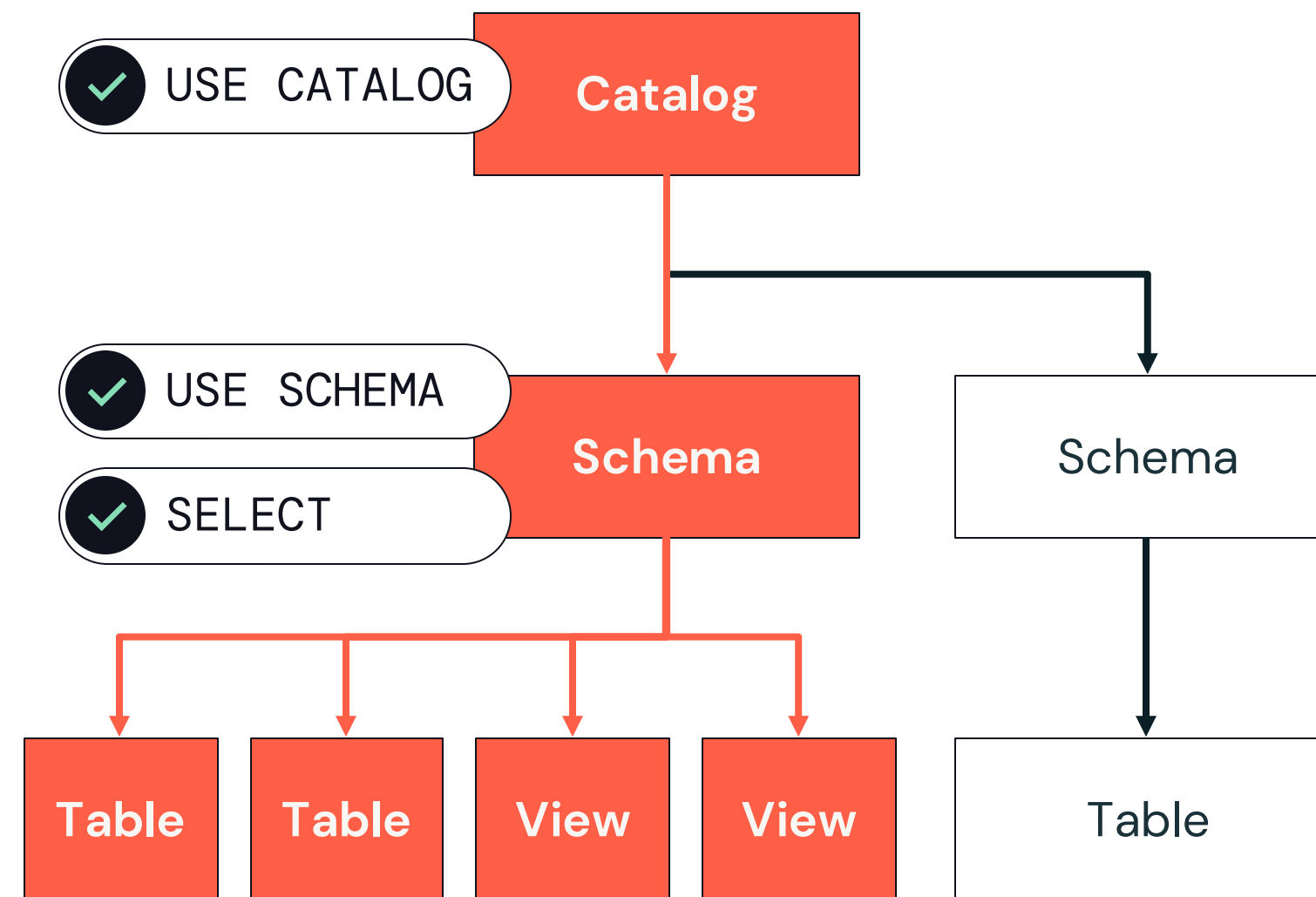
Privilege Scenarios

Querying a Table

Explicit



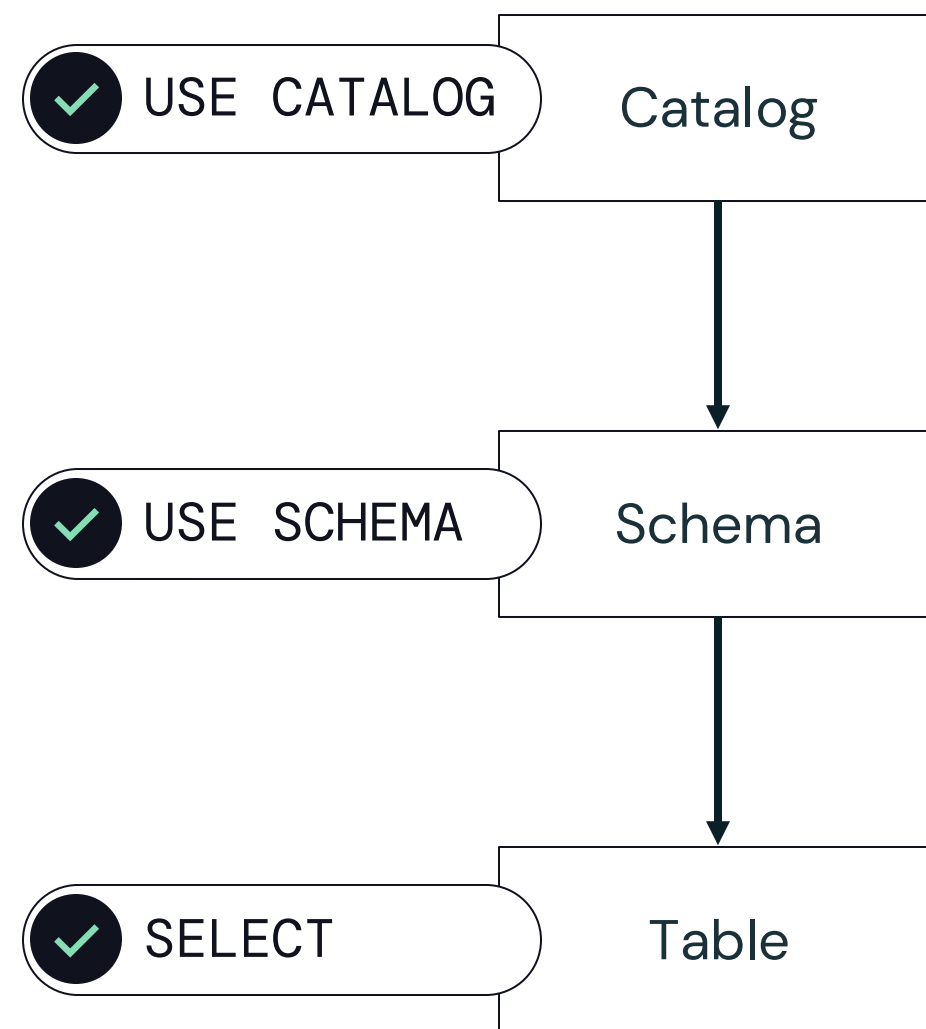
Inherited from schema



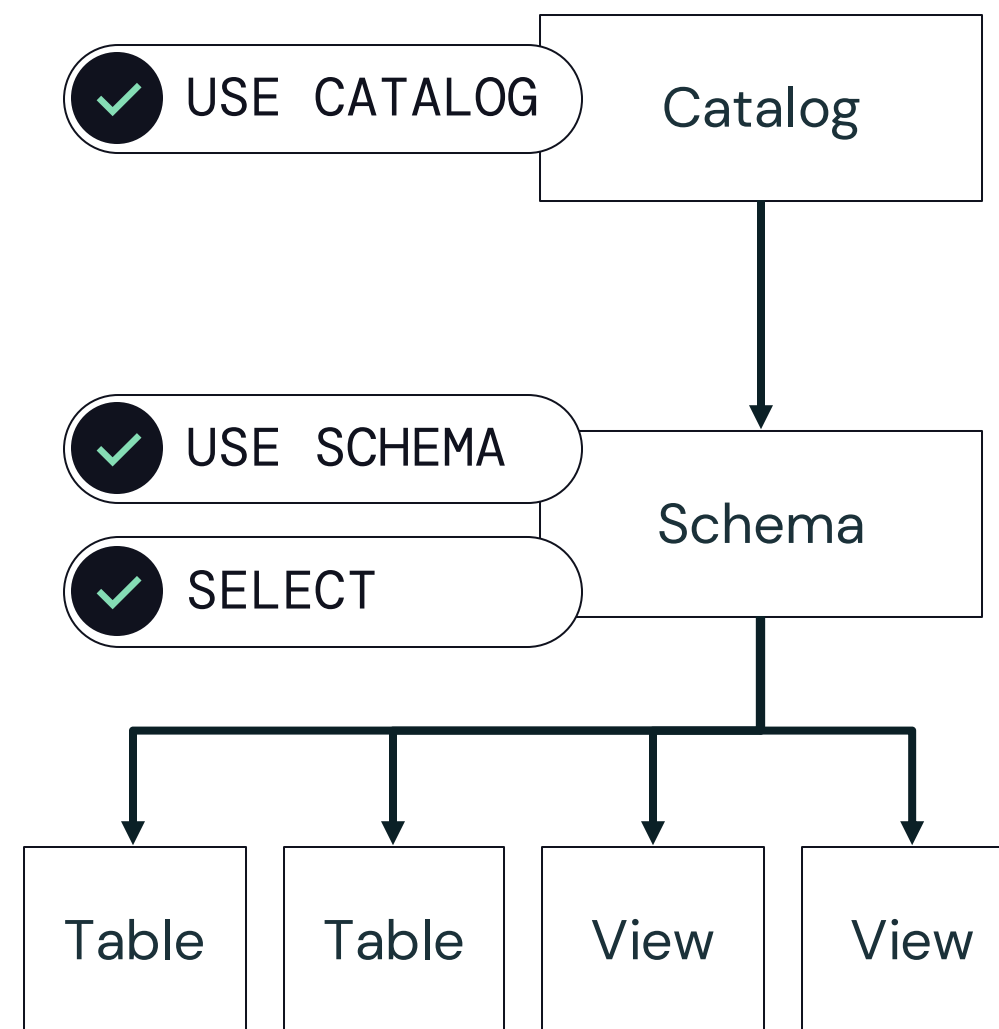
Privilege Scenarios

Querying a Table

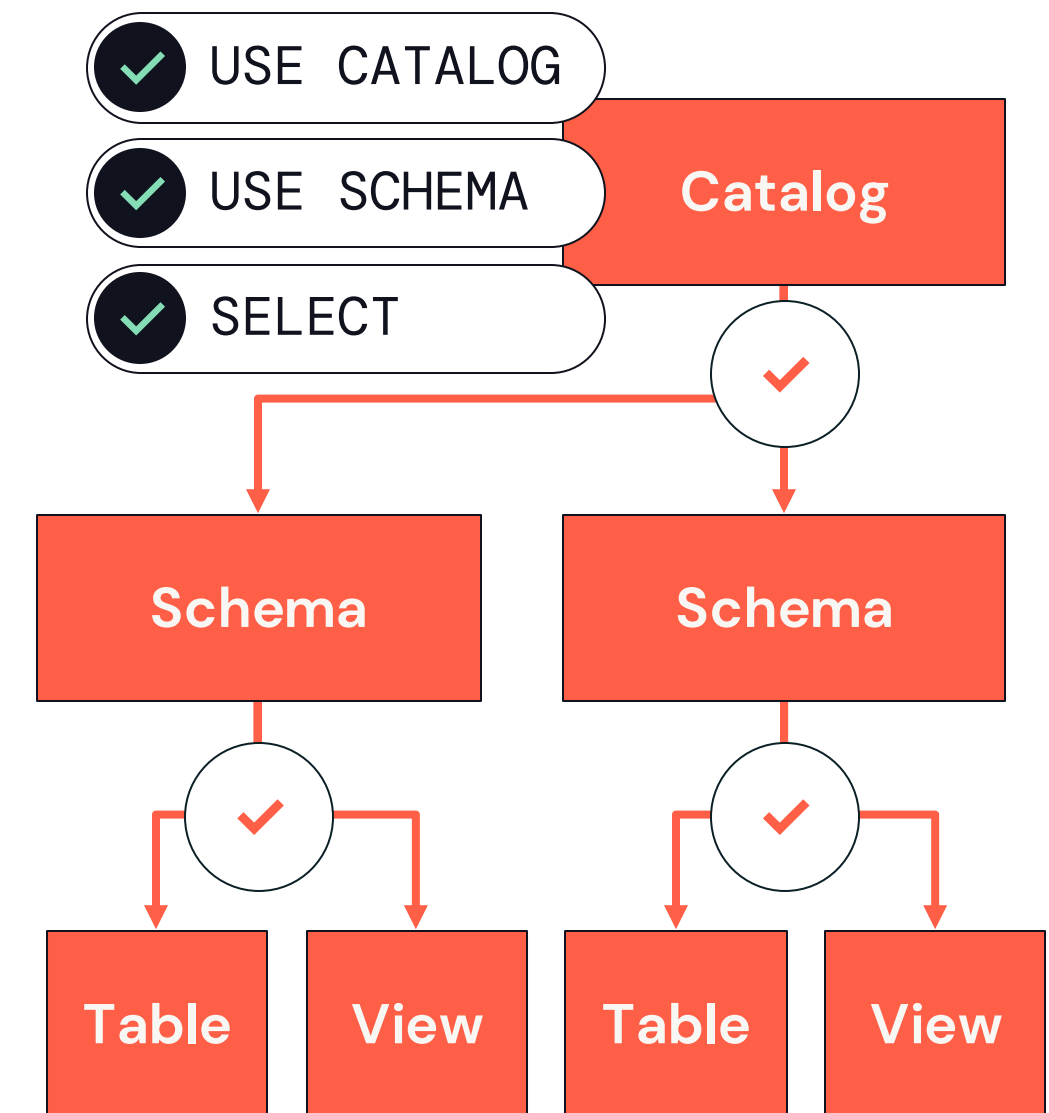
Explicit



Inherited from schema



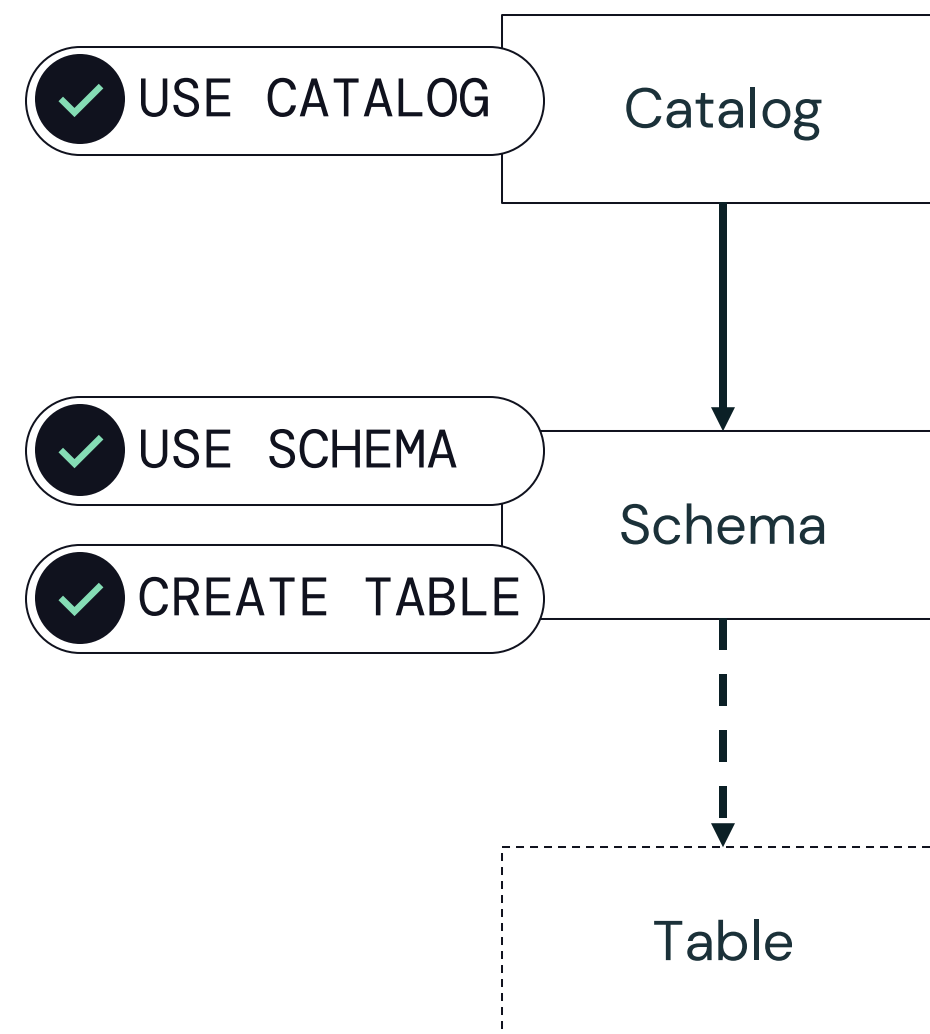
Inherited from catalog



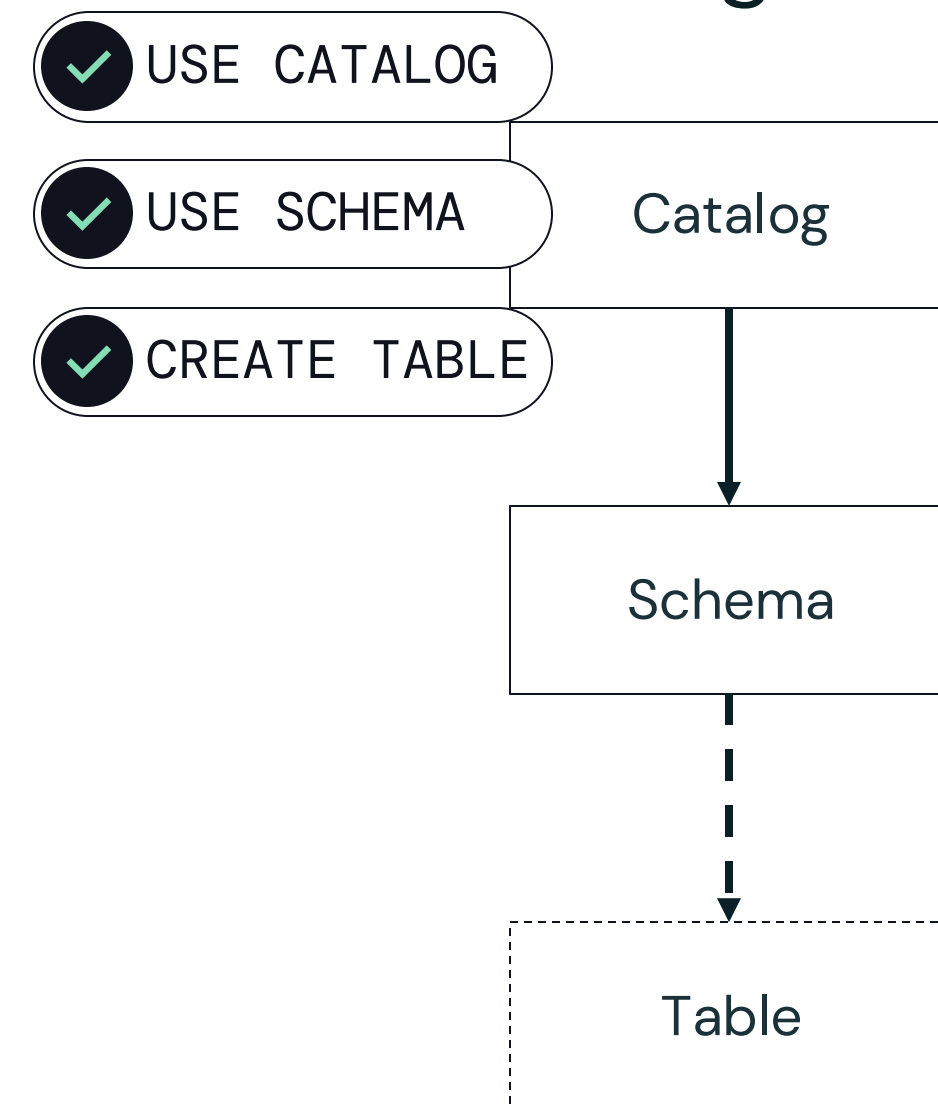
Privilege Scenarios

Creating a Table

Explicit



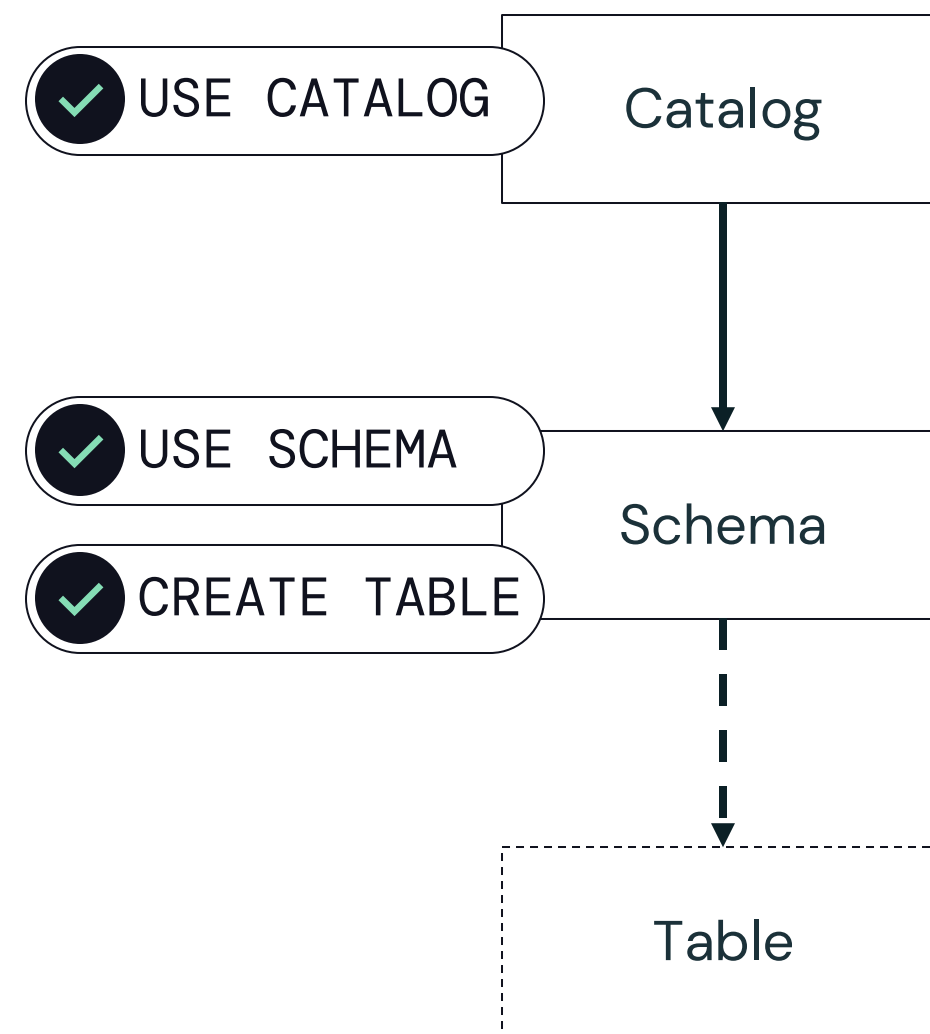
Inherited from catalog



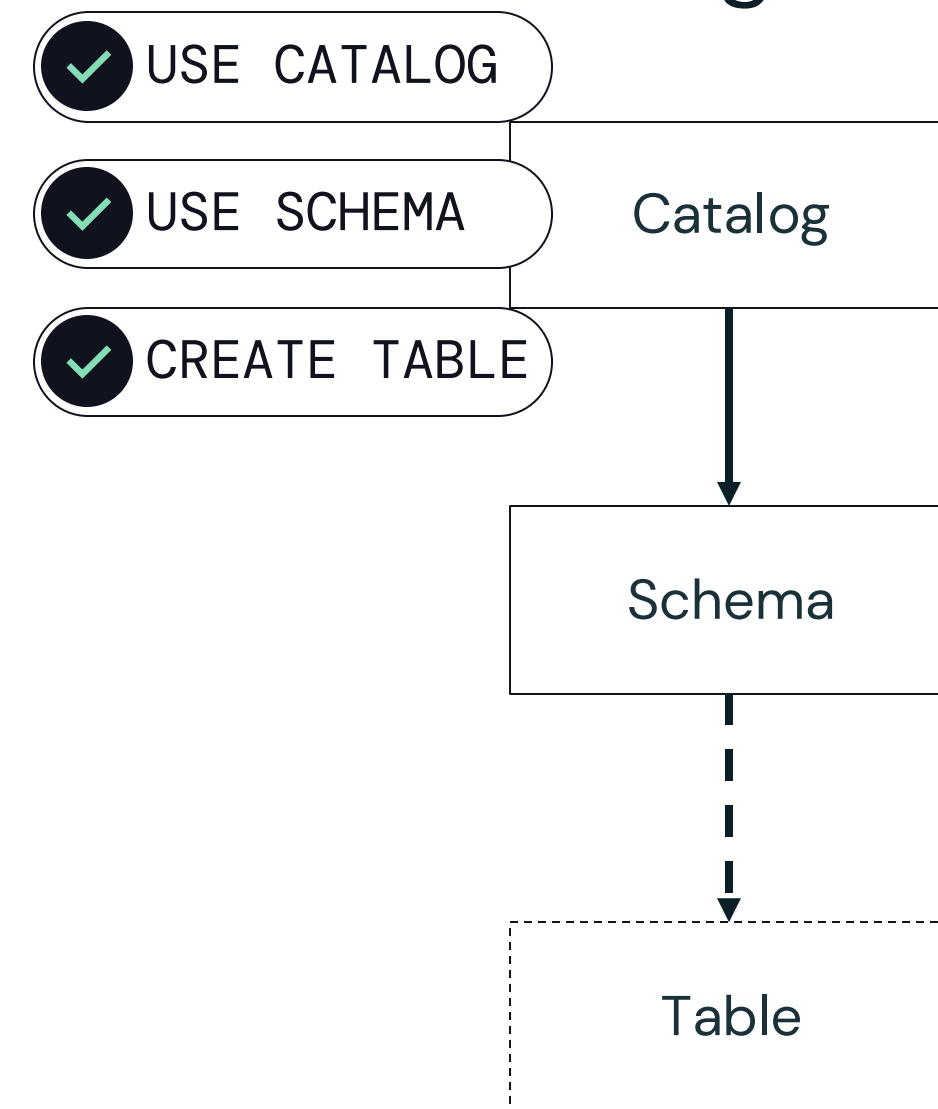
Privilege Scenarios

Creating a Table

Explicit



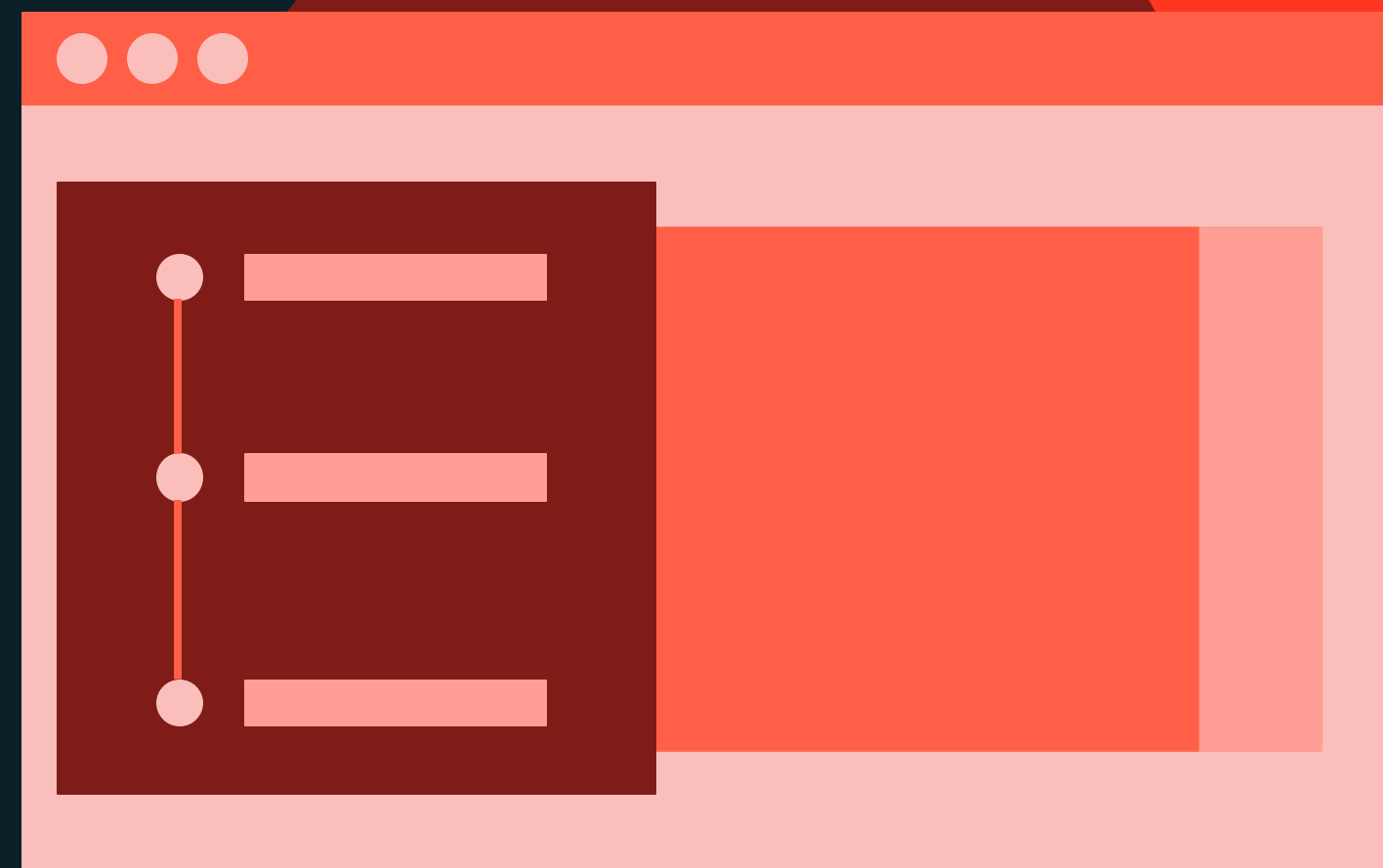
Inherited from catalog





DEMONSTRATION

Controlling Access to Data





Fine-grained Access Control

Data Management and Governance with Unity Catalog

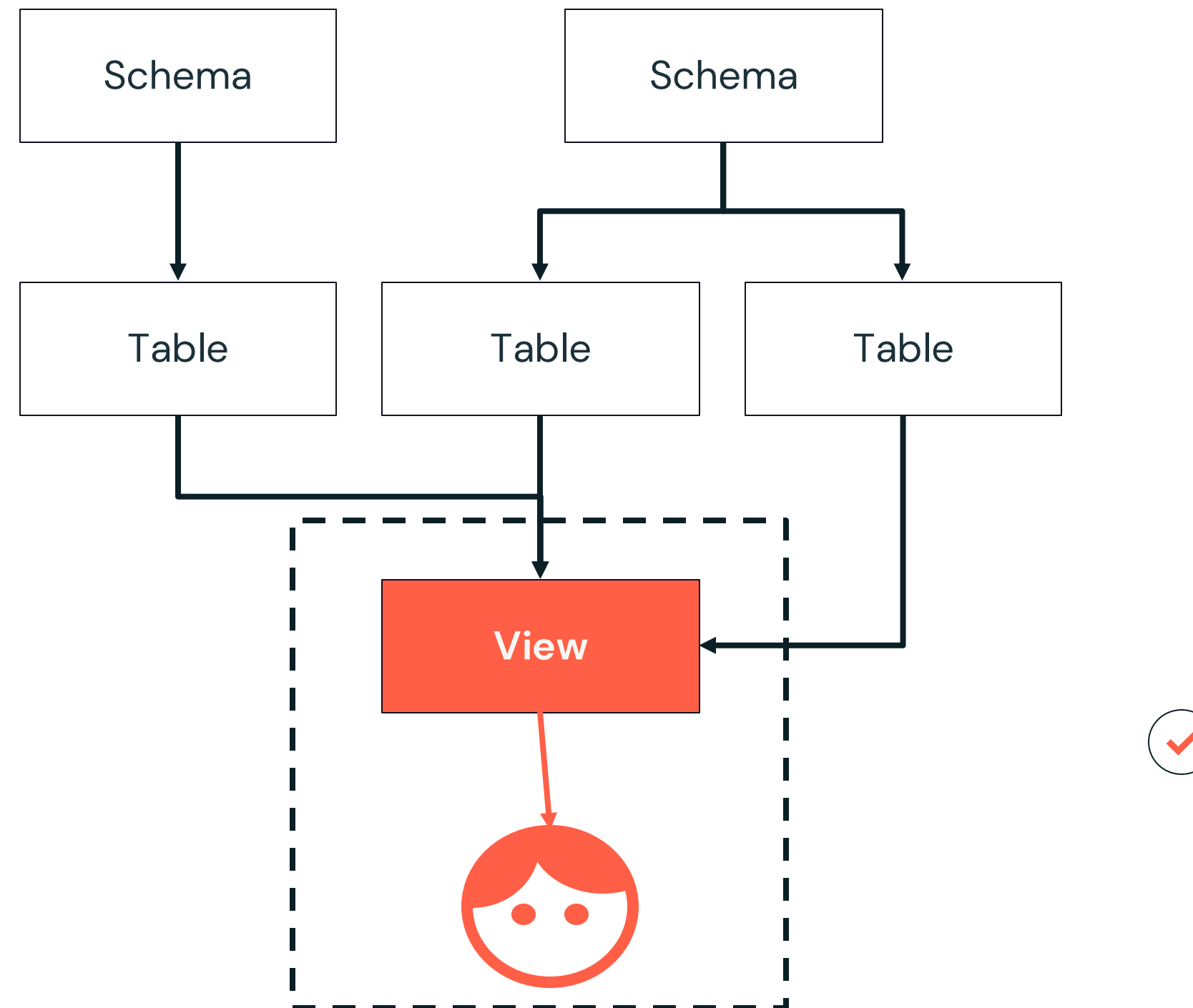


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).



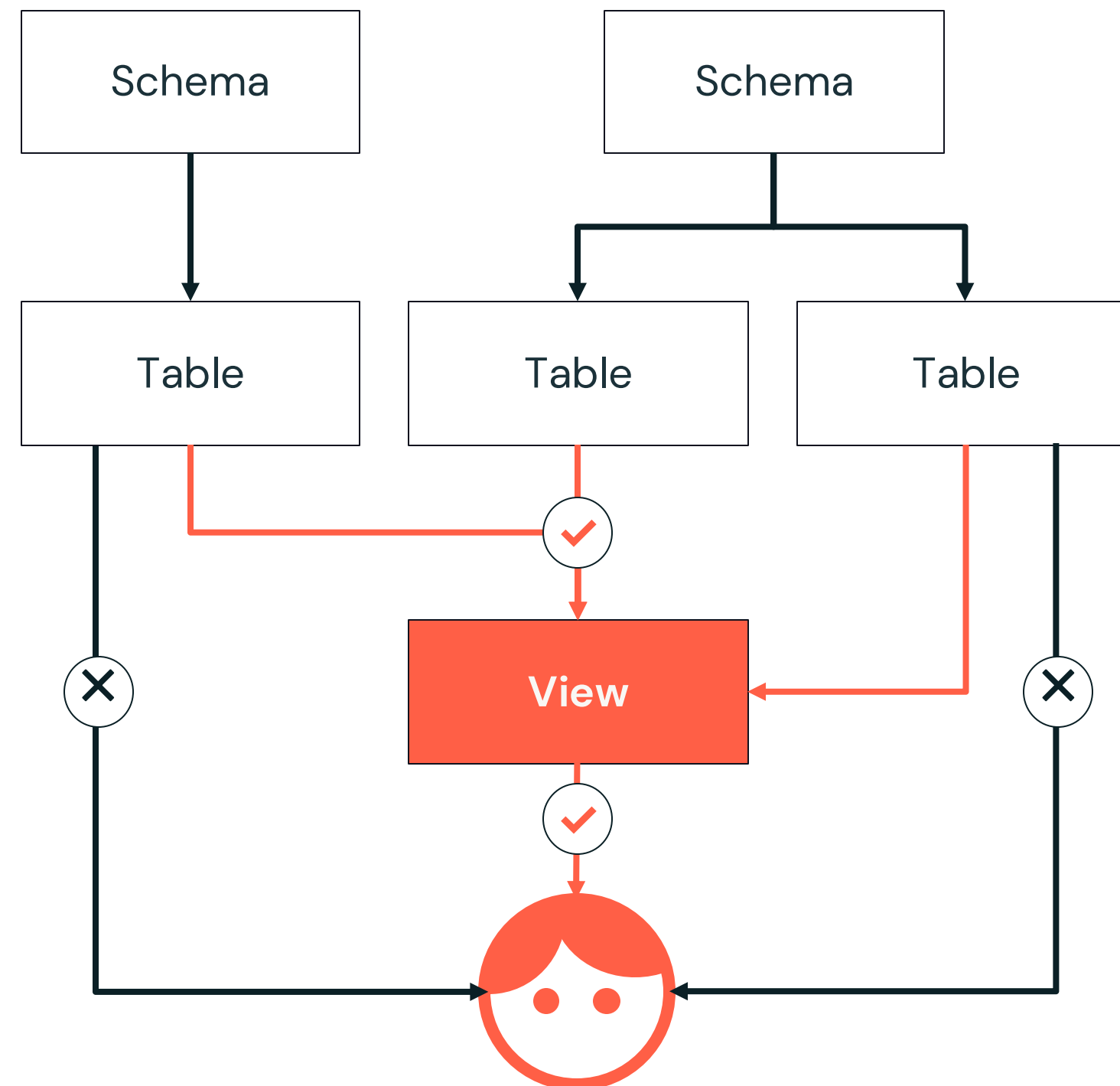
Fine-grained Access Control

Dynamic Views



Fine-grained Access Control

Dynamic Views



Fine-grained Access Control

Common Use Cases

Limit Access to Columns

Omit column values from output

Limit Access to Rows

Omit rows from output

Data Masking

Obscure data

.....@databricks.com

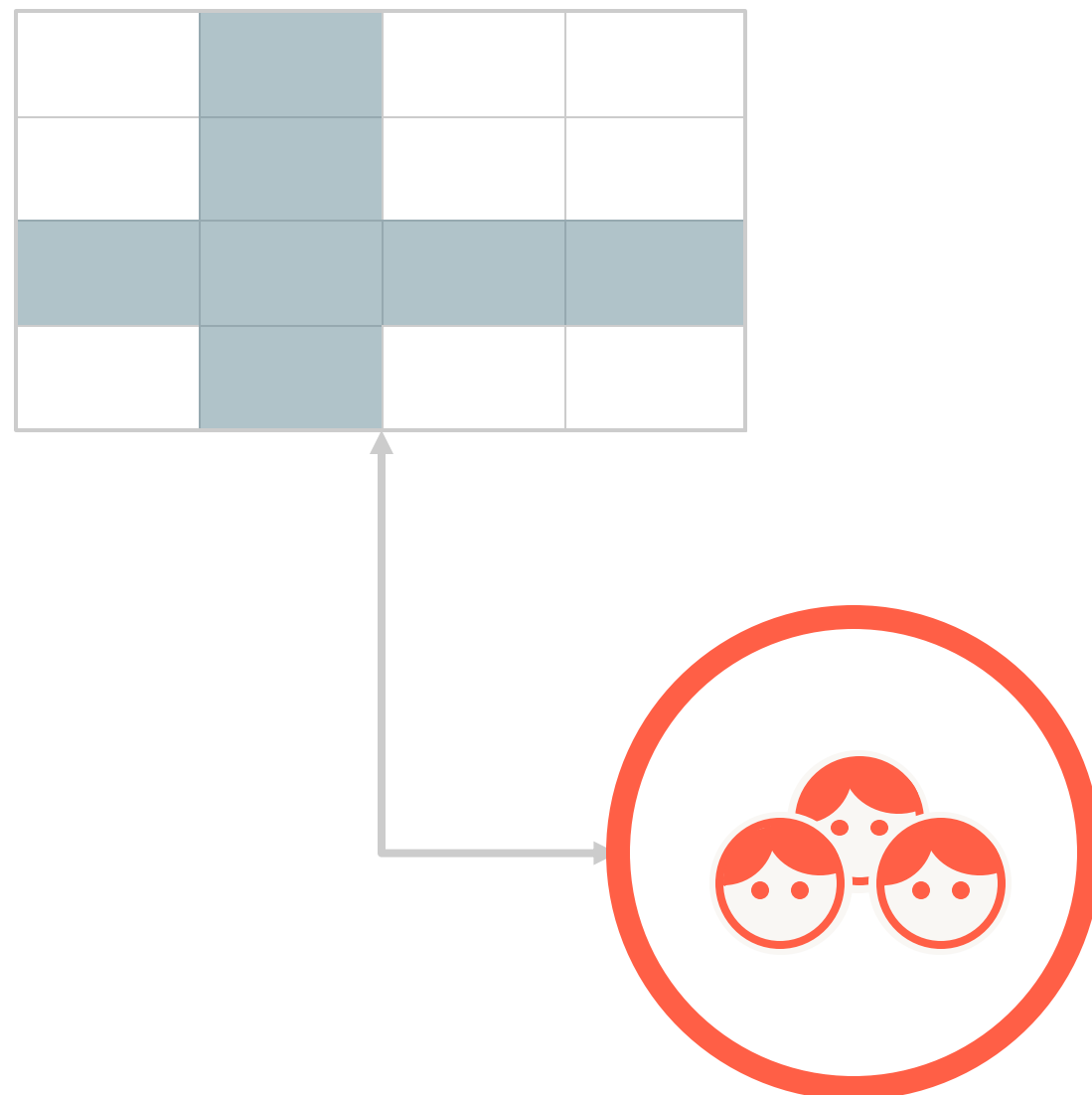
Can be conditional on a specific user/service principal or group membership through Databricks-provided functions



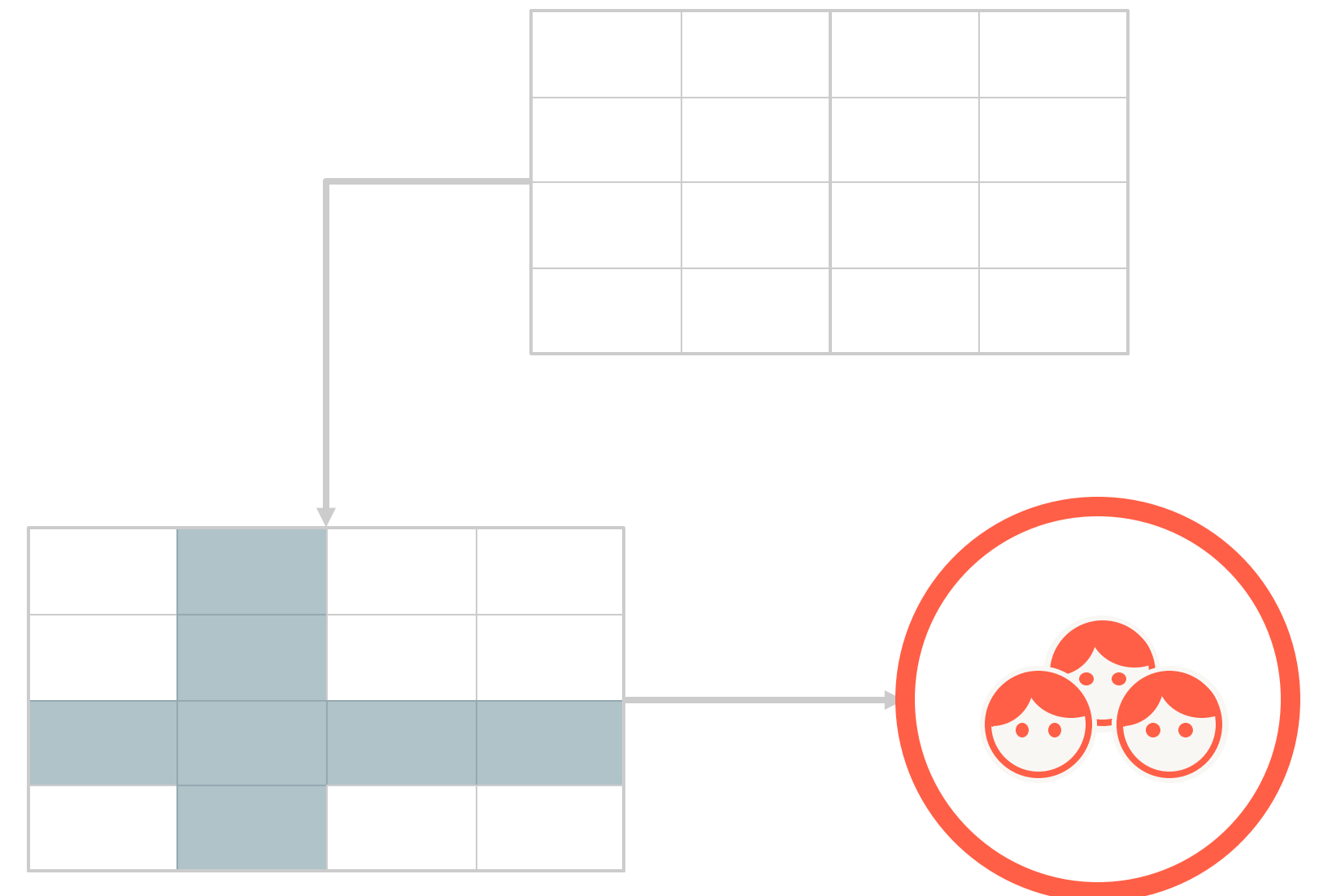
Fine-grained Access Control

Two Approaches

Row Column Security



Dynamic View



Row and Column Security

Overview



employee_id	name	contact_info	country	tin	pay_rate
005041	Barbara...	1472 Brew...	US	123456789	59428.00
005042	Mark...	177 Woodp...	US	987654321	59300.00
005078	Aarna...	2, Sukh...	IN	ABCTY1234D	1060350
005311	Isla...	Drumcairn...	IE	8765432WA	45000



Row and Column Security

Column Masking

employee_id	name	contact_info	country	tin	pay_rate
005041	Barbara...	1472 Brew...	US	123456789	59428.00
005042	Mark...	177 Woodp...	US	987654321	59300.00
005078	Aarna...	2, Sukh...	IN	ABCTY1234D	1060350
005311	Isla...	Drumcairn...	IE	8765432WA	45000

mask_employee_id
(function)

redact_tin
(function)

redact_pay_rate
(function)



Row and Column Security

Row Filtering

employee_id	name	contact_info	country	tin	pay_rate
005041	Barbara...	1472 Brew...	US	123456789	59428.00
005042	Mark...	177 Woodp...	US	987654321	59300.00
005078	Aarna...	2, Sukh...	IN	ABCTY1234D	1060350
005311	Isla...	Drumcairn...	IE	8765432WA	45000

filter_us
(function)



Row Level Security and Column Level Masking

Provide differential fine grained access to file based datasets and foreign tables

Only show specific rows

```
CREATE FUNCTION <name> ( <parameter_name >  
<parameter_type> .. )  
RETURN {filter clause whose output must be a boolean}
```

```
CREATE FUNCTION us_filter(region STRING)  
RETURN IF(IS_MEMBER('admin'), true, region="US");
```

```
ALTER TABLE sales SET ROW FILTER us_filter ON region;
```

Test for group
membership

Assign reusable
filter to table

Specify filter
predicates

- 'admin' group can access all rows in **region** column.
- Other groups only see rows where **region** = 'US'

Row Level Security and Column Level Masking

Provide differential fine grained access to file based datasets and foreign tables

Only show specific rows

```
CREATE FUNCTION <name> ( <parameter_name >  
<parameter_type> .. )  
RETURN {filter clause whose output must be a boolean}  
  
CREATE FUNCTION us_filter(region STRING)  
RETURN IF(IS_MEMBER('admin'), true, region="US");  
  
ALTER TABLE sales SET ROW FILTER us_filter ON region;
```

Mask or redact sensitive columns

```
CREATE FUNCTION <name> (<parameter_name>,  
<parameter_type>, [, <column>...])  
RETURN {expression with the same type as the first  
parameter}  
  
CREATE FUNCTION ssn_mask(ssn STRING)  
RETURN CASE WHEN IS_MEMBER('admin') THEN name  
            ELSE '*****'  
            END;  
  
ALTER TABLE users ALTER COLUMN table_ssn  
SET MASK ssn_mask;
```

Test for group membership Assign reusable mask to column Specify mask or function to mask

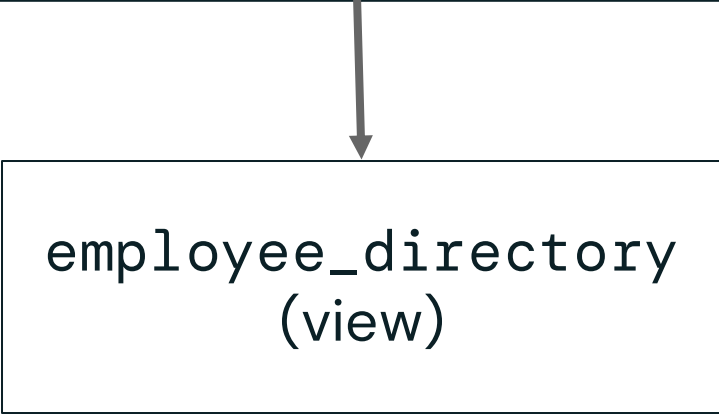
- Members of **'admin'** group can access all column values.
- Other groups see **'*****'** for the **name** column.



Dynamic Views

Column Masking and Row Filtering

employee_id	name	contact_info	country	tin	pay_rate
005041	Barbara...	1472 Brew...	US	123456789	59428.00
005042	Mark...	177 Woodp...	US	987654321	59300.00
005078	Aarna...	2, Sukh...	IN	ABCTY1234D	1060350
005311	Isla...	Drumcairn...	IE	8765432WA	45000



Dynamic Views

Comparison to Row and Column Security

What are the downsides?

- Need to manage multiple objects
- More difficult to maintain
- Views are read-only

When to use them?

- Read-only tables
- Need to abstract source table
- Need to present information from more than one table





LAB EXERCISE

Securing Data in Unity Catalog





Lakehouse Monitoring

**Data Management and Governance with Unity
Catalog**



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).



Lakehouse Monitoring

Unified monitoring for reliable, insightful, and simple data-to-AI pipelines



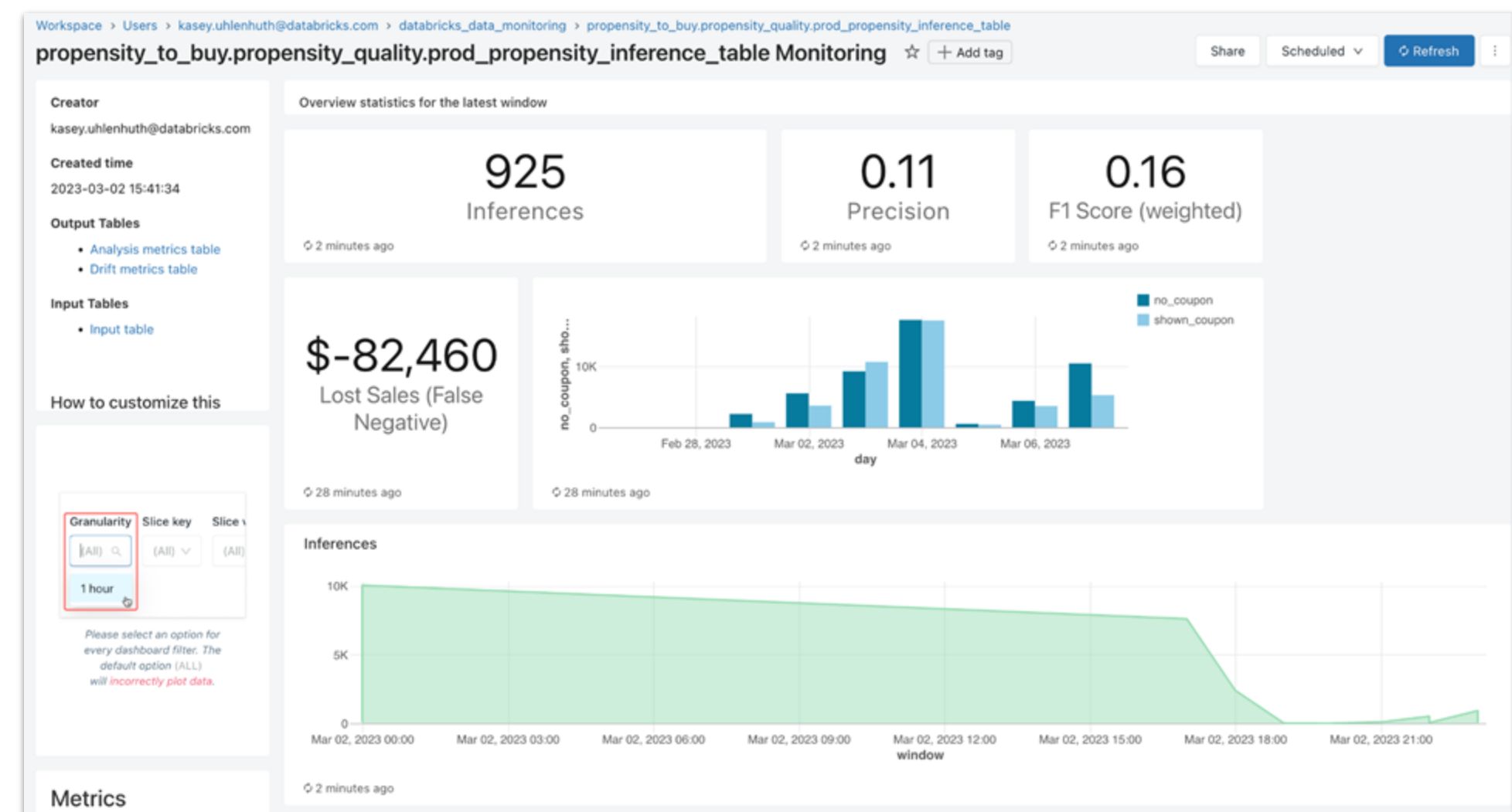
Simple: Log inference tables automatically, and generate metric tables and SQL dashboards.



Proactive: Automate alerts on table quality and custom metrics, and diagnose data or model issues.



Integrated: Track end-to-end lineage in the Unity Catalog for training data, feature tables, models, and inference logs, for simpler governance.





DEMONSTRATION

(Bonus) Lakehouse Monitoring

